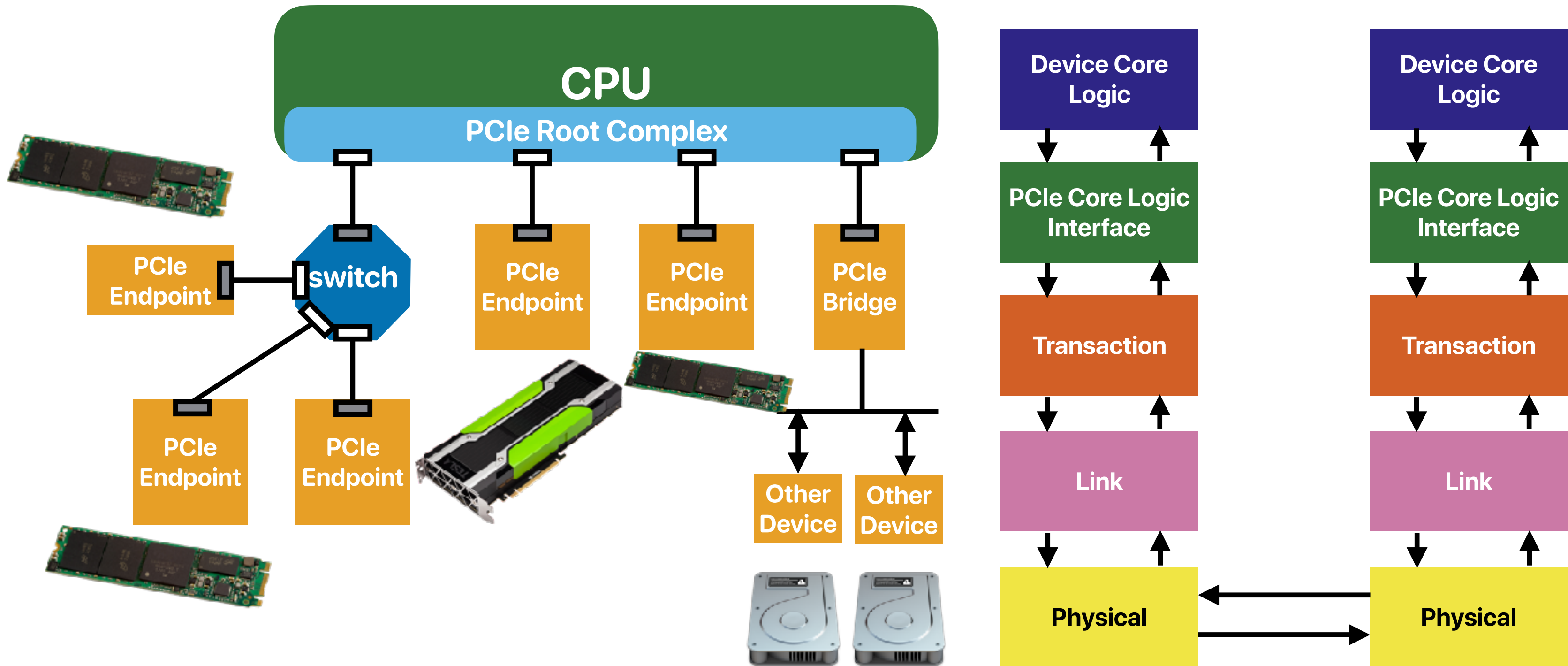


Non-volatile Storage & Streamline Data Exchanges

Hung-Wei Tseng

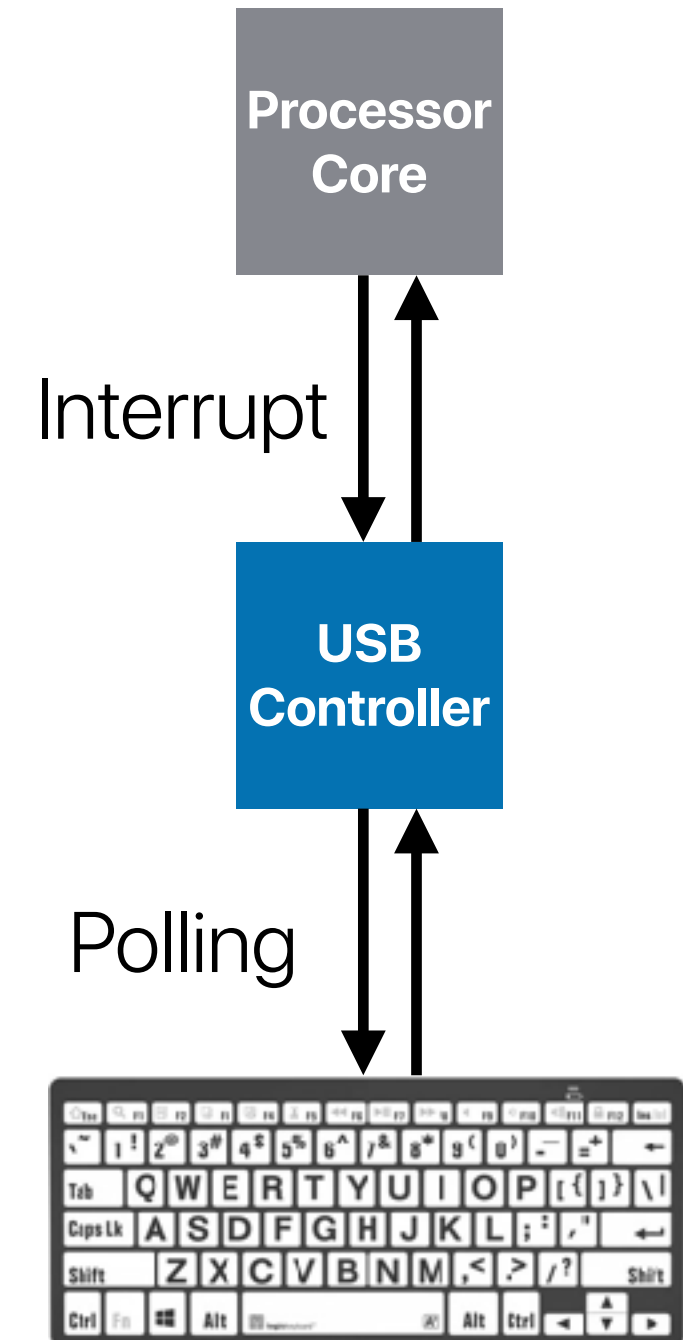
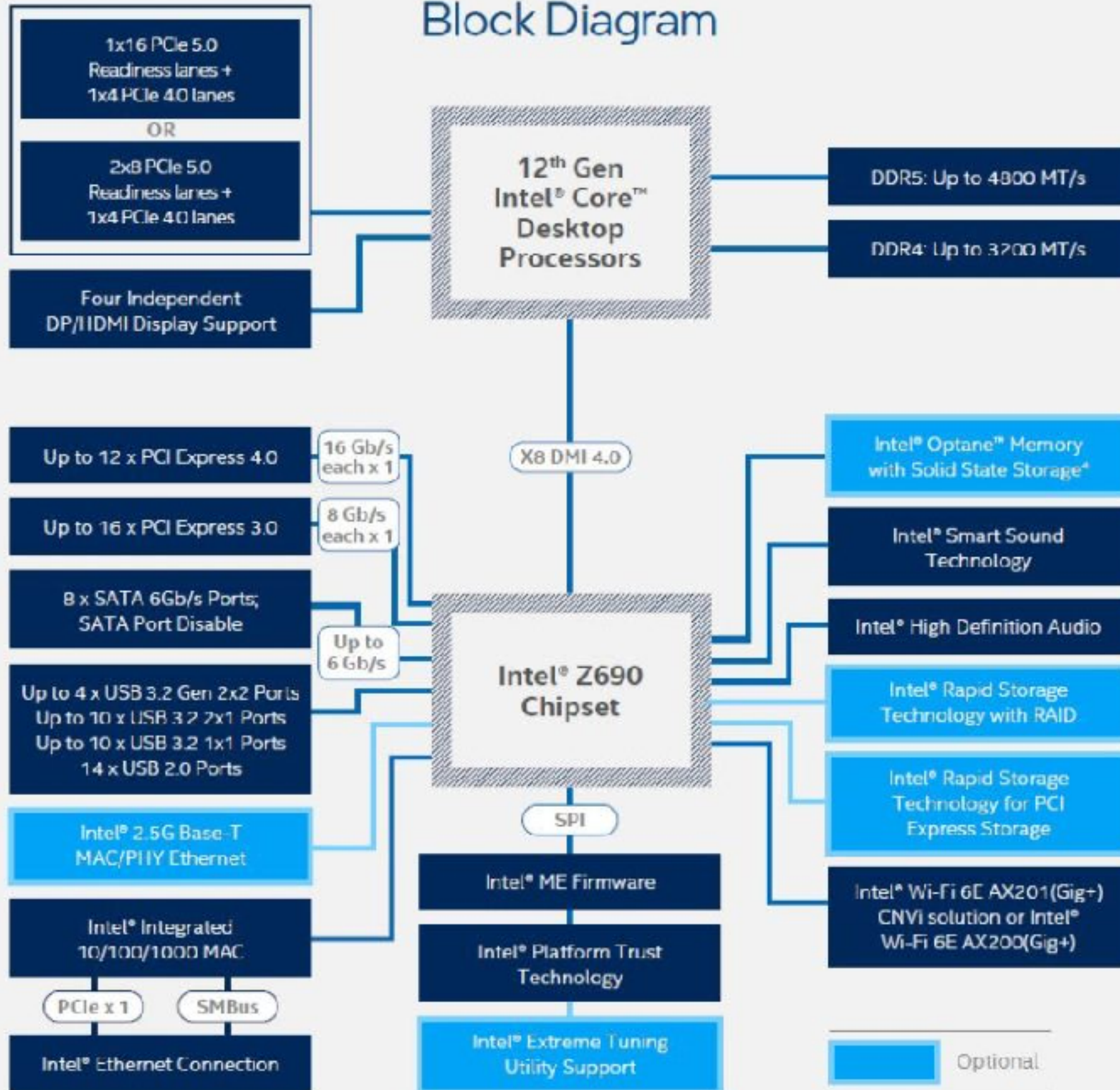
Recap: PCIe "Interconnect"



Polling/Interrupt/DMA/PIO?

	Polling	Interrupt
DMA	CPU OPs more than IRQ Shorter latency High throughput	Lowest CPU Operations Longer latency High throughput
PIO	Most CPU operations Shorter latency Low throughput	CPU OPs more than IRQ Longer latency Low throughput

Intel® Z690 Chipset Block Diagram

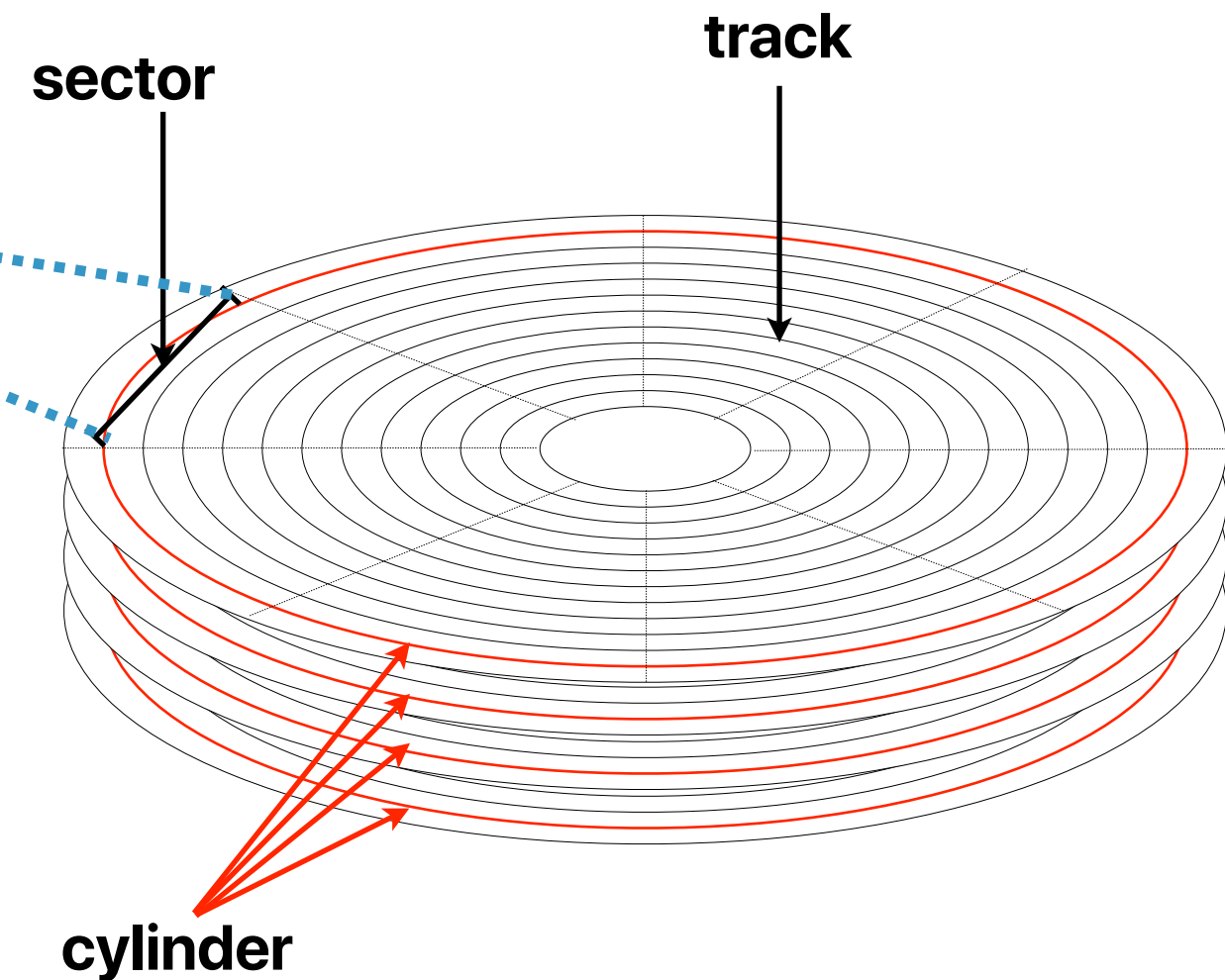


Recap: why block addresses

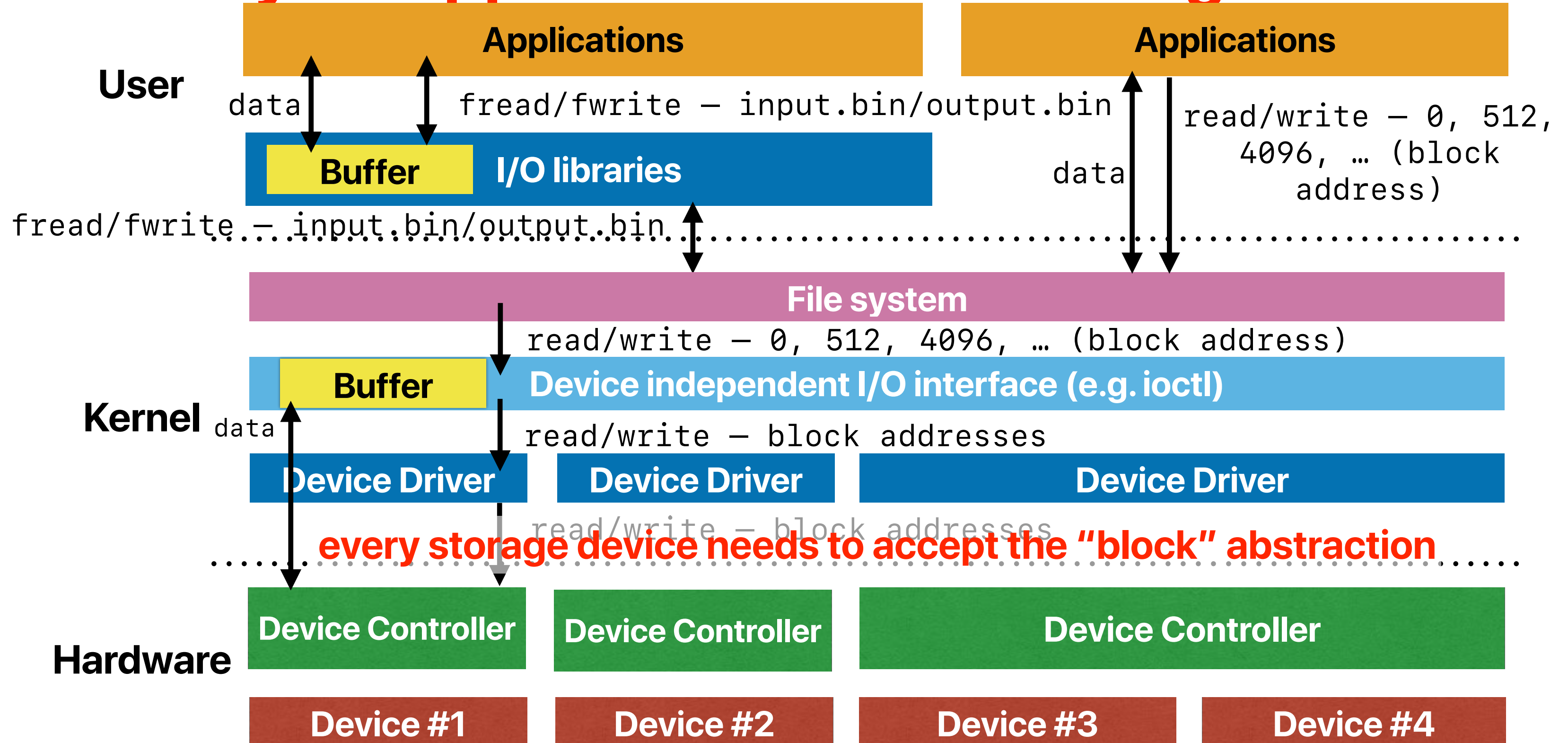
Disk blocks

0								7
8								15
16								23
24								31
32								39
40								47
48								55
56								63

...



How your application reaches storage devices



Recap: NAND Flash v.s. hard disks

		NAND	Hard Disks
granularities	read	page	sector
	write	page	sector
	erase	block (a group of pages)	sector
performance	read	us	ms
	write	100s of us — ms	ms
	erase	ms	ms
program		1,000—100,000 times	unlimited

Recap: NAND flash is just odd!

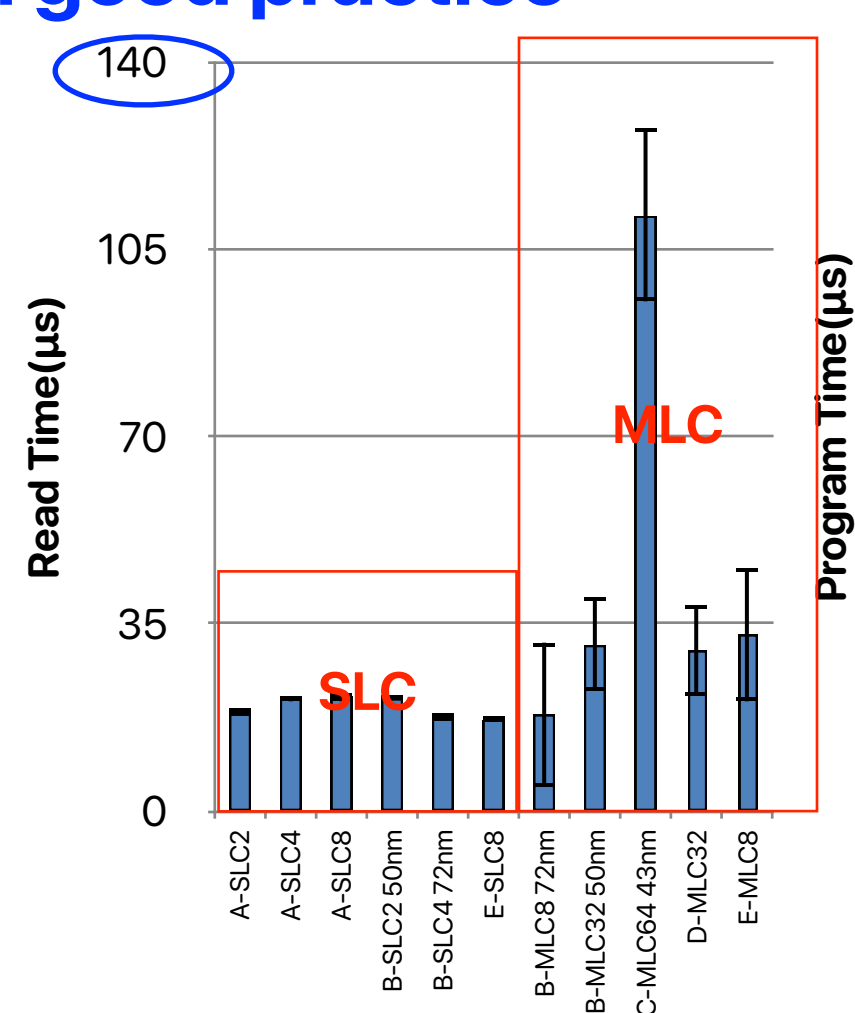
- Modern SSDs are based on NAND flash memory
- Different operation granularities
 - Read/Program in pages
 - Erase in blocks (64-384 pages)
- Performance of operation varies
 - Read — tens of us
 - Program — hundreds of us
 - Erase — ms
- Limited erase cycles
 - Only 1000 times for "QLC"

Outline

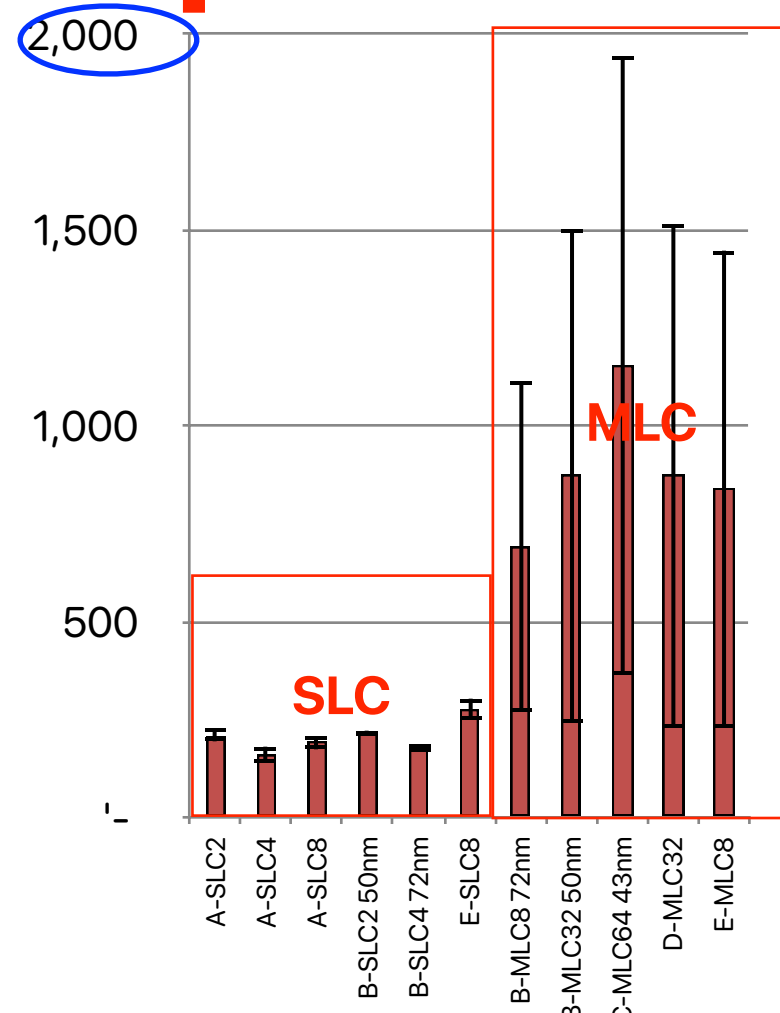
- NVMe (Non-volatile memory express)
- Efficient communication on PCIe
- Near-storage processing
 - SmartSSDs
 - ActiveDisks

Not a good practice

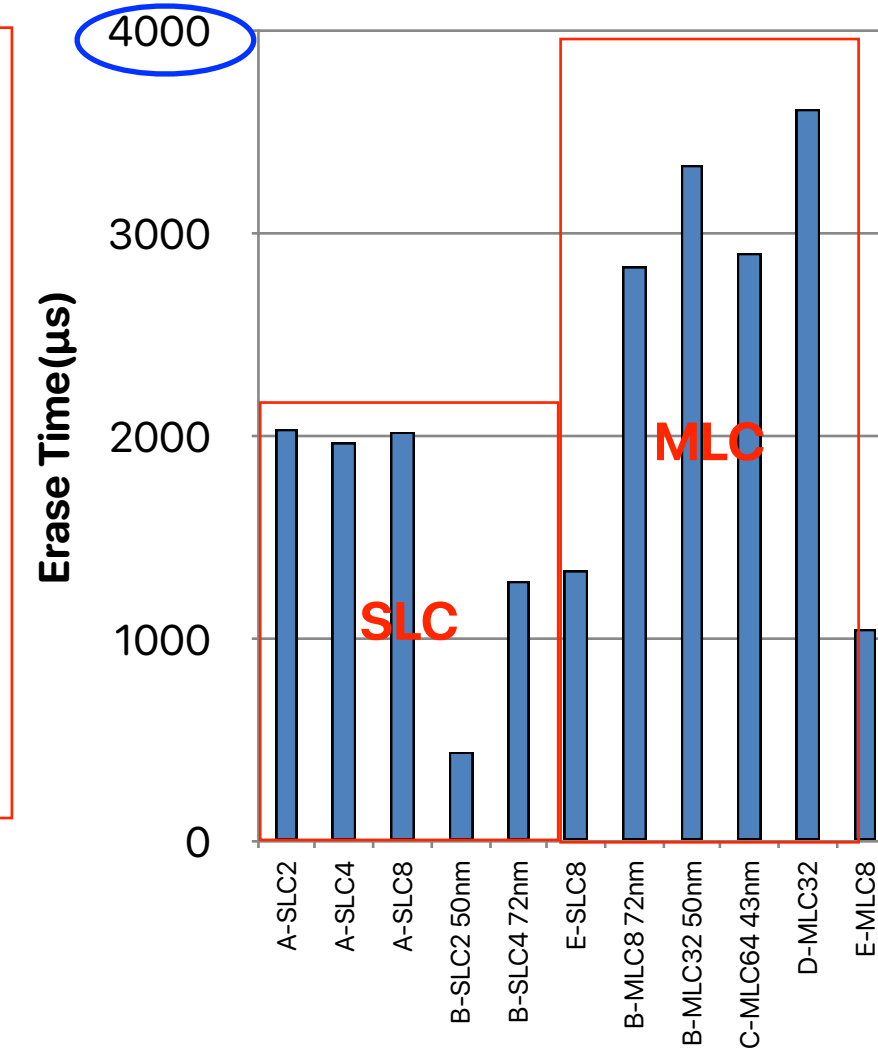
Flash performance



Reads:
less than 150us



Program/write:
less than 2ms

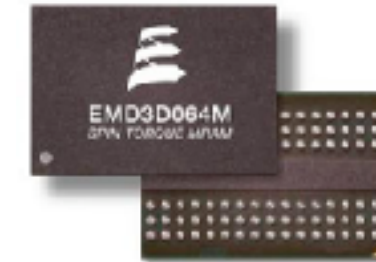


Erase:
less than 3.6ms

Similar relative performance for reads, writes and erases

Laura M. Grupp, Adrian M. Caulfield, Joel Coburn, Steven Swanson, Eitan Yaakobi, Paul H. Siegel, and Jack K. Wolf.
Characterizing flash memory: anomalies, observations, and applications. In MICRO 2009.

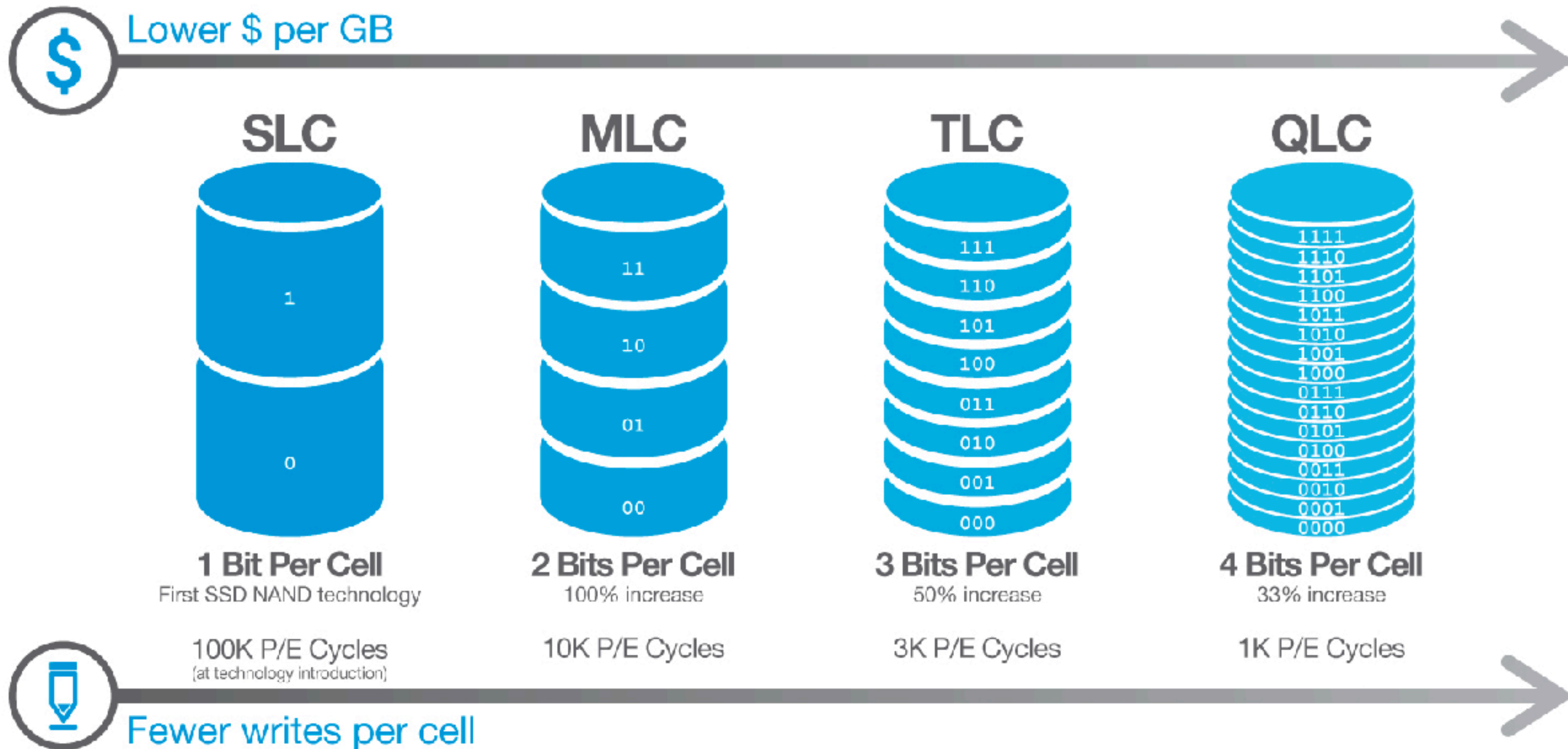
Non-volatile memory technologies



	H.D.D	Flash	Optane	STT-MRAM
Latency	~ 10-15 ms	~ 100 us (read) ~ 1 ms (write)	7 us (read) 18 us (write)	35 ns
Bandwidth	~200 MB/Sec	3.5 GB/sec (read) 2.1 GB/sec (write)	1.35 GB/sec (read) 290 MB/sec (write)	
Dollar/GB	0.0295	0.583	2.18	

Flash is still the most convincing technology for now

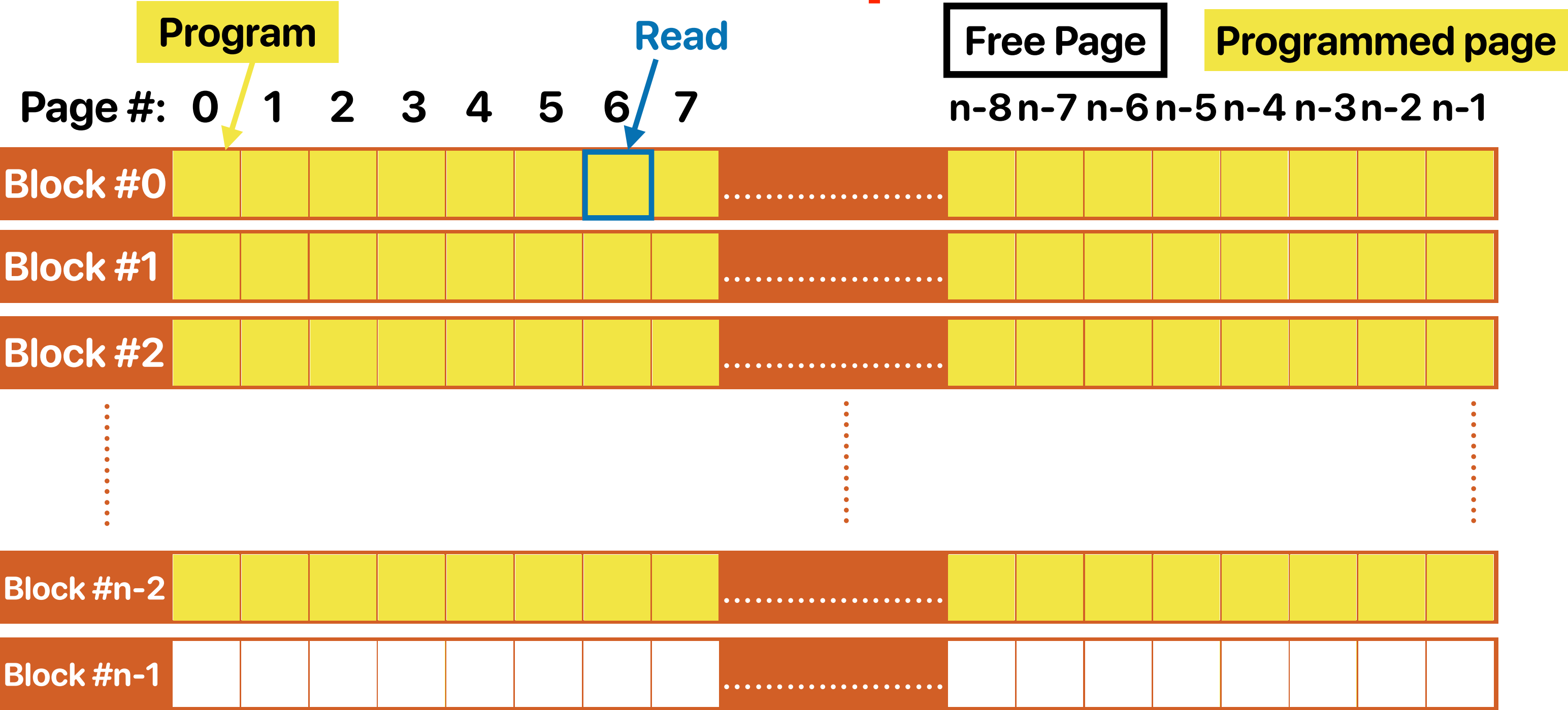
QLC = More Density Per NAND Cell



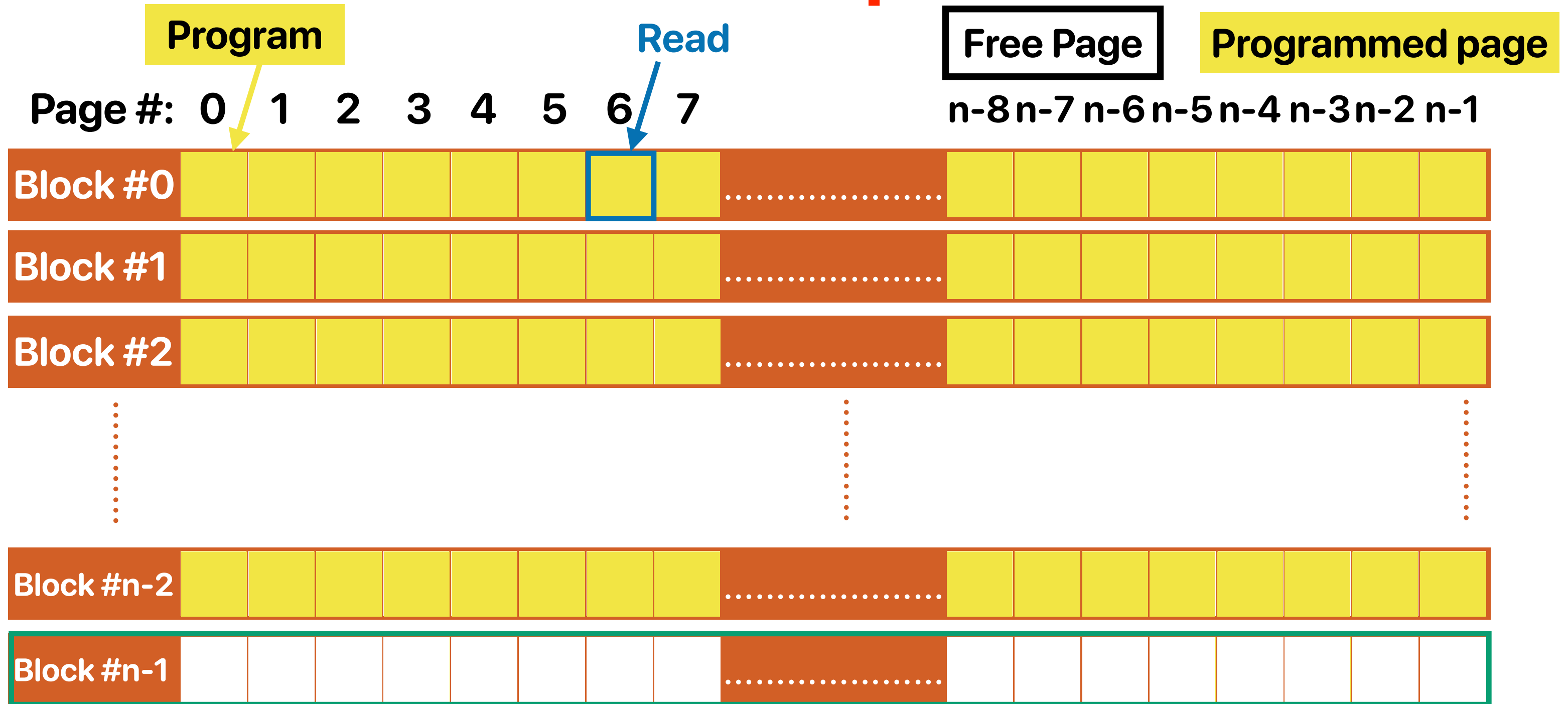
The architecture of an SSD



Basic flash operations



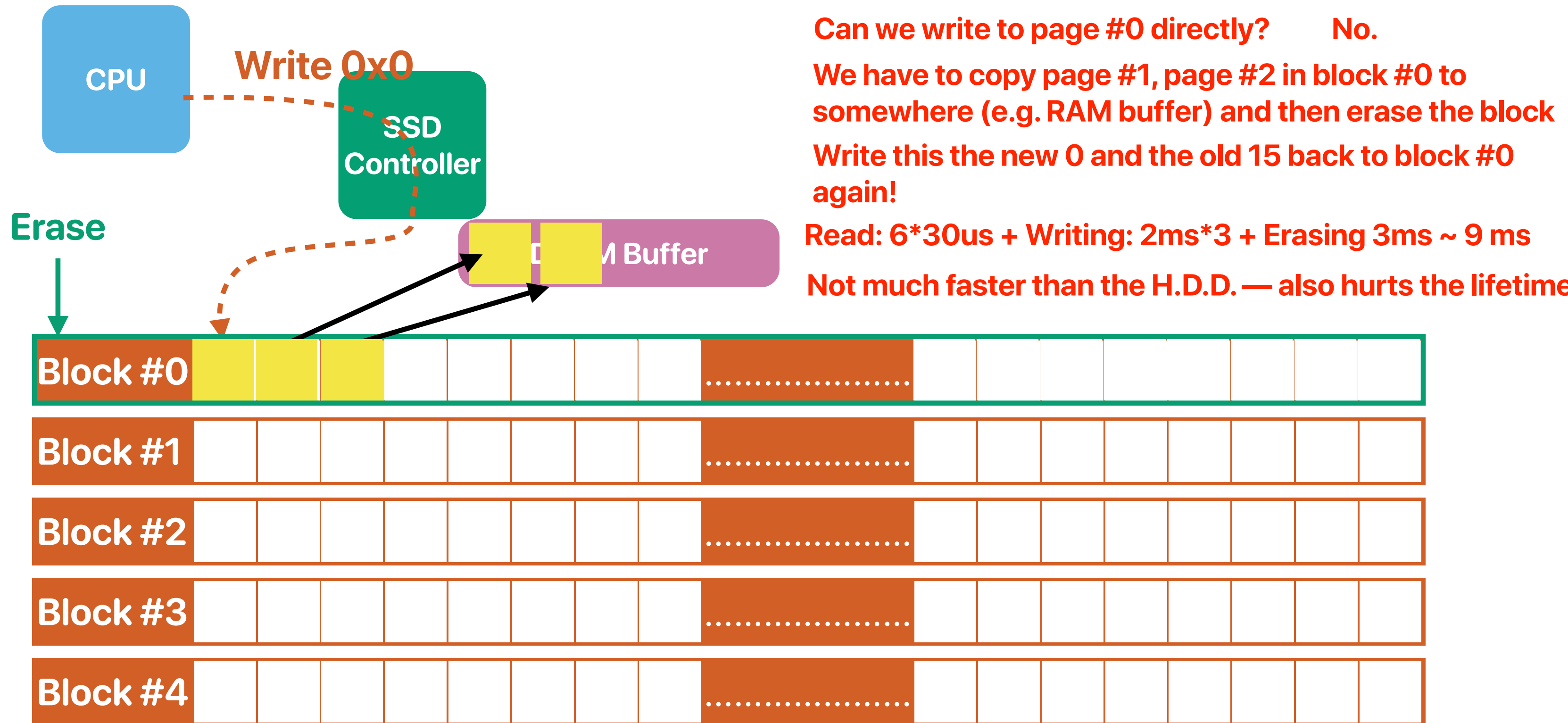
Basic flash operations



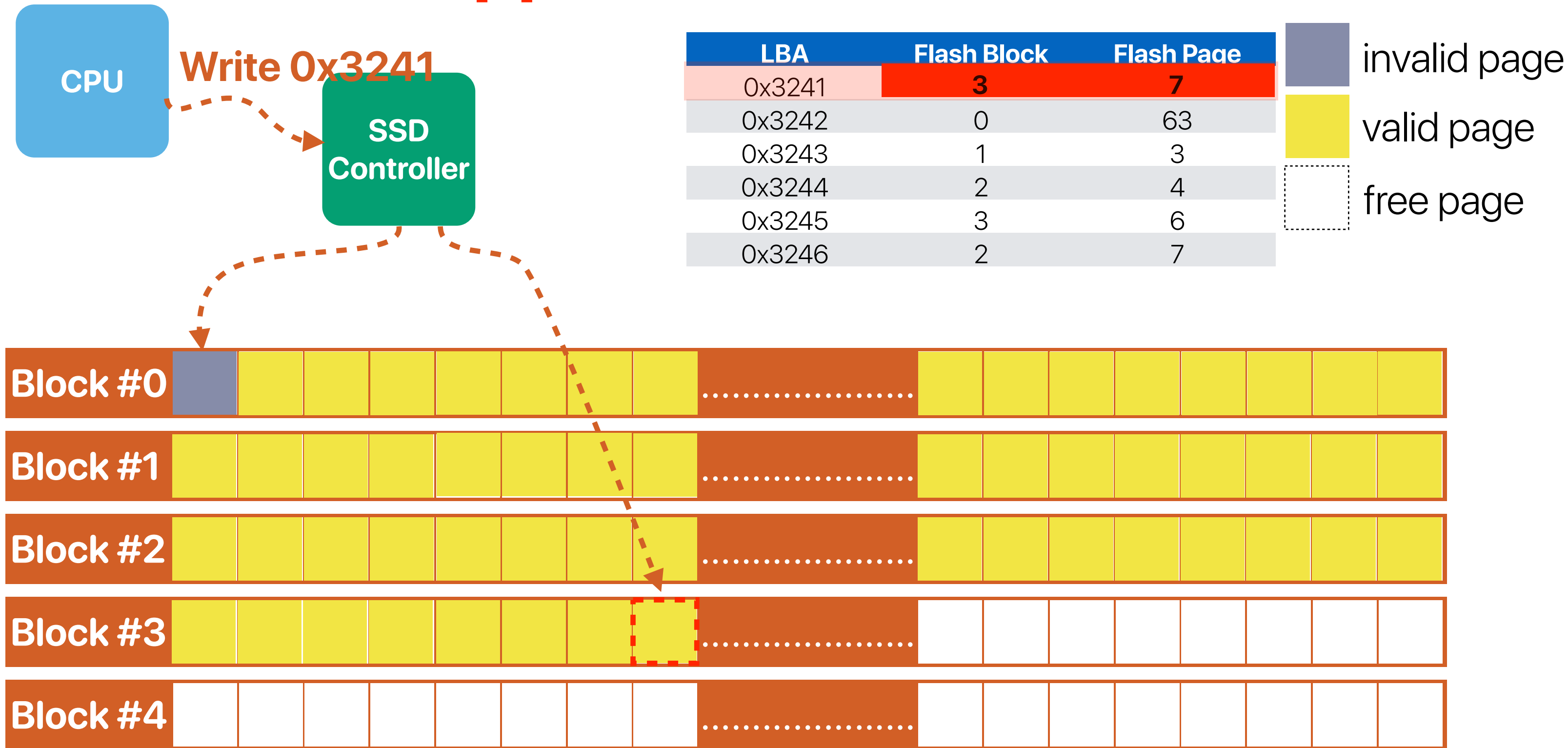
- Flash pages must be erased in "blocks"

What's the potential problem of writes in NAND-based SSDs?

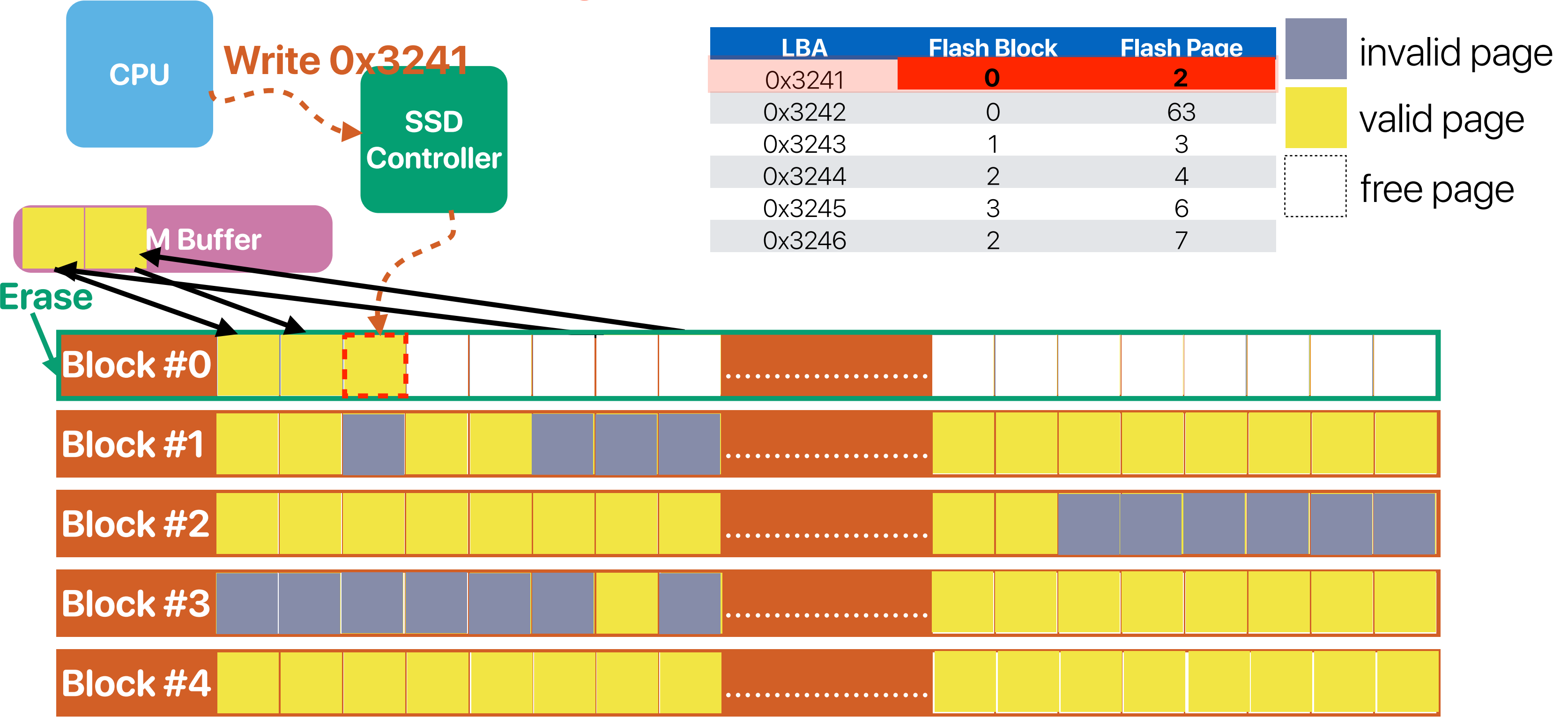
What happens on a write if we use the same abstractions as H.D.D.



What happens on a write with FTL



Garbage Collection in FTL



The architecture of an SSD

Stores the mapping table



Controller + Registers

Perform FTL algorithms

- Maintaining the block device abstraction
- Dealing with the “weird” device characteristics
- Interfacing with the interconnect

The tasks of an SSD controller/firmware

Parse NVMe/SATA/PCIe commands

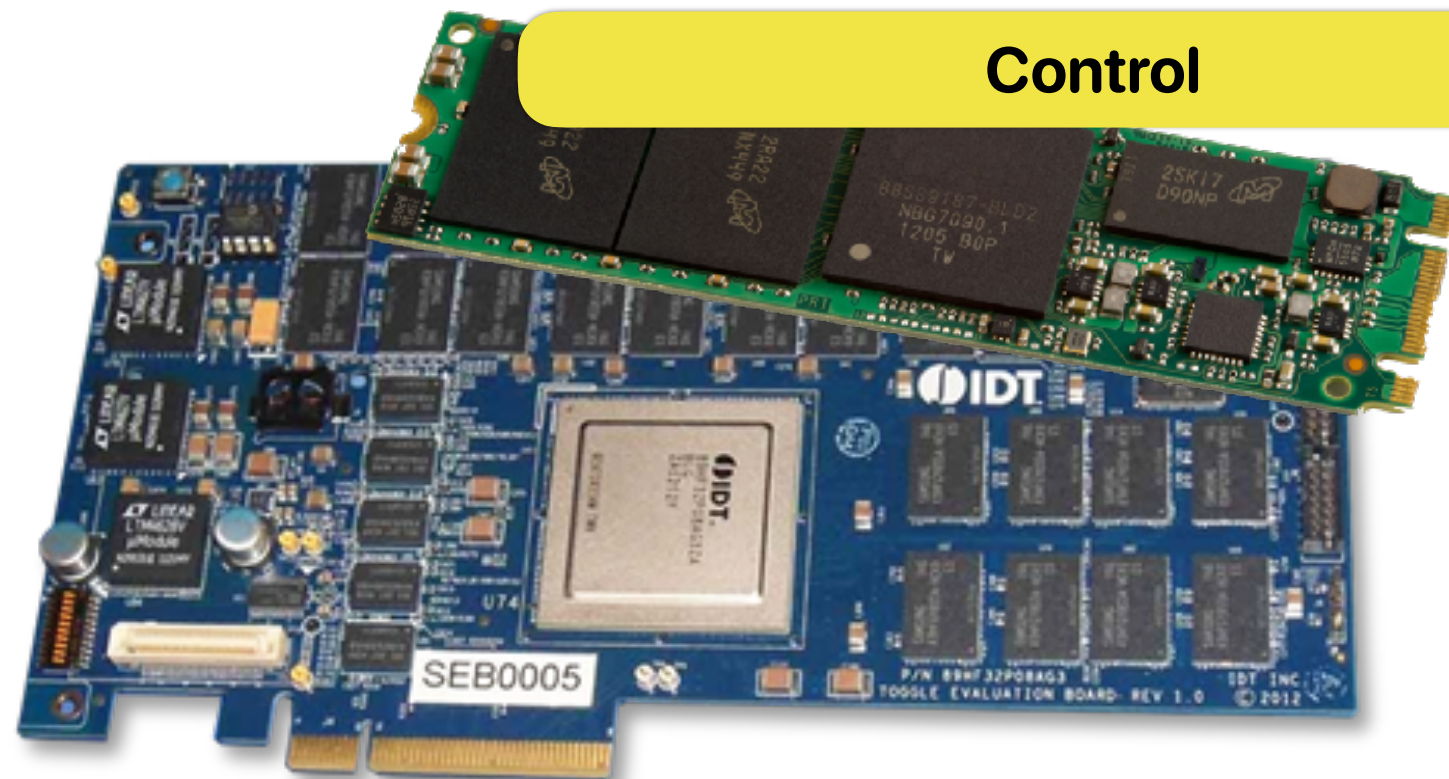
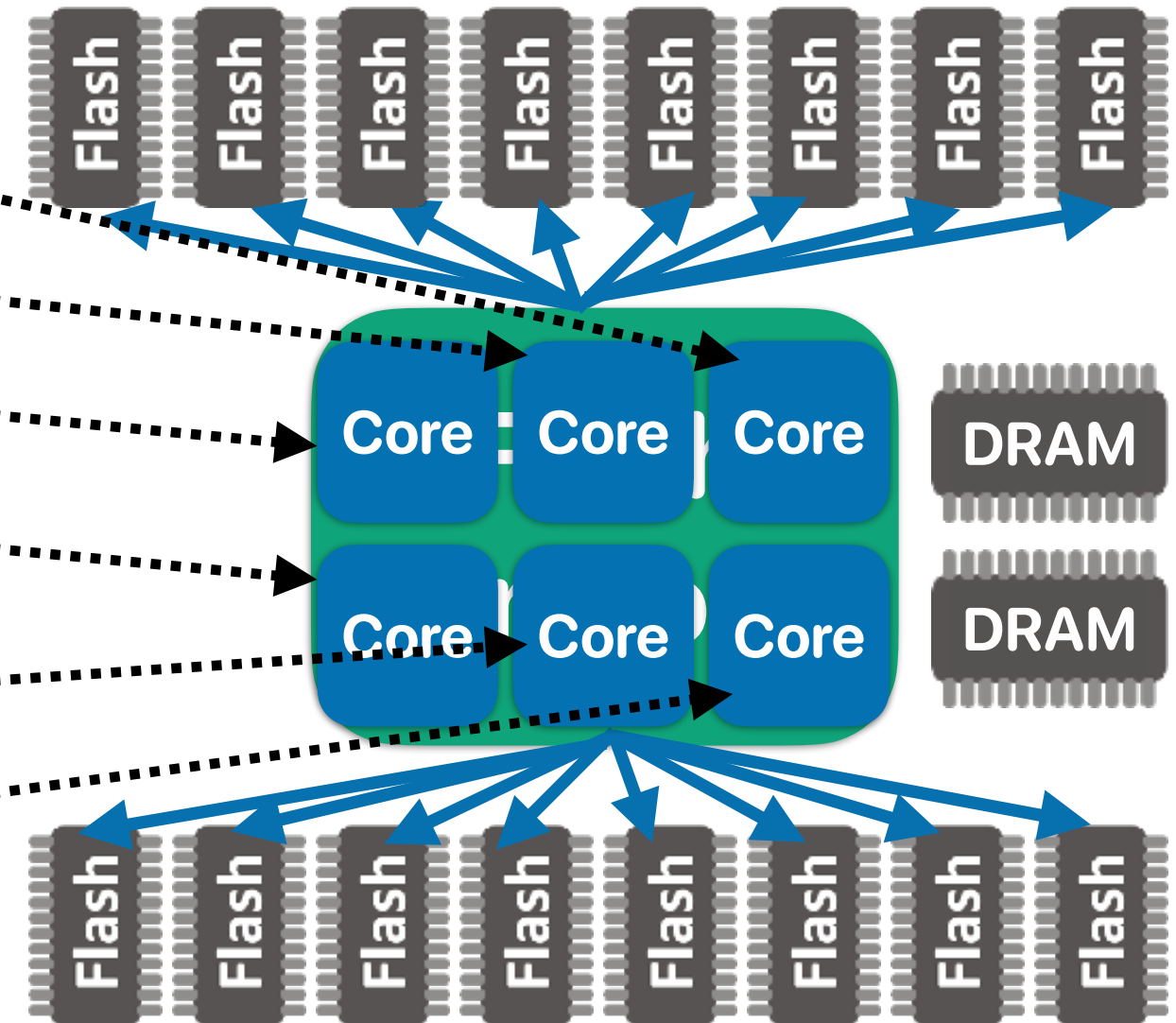
Address mapping/lookup

Wear-leveling/Garbage Collection

Read

Program

Control



What is Firmware?

Firmware is ...

- A software program directly interacts with hardware without an operating system
 - No virtual memory
 - No multitasking
- Typically a program works in between hardware and application "software"
- C is the most frequently used language in programming firmware (rust is catching up, but still a long way to go)

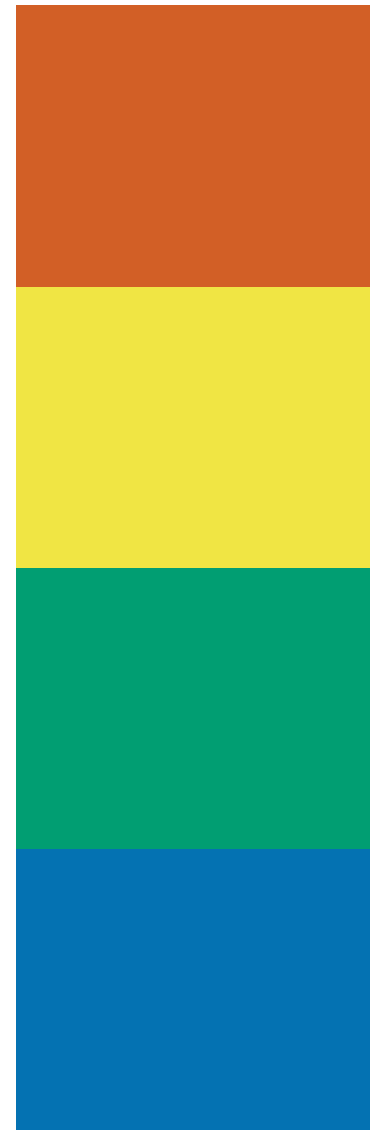
Types of Flash Chips

2 voltage levels,
1-bit



**Single-Level Cell
(SLC)**

4 voltage levels,
2-bit



**Multi-Level Cell
(MLC)**

8 voltage levels,
3-bit



**Triple-Level Cell
(TLC)**

16 voltage levels,
4-bit



**Quad-Level Cell
(QLC)**

Programming in MLC

4 voltage levels,
2-bit

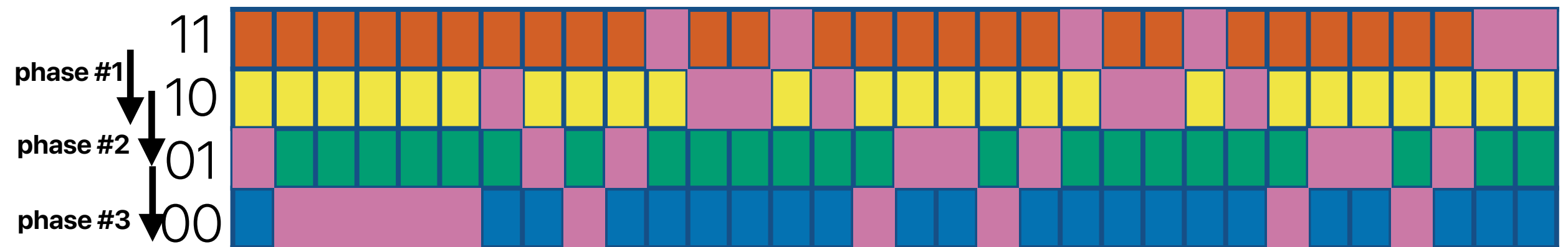


**Multi-Level Cell
(MLC)**

3.140000000000000001243449787580

= 0x40091EB851EB851F

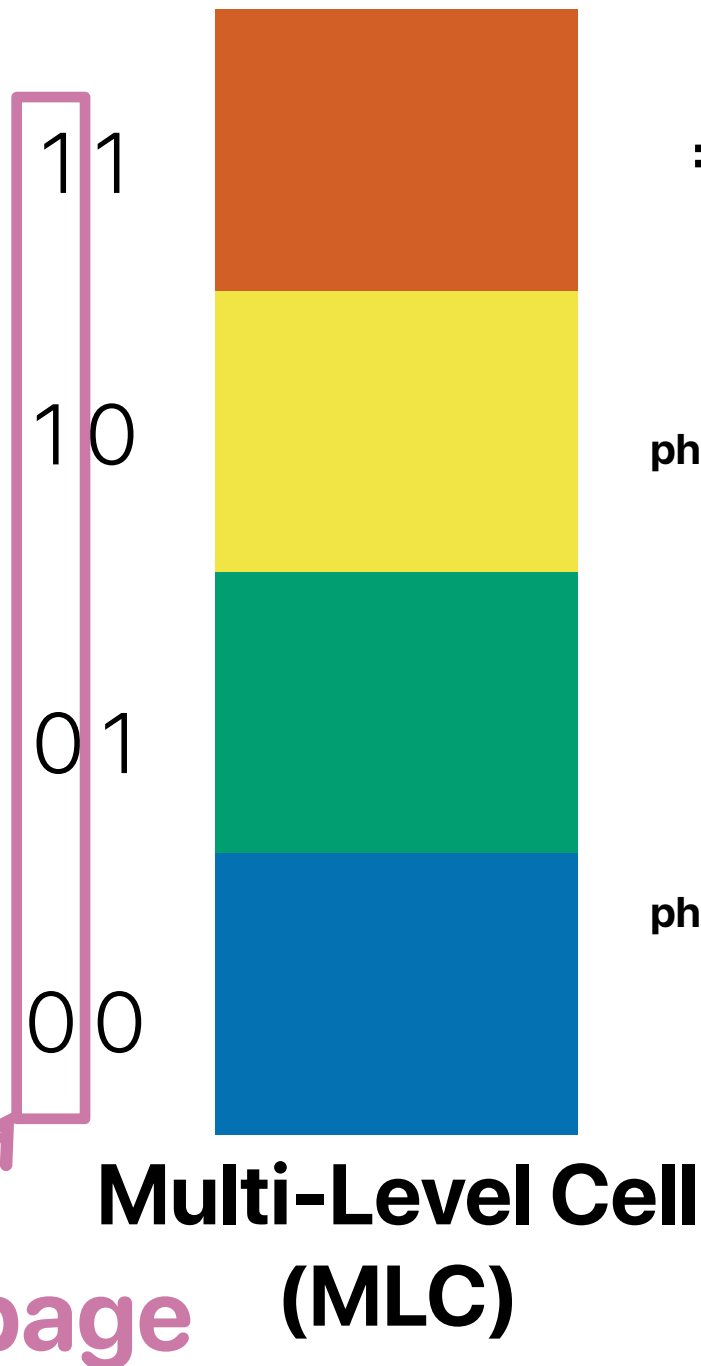
= 01000000 00001001 00011110 10111000 01010001 11101011 10000101 00011111



3 Cycles/Phases to finish programming

Programming in MLC

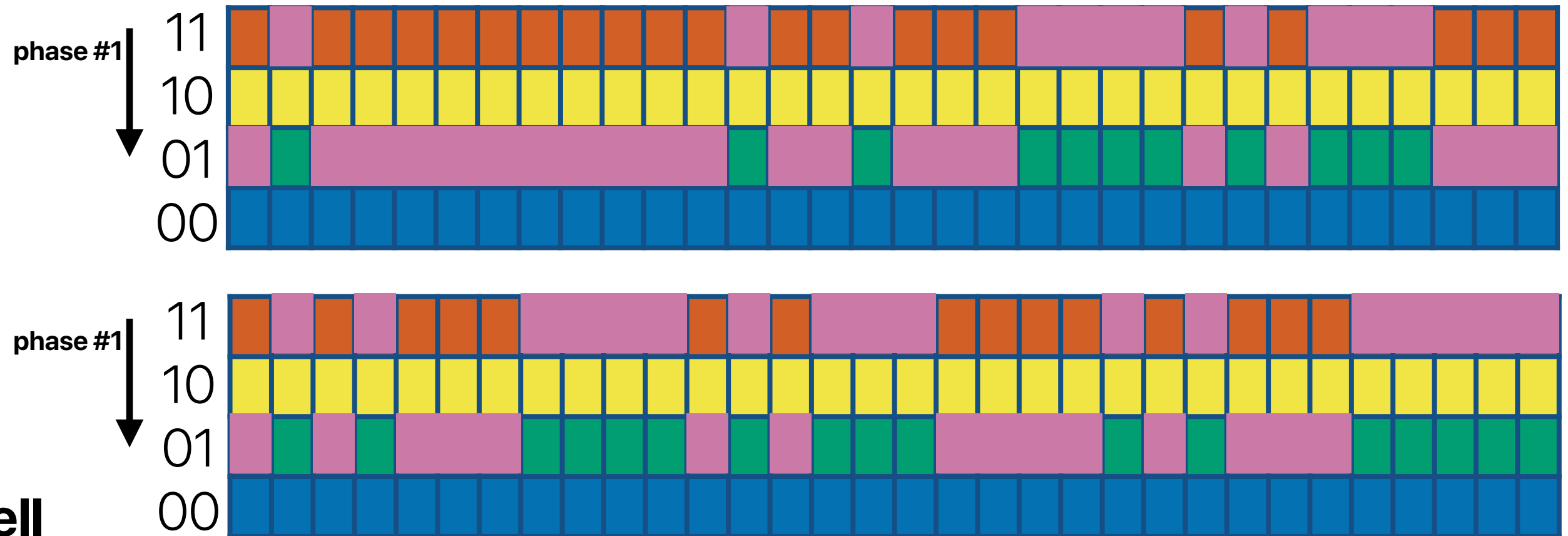
4 voltage levels,
2-bit



3.1400000000000000001243449787580

= 0x40091EB851EB851F

= 01000000 00001001 00011110 10111000 01010001 11101011 10000101 00011111



1 Phase to finish programming the first page!

Programming the 2nd page in MLC

4 voltage levels,
2-bit

2nd page

3.140000000000000000001243449787580

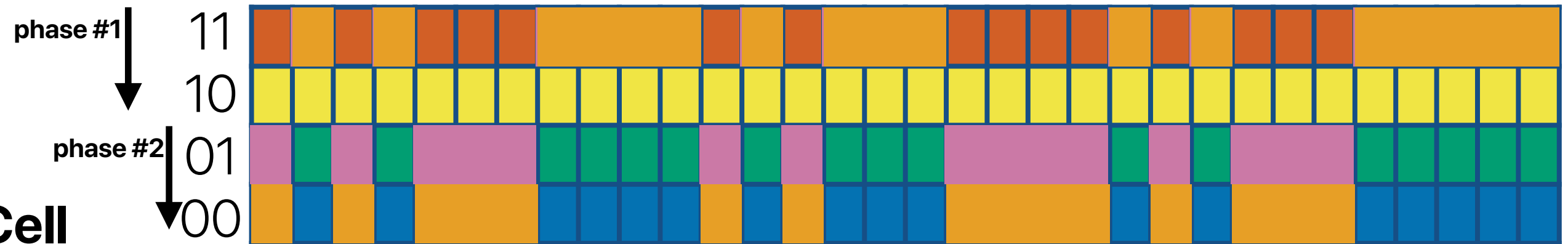
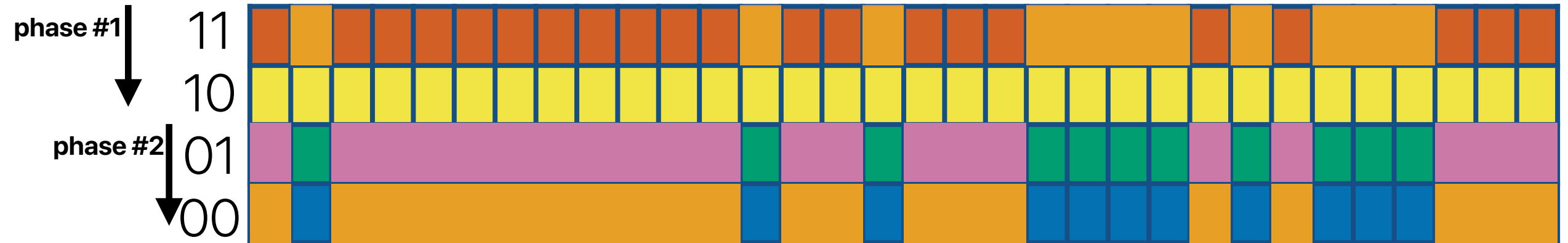
= 0x40091EB851EB851F

= 01000000 00001001 00011110 10111000 01010001 11101011 10000101 00011111

= 01000000 00001001 00011110 10111000 01010001 11101011 10000101 00011111



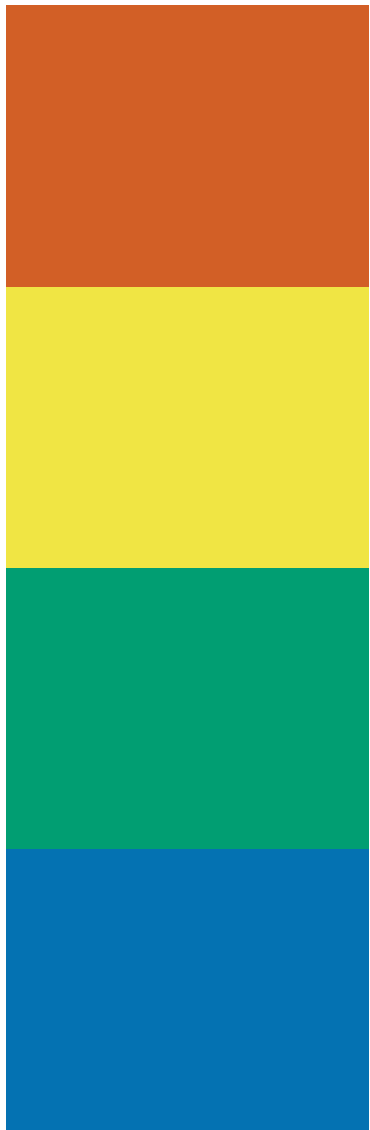
Multi-Level Cell
(MLC)



2 Phase to finish programming the second page!

Optimizing 1st Page Programming in MLC

4 voltage levels,
2-bit

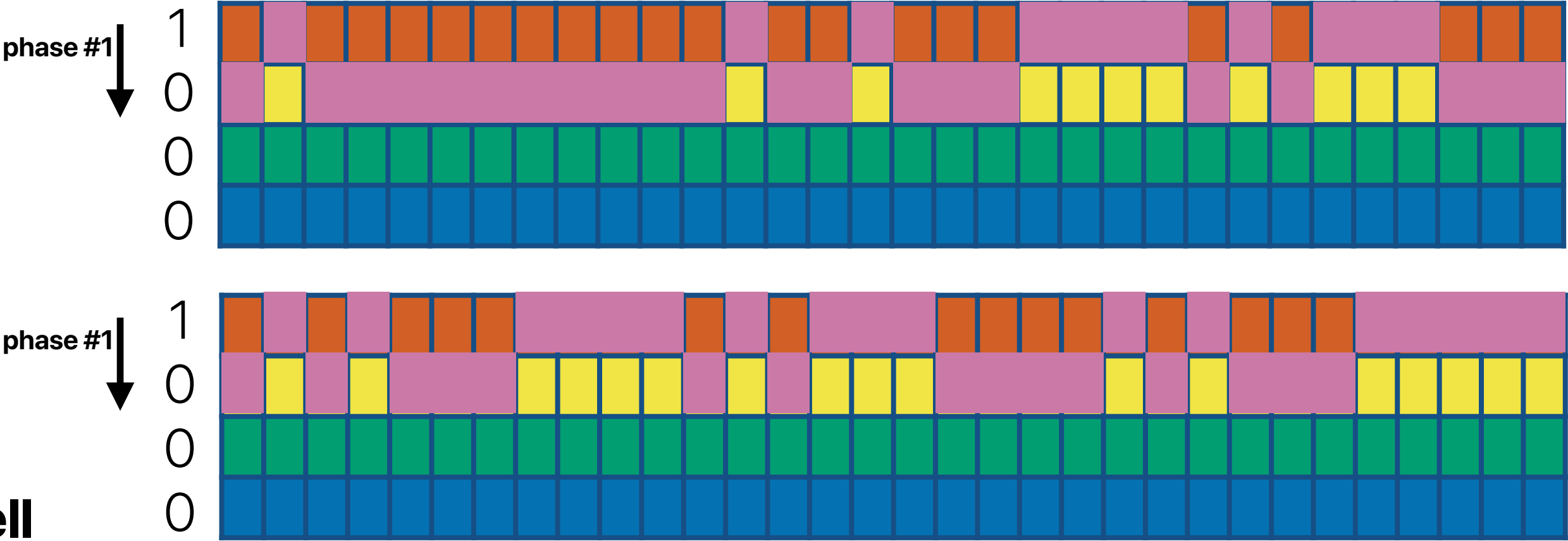


Multi-Level Cell
1st page (MLC)

3.140000000000000000001243449787580

= 0x40091EB851EB851F

= 01000000 00001001 00011110 10111000 01010001 11101011 10000101 00011111



1 Phase to finish programming the first page!

28 — the phase is shorter now

2nd Page Programming in MLC

2nd page 4 voltage levels,
2-bit



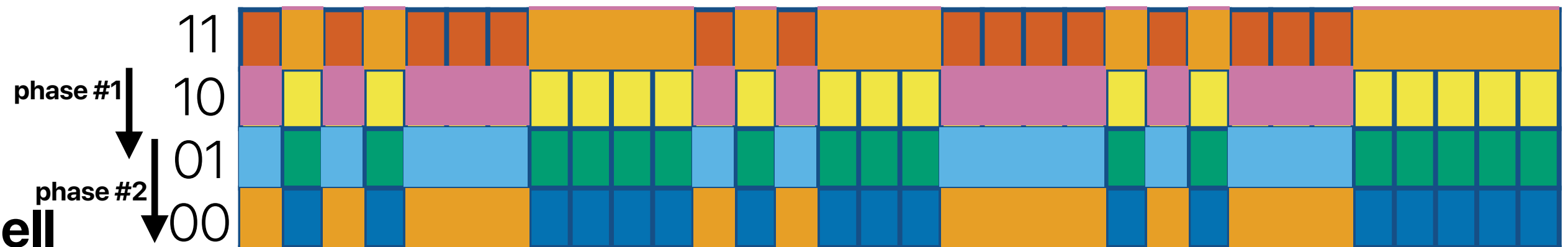
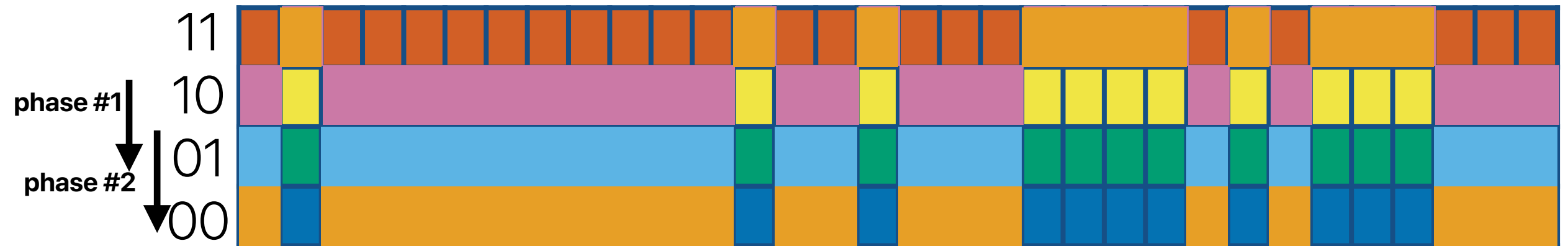
Multi-Level Cell
(MLC)

3.1400000000000000000001243449787580

= 0x40091EB851EB851F

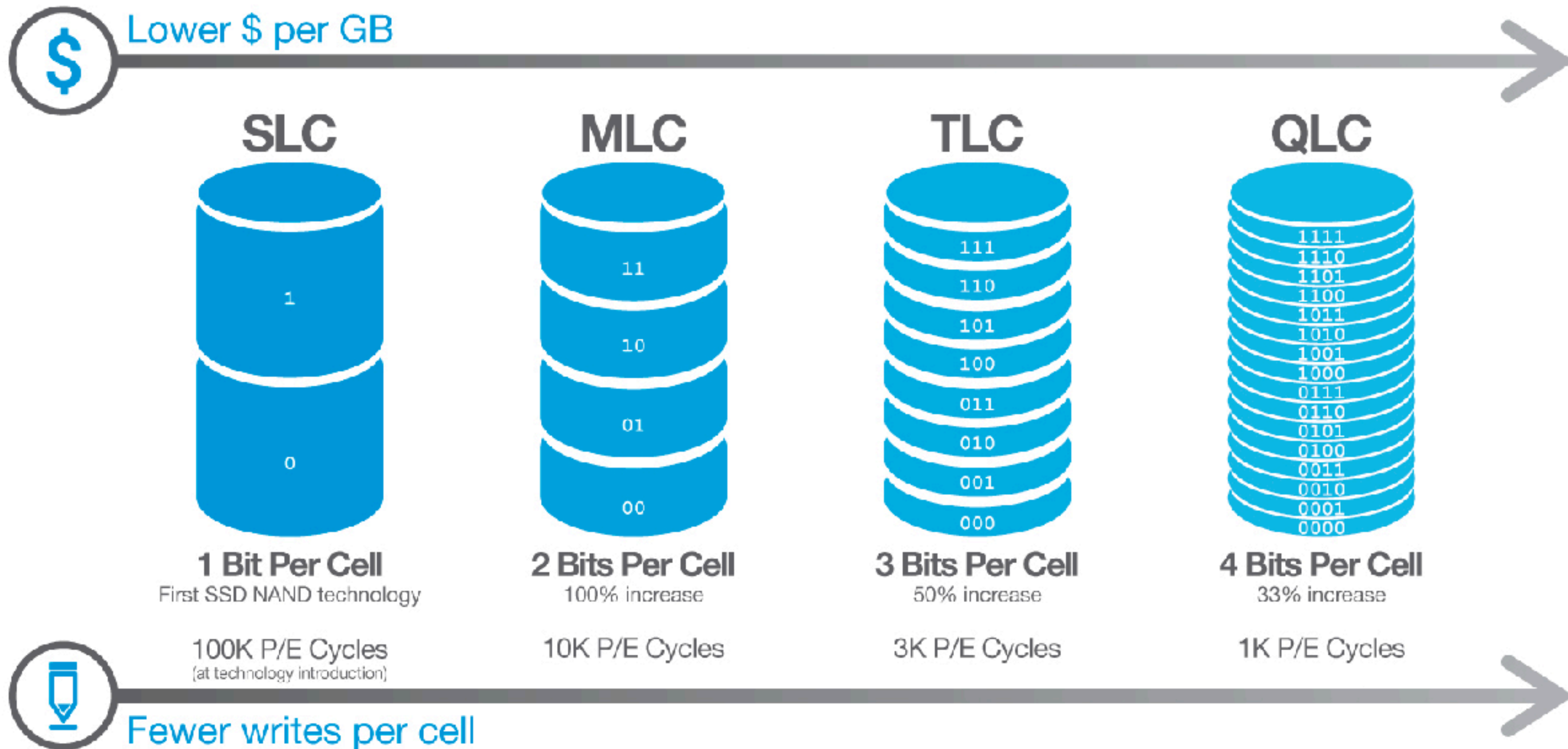
= 01000000 00001001 00011110 10111000 01010001 11101011 10000101 00011111

= 01000000 00001001 00011110 10111000 01010001 11101011 10000101 00011111



2 Phase to finish programming the second page!

QLC = More Density Per NAND Cell



The bandwidth of a flash chip

- Organization
 - Page size x8: 18,592 bytes (16,384 + 2208 bytes)
 - Block size: 2304 pages, (36,864K + 4968K bytes)
 - Plane size: 4 planes x 504 blocks
 - Device size: 512Gb: 2016 blocks; 1Tb: 4032 blocks; 2Tb: 8064 blocks; 4Tb: 16,128 blocks; 8Tb: 32,256 blocks



TLC 512Gb-8Tb NAND
Electrical Specifications – Array Characteristics

is suspended or ongoing					
READ PAGE operation time without/with V _{pp}	t _R	57/41	60	μs	2, 12
SNAP READ operation time	t _{RSNAP}	27	37	μs	
Cache read busy time	t _{RCBSY}	11	60	μs	2, 8, 12

$$\frac{16KB}{60\mu s} = 267MB/sec$$

Recap: Seagate Barracuda 12

- SATA 3.0 (600MB/s in theory), 7200 R.P.M., seek time around 8 ms. Assume the controller overhead is 0.2ms. What's the **latency** and **bandwidth** of accessing consecutive **4MB** data?

Latency = seek time + rotational delay + transfer time + controller overhead

$$8 \text{ ms} + \frac{1}{2} \times \frac{1}{\frac{7200}{60}} + \frac{4}{600} + 0.2 \text{ ms}$$

$$= 8 \text{ ms} + 4.17 \text{ ms} + 6.67 \text{ ms} + 0.2 \text{ ms} = 19.04 \text{ ms}$$

Bandwidth = volume_of_data over period_of_time

$$= \frac{4MB}{19.04ms} = 210.1MB/sec$$

Trading latencies with bandwidth

Multi-channel to optimize bandwidth

Flashtec™ NVMe2032 and NVMe2016 Controllers

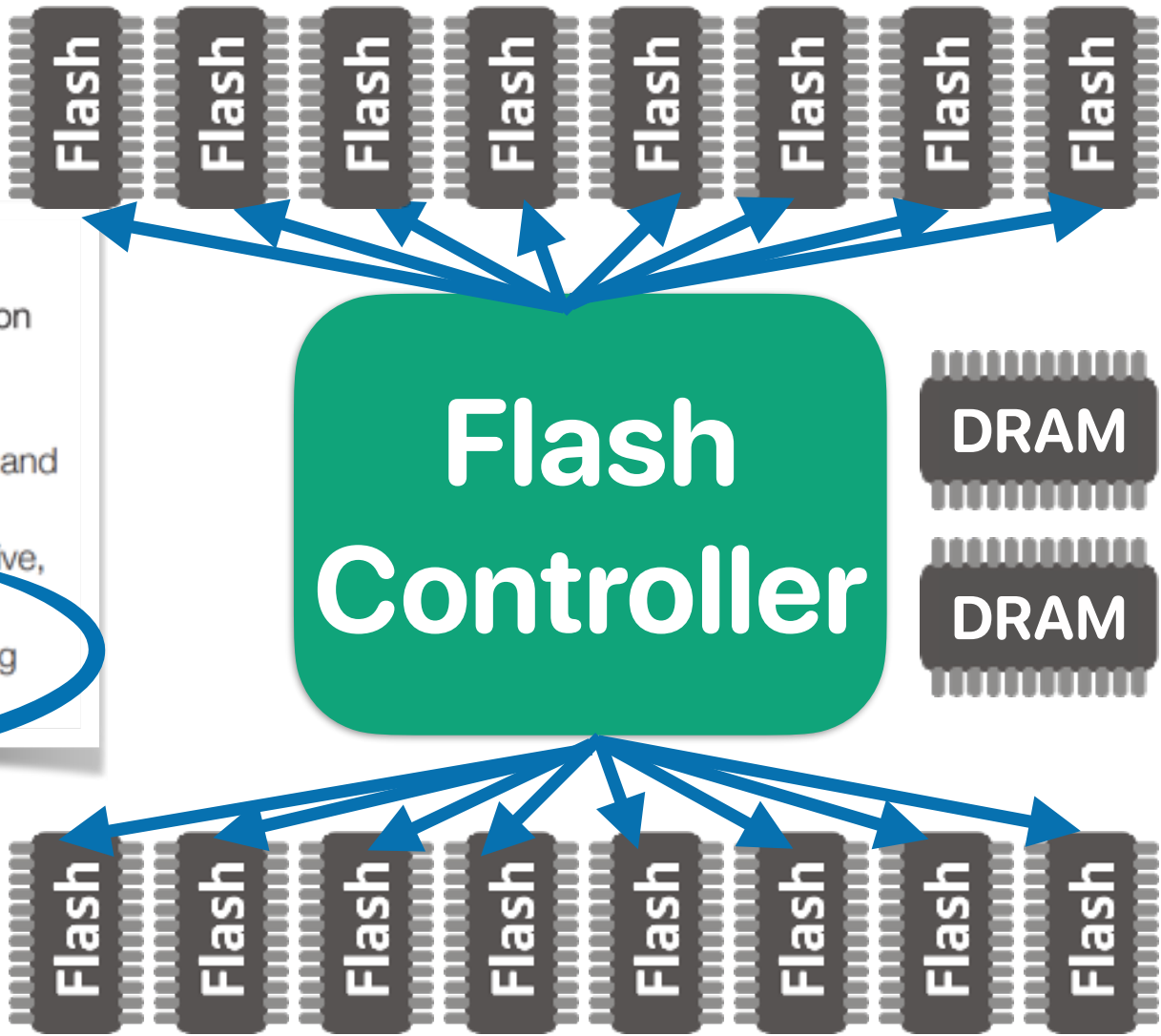
32- and 16-Channel PCIe Flash Controller Products

Summary

The Flashtec™ 2nd generation NVMe Controller Family enables the server and data centers to realize the highest performance SSDs utilizing new technologies. Combining world-class capacity and flexibility, the Flashtec is a reliable choice. The Flashtec NVMe2032 and NVMe2016 controllers support PCIe Express (NVMe) host interface and are optimized for high-performance operations, performing all flash management operations on-chip and utilizing processing and memory resources.

Features

- Flashtec NVMe2032 controller can achieve up to 1 million random read IOPS on 4 KB operations
- Up to 20 TB Flash capacity using 256 GB Flash
- SLC, MLC, Enterprise MLC, and TLC Flash with toggle and ONFI interface
- PCIe Gen 3 x8 or dual independent PCIe Gen 3 x4 (active, active/standby) host interface
- 16 and 32 independent Flash channels, each supporting up to 8 CE



Each ~267MB/sec
16 channels == 4.2 GB/sec



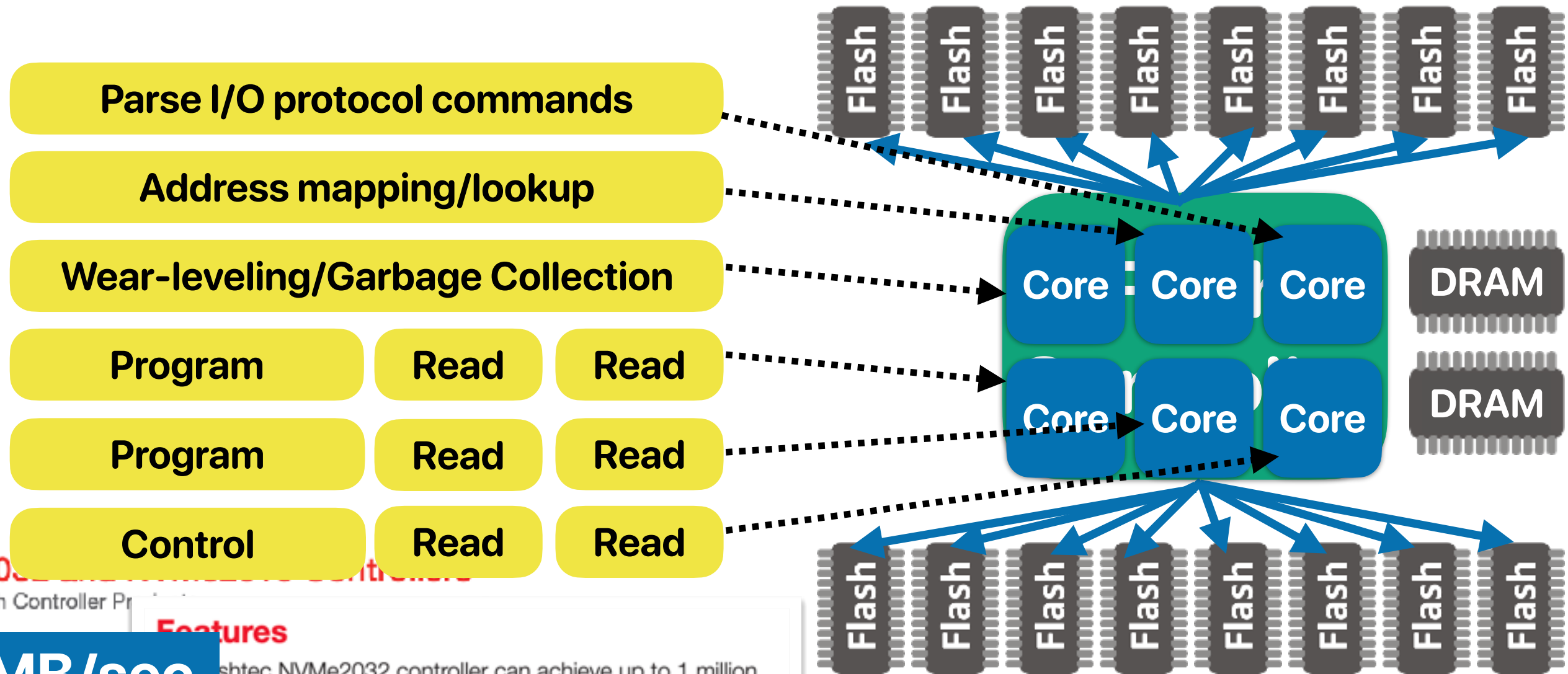
- Organization
 - Page size x8: 18,592 bytes (16,384 + 2208 bytes)
 - Block size: 2304 pages, (36,864K + 4968K bytes)
 - Plane size: 4 planes x 504 blocks
 - Device size: 512Gb: 2016 blocks; 1Tb: 4032 blocks; 2Tb: 8064 blocks; 4Tb: 16,128 blocks; 8Tb: 32,256 blocks

TLC 512Gb-8Tb NAND – Array Characteristics

is suspended or ongoing					
READ PAGE operation time without/with V _{pp}	^t R	57/41	60	μs	2, 12
SNAP READ operation time	^t RSNAP	27	37	μs	
Cache read busy time	^t RCBSY	11	60	μs	2, 8, 12

What would be the right processor architecture of the firmware?

Multi-channel to optimize bandwidth



Each ~267MB/sec
16 channels ==
4.2GB/sec

Flashtec™ NVMe2032
32- and 16-Channel PCIe Flash Controller Pr

Features

Flashtec NVMe2032 controller can achieve up to 1 million random read IOPS on 4 KB operations
to 20 TB Flash capacity using 256 GB Flash
C, MLC, Enterprise MLC, and TLC Flash with to
FI interface
le Gen 3 x8 or dual independent PCIe Gen 3 x4 (active,
ative/standby) host interface

- 16 and 32 independent Flash channels, each support up to 8 CE



Organization

- Page size x8: 18,592 bytes (16,384 + 2208 bytes)
- Block size: 2304 pages, (36,864K + 4968K bytes)
- Plane size: 4 planes x 504 blocks
- Device size: 512Gb: 2016 blocks; 1Tb: 4032 blocks; 2Tb: 8064 blocks; 4Tb: 16,128 blocks; 8Tb: 32,256 blocks

TLC 512Gb-8Tb NAND - Array Characteristics

is suspended or ongoing					
READ PAGE operation time without/with V _{pp}	^t R	57/41	60	μs	2, 12
SNAP READ operation time	^t RSNAP	27	37	μs	
Cache read busy time	^t RCBSY	11	60	μs	2, 8, 12

If you want to design an I/O protocol for the host to interact with SSD, what would be the right practices?

Right protocol for SSDs

- Interrupt or polling?
- DMA or PIO?
- What other characteristics that conventional HDD cannot support?

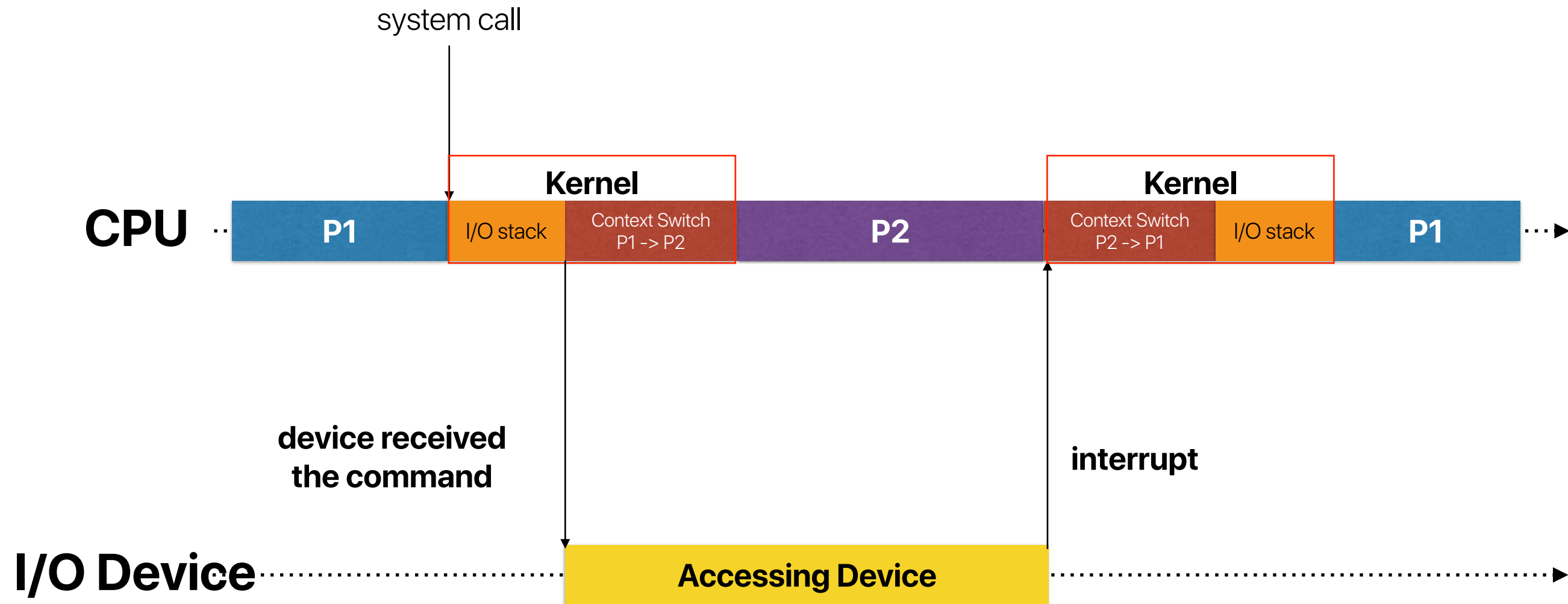
Multiple, parallel access at the same time!

What happened during a context switch?

Recap: What happens during context switch

- Load architectural states from process control block (somewhere in the main memory, potentially a cache miss, TLB miss) — takes several microseconds if everything is in the physical memory
- Set processor registers according to the loaded architectural states
 - **Set the CR3 (page table base register in x86) register to identify the root page table node in the hierarchical page table**
 - Set the RIP (program counter in x86) to the previous execution
- **Restore virtual memory address**
 - **You must load the root page table node to the main memory at least.**
 - **TLB flush**
 - Invalidate all entries in the TLB
 - Most TLBs are not tagged, so you've to do this
- You DO NOT have to load every page content back from disk — remember that we have demand paging!

To switch or not to switch that's the question.

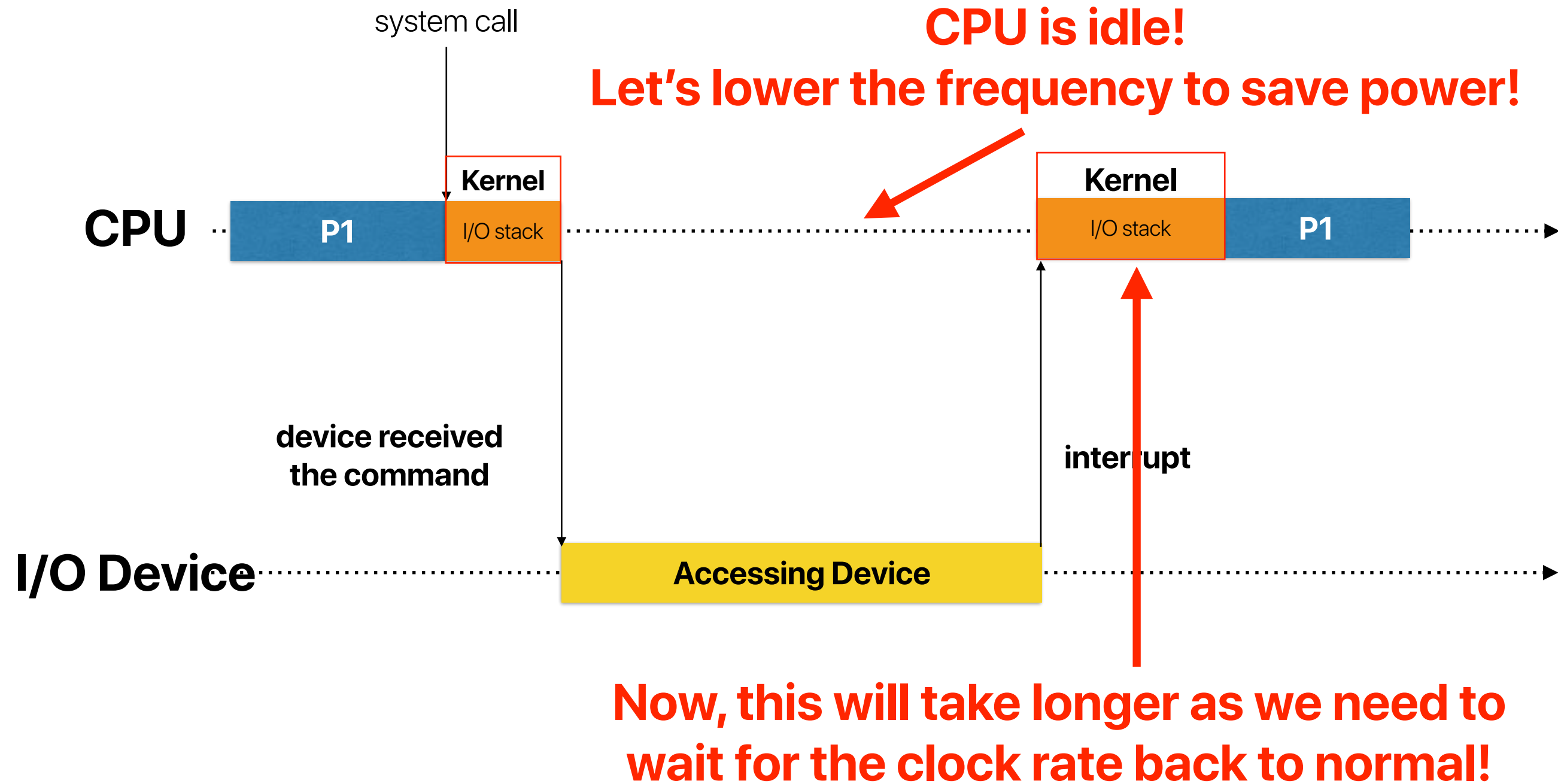


If $T_{\text{Context switch P1} \rightarrow \text{P2}} + T_{\text{Context switch P2} \rightarrow \text{P1}} < T_{\text{Accessing peripherals}}$
makes sense to context switch

But context switch overhead is not the only thing

- Cache warm up cost when you switch back
- TLB warm up cost

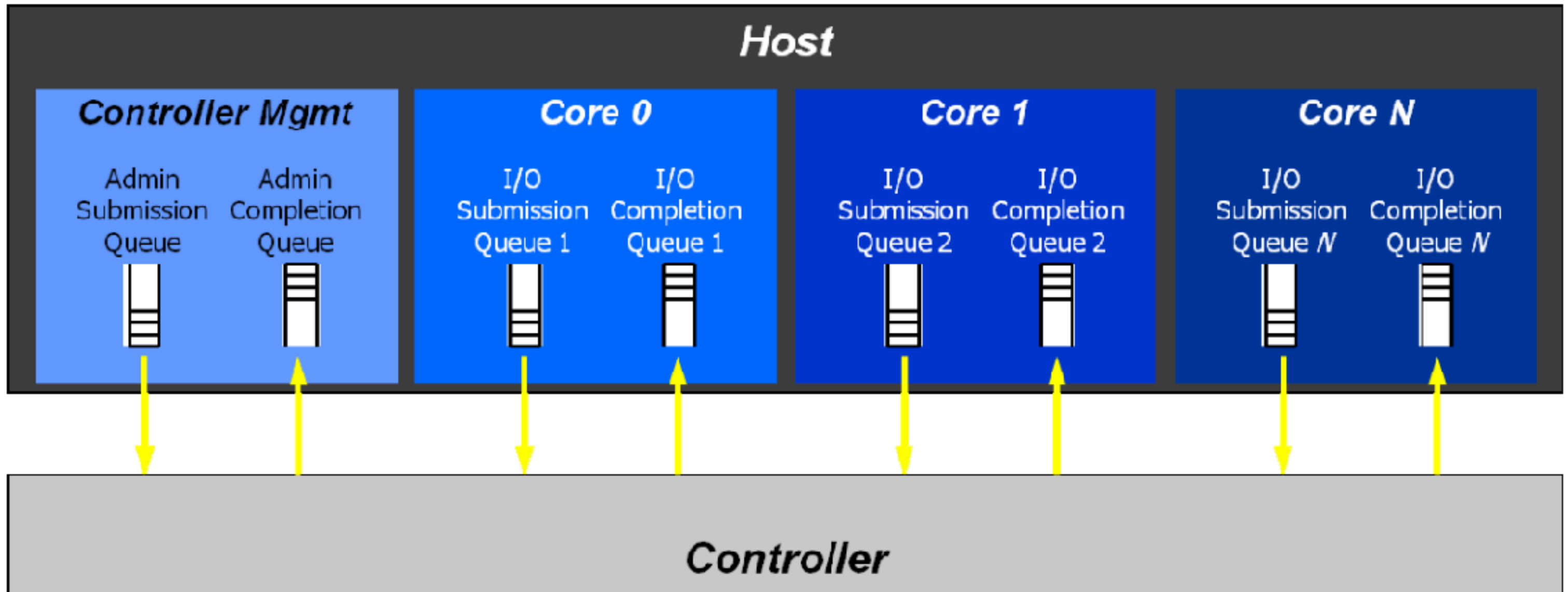
What if we don't switch?



What can you do to take advantage of multiple access channels in SSDs?

NVMe Model

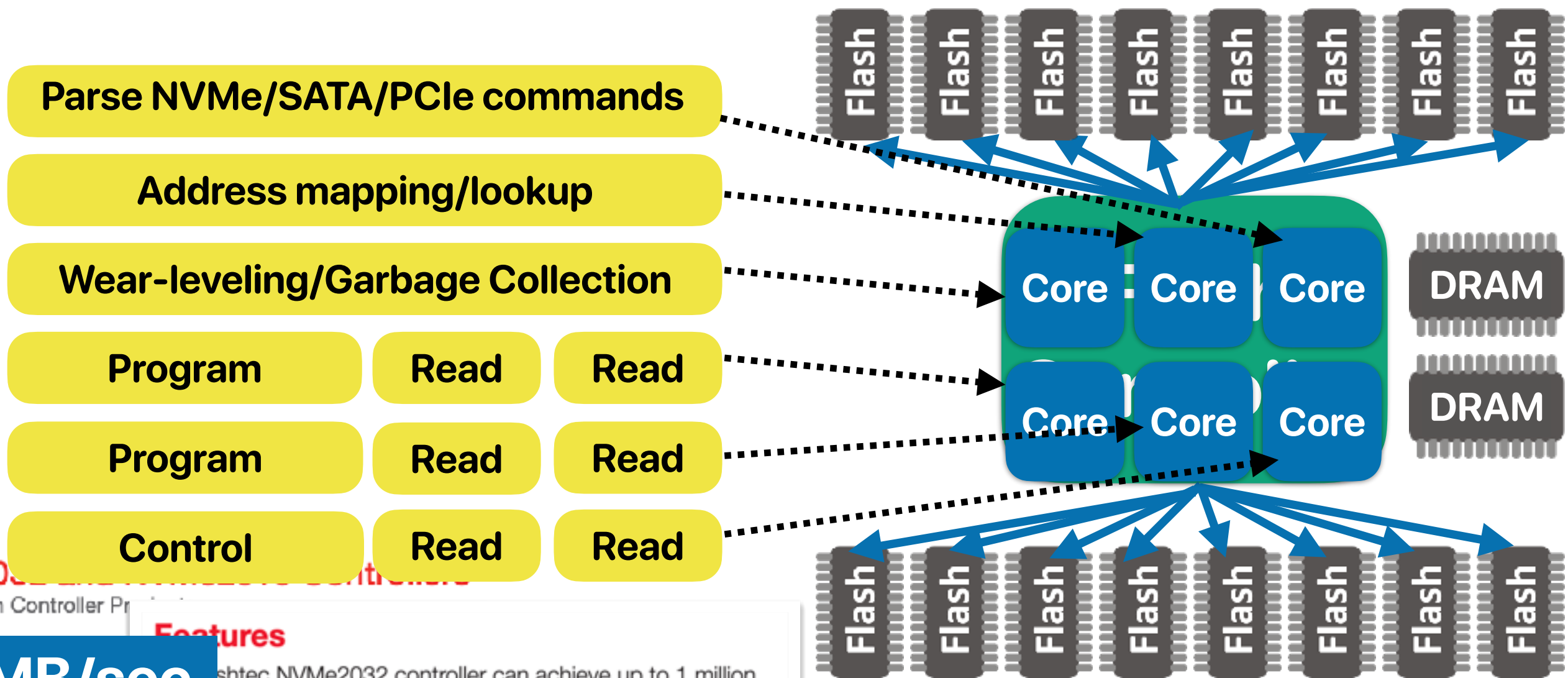
Figure 3: Queue Pair Example, 1:1 Mapping



https://nvmexpress.org/wp-content/uploads/2013/04/NVM_whitepaper.pdf

Why does MQ make sense?

Multi-channel to optimize bandwidth



Each ~267MB/sec
16 channels ==
4.2GB/sec

Flashtec™ NVMe2032
32- and 16-Channel PCIe Flash Controller Pr

Features

Flashtec NVMe2032 controller can achieve up to 1 million random read IOPS on 4 KB operations
to 20 TB Flash capacity using 256 GB Flash
C, MLC, Enterprise MLC, and TLC Flash with to
FI interface
le Gen 3 x8 or dual independent PCIe Gen 3 x4 (active,
ative/standby) host interface

- 16 and 32 independent Flash channels, each support up to 8 CE



Organization

- Page size x8: 18,592 bytes (16,384 + 2208 bytes)
- Block size: 2304 pages, (36,864K + 4968K bytes)
- Plane size: 4 planes x 504 blocks
- Device size: 512Gb: 2016 blocks; 1Tb: 4032 blocks; 2Tb: 8064 blocks; 4Tb: 16,128 blocks; 8Tb: 32,256 blocks

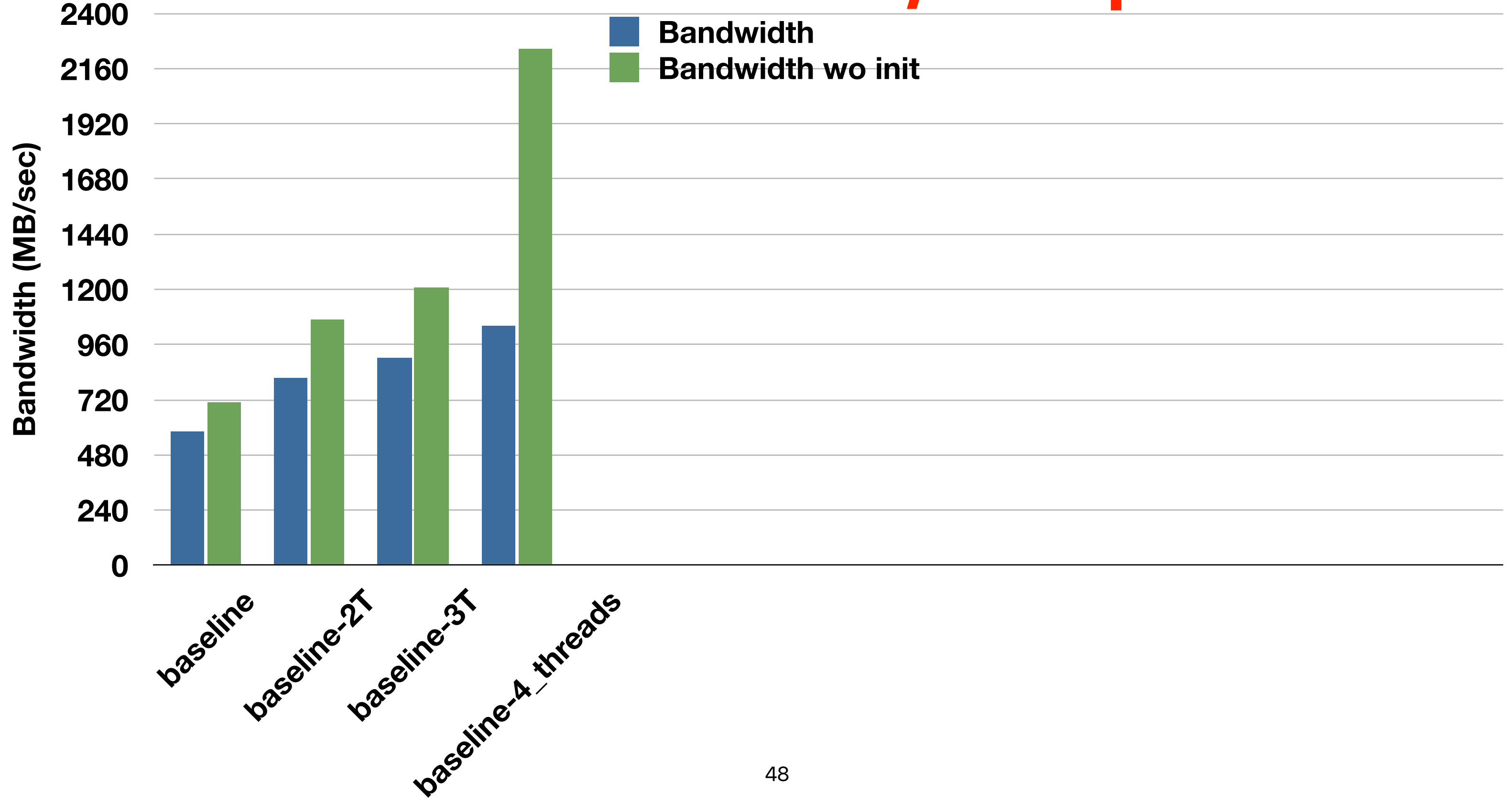
TLC 512Gb-8Tb NAND - Array Characteristics

is suspended or ongoing					
READ PAGE operation time without/with V _{pp}	^t R	57/41	60	μs	2, 12
SNAP READ operation time	^t RSNAP	27	37	μs	
Cache read busy time	^t RCBSY	11	60	μs	2, 8, 12

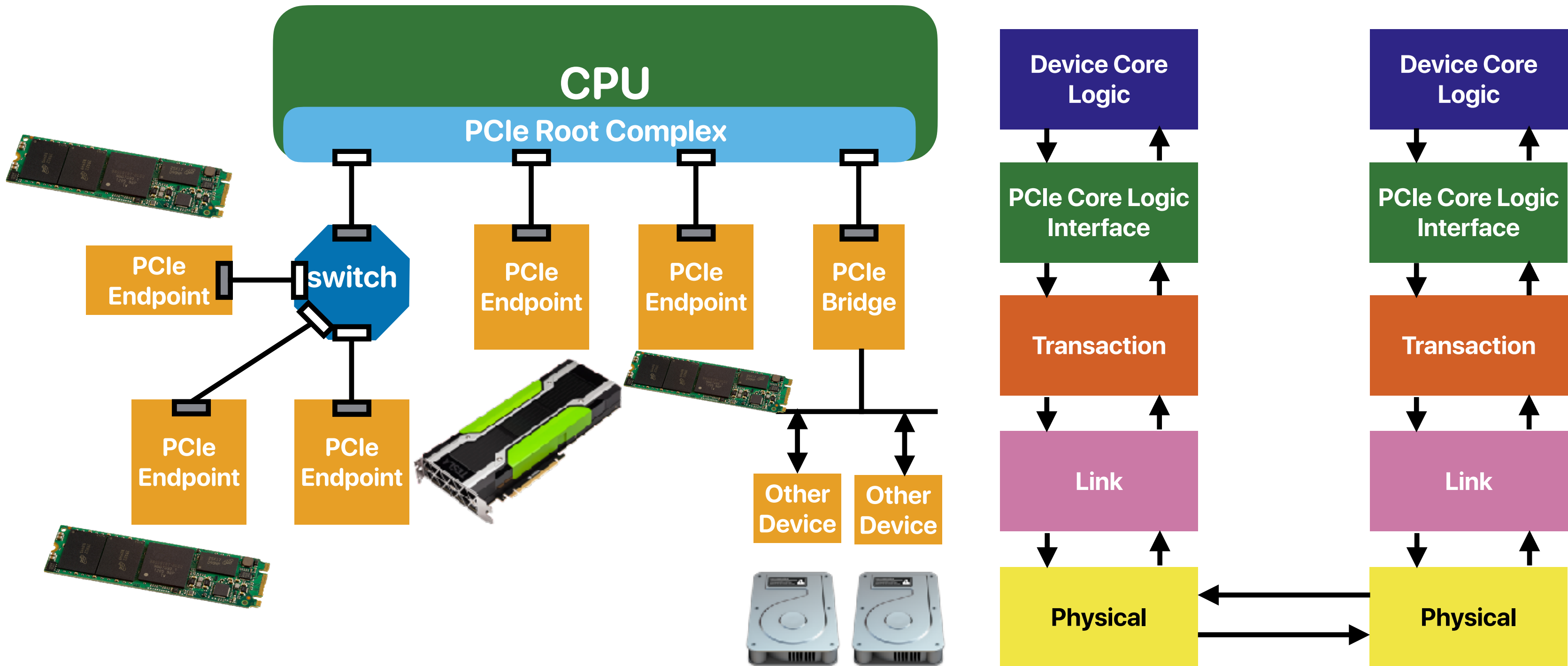
Why does MQ makes sense?

- Flash SSDs have multiple channels inside!
 - You need multiple requests to fill the requests
- The same design cannot work on HDDs — HDDs have only one head

Multi-threaded I/O helps

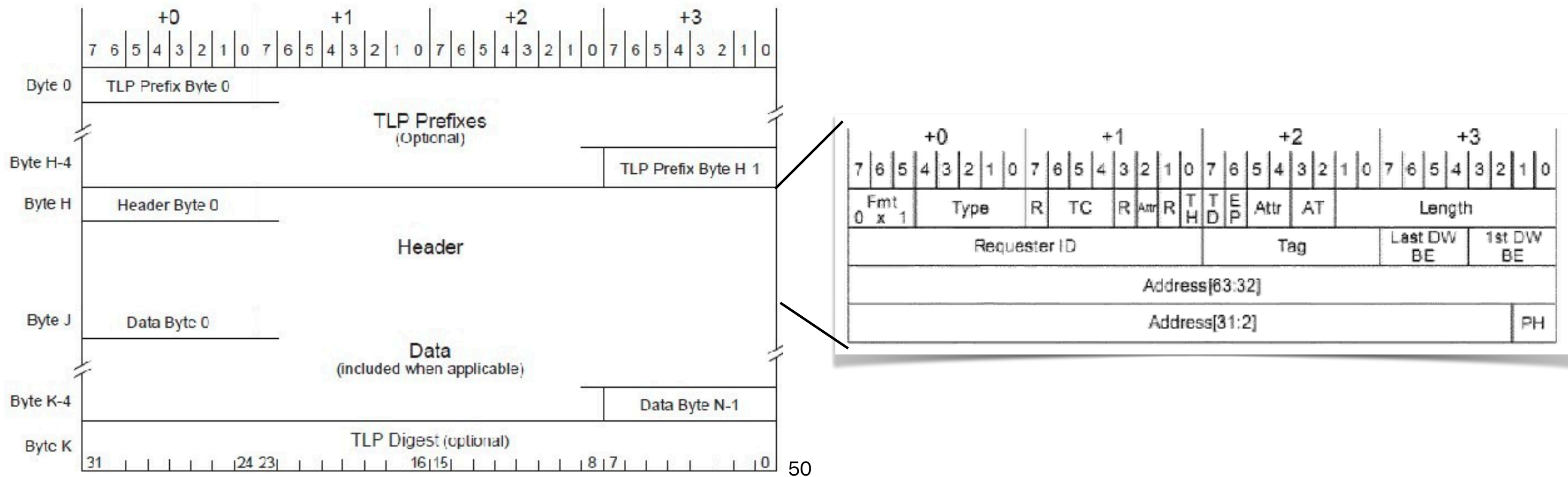


PCIe "Interconnect"



PCIe interconnect

- Very similar to computer networks
- Use "memory addresses" as the identifier for routing
- Peer-to-peer communication is possible



NVMe

- The standard of PCIe SSD devices now
 - Provides multiple command queues to better support multithreading hardware
 - Allows more parallelism inside the SSD
- The “payload” of a PCIe packet

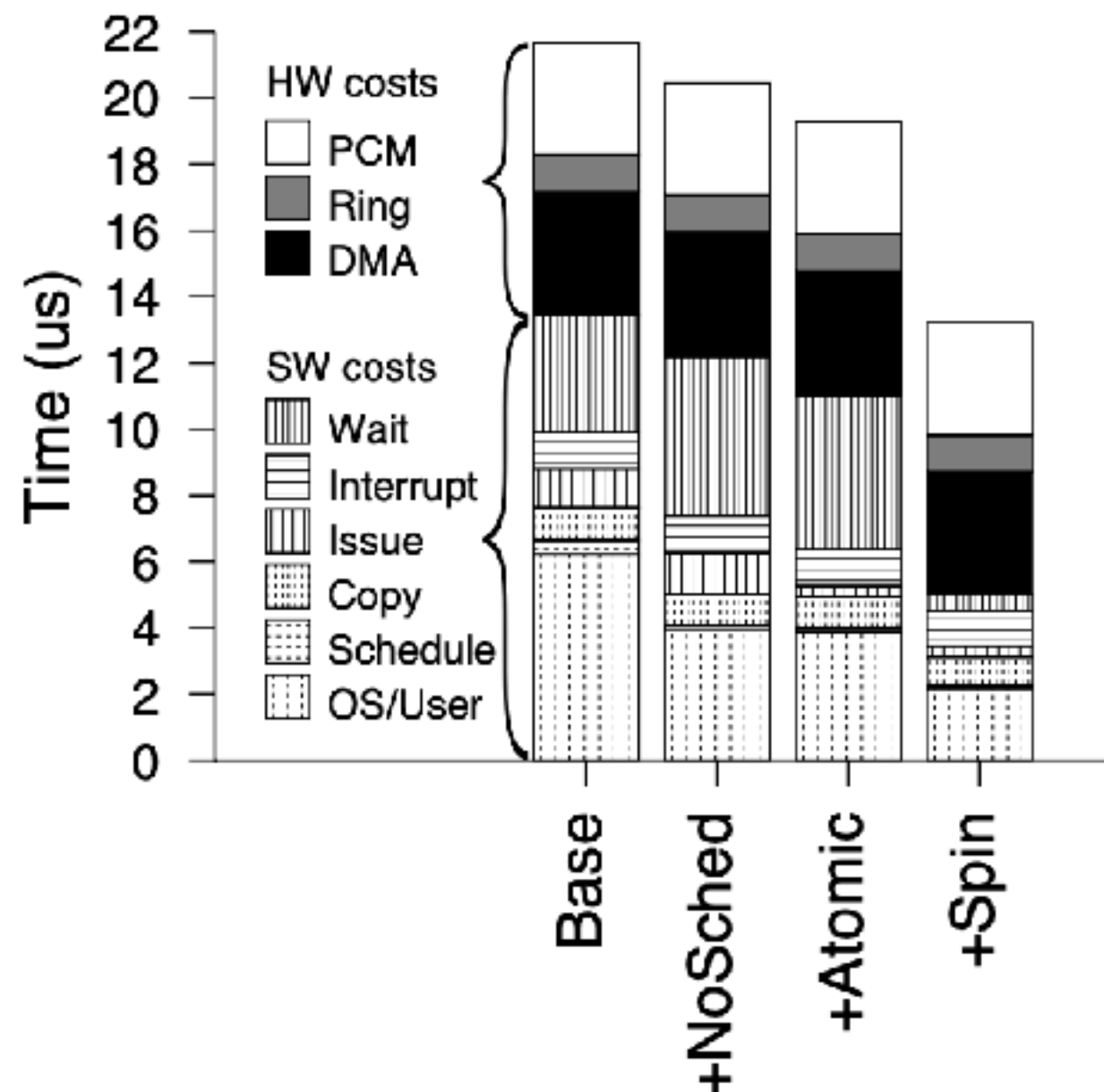
0	8	16	32	48	63
OPCODE	FLAGS	command id	Namespace ID		
reserved					
metadata					
PRP1					
PRP2					
Start LBA					
length		control	Dataset management		
Reference tag			App tag		App mask

Let's walk through an NVMe driver



Software overhead

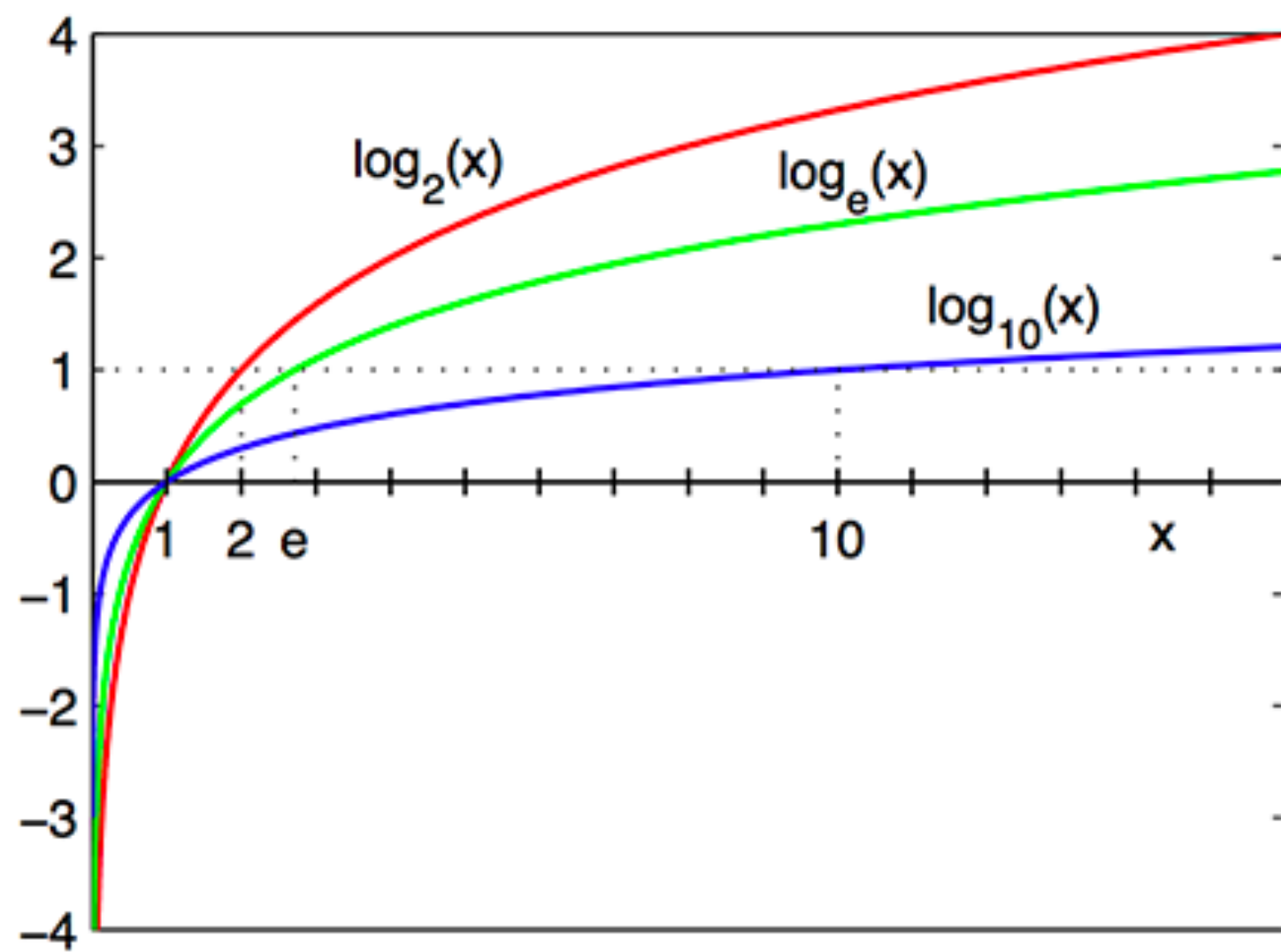
Label	Description	Baseline latency (μ s)	
		Write	Read
OS/User	OS and userspace overhead	1.98	1.95
OS/User	Linux block queue and no-op scheduler	2.51	3.74
Schedule	Get request from queue and assign tag	0.44	0.51
Copy	Data copy into DMA buffer	0.24/KB	-
Issue	PIO command writes to Moneta	1.18	1.15
DMA	DMA from host to Moneta buffer	0.93/KB	-
Ring	Data from Moneta buffer to mem ctrl	0.28/KB	-
PCM	4 KB PCM memory access	4.39	5.18
Ring	Data from mem ctrl to Moneta buffer	-	0.43/KB
DMA	DMA from Moneta buffer to host	-	0.65/KB
Wait	Thread sleep during hw	11.8	12.3
Interrupt	Driver interrupt handler	1.10	1.08
Copy	Data copy from DMA buffer	-	0.27/KB
OS/User	OS return and userspace overhead	1.98	1.95
Hardware total for 4 KB (accounting for overlap)		8.2	8.0
Software total for 4 KB (accounting for overlap)		13.3	12.2
File system additional overhead		5.8	4.2



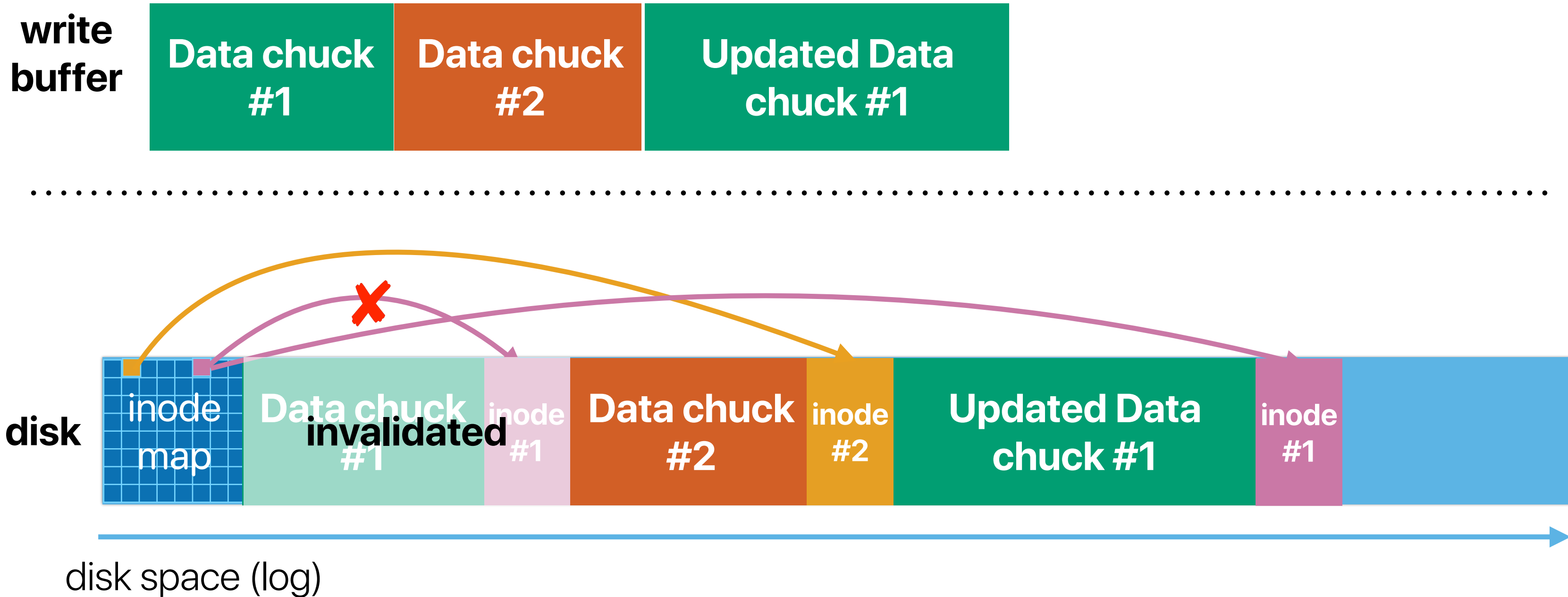
A. M. Caulfield, A. De, J. Coburn, T. I. Mollow, R. K. Gupta and S. Swanson, "Moneta: A High-Performance Storage Array Architecture for Next-Generation, Non-volatile Memories," 2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture, 2010, pp. 385-395, doi: 10.1109/MICRO.2010.33.

Don't stack your log on my log

**Jingpei Yang, Ned Plasson, Greg Gillis, Nisha Talagala, and
Swaminathan Sundararaman
SanDisk Corporation**



LFS in motion

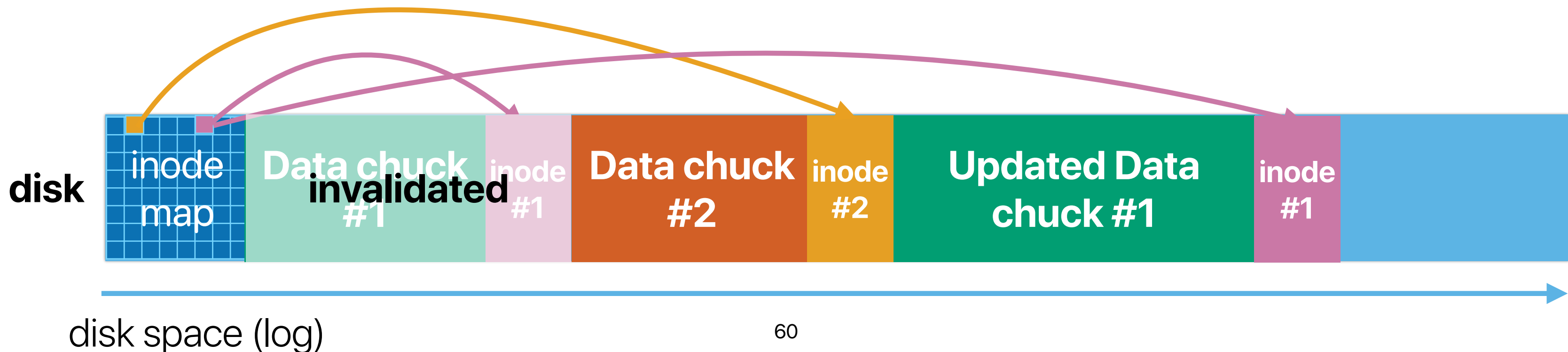


What's the good things and bad things about "Logs"

Pros/Cons of Logs

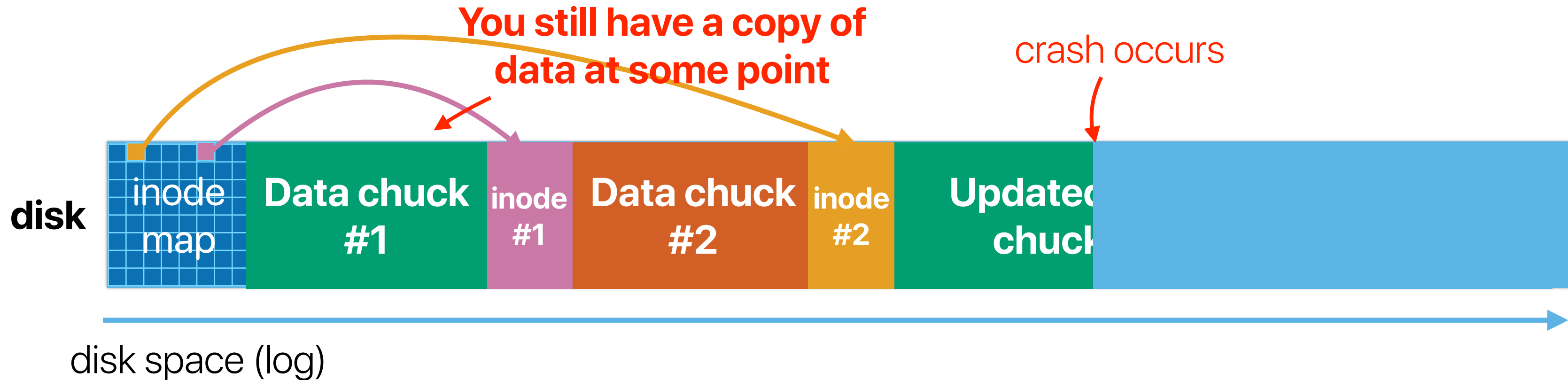
Log

- An append only data structure that records all changes
- Advantages of logs
 - Better performance — always sequential accesses
 - Faster writes — you just need to append without sanitize existing data first
 - Ease of recovery — you can find previous changes within the log
- Disadvantage of logs — you will need to explicit perform garbage collections to reclaim spaces

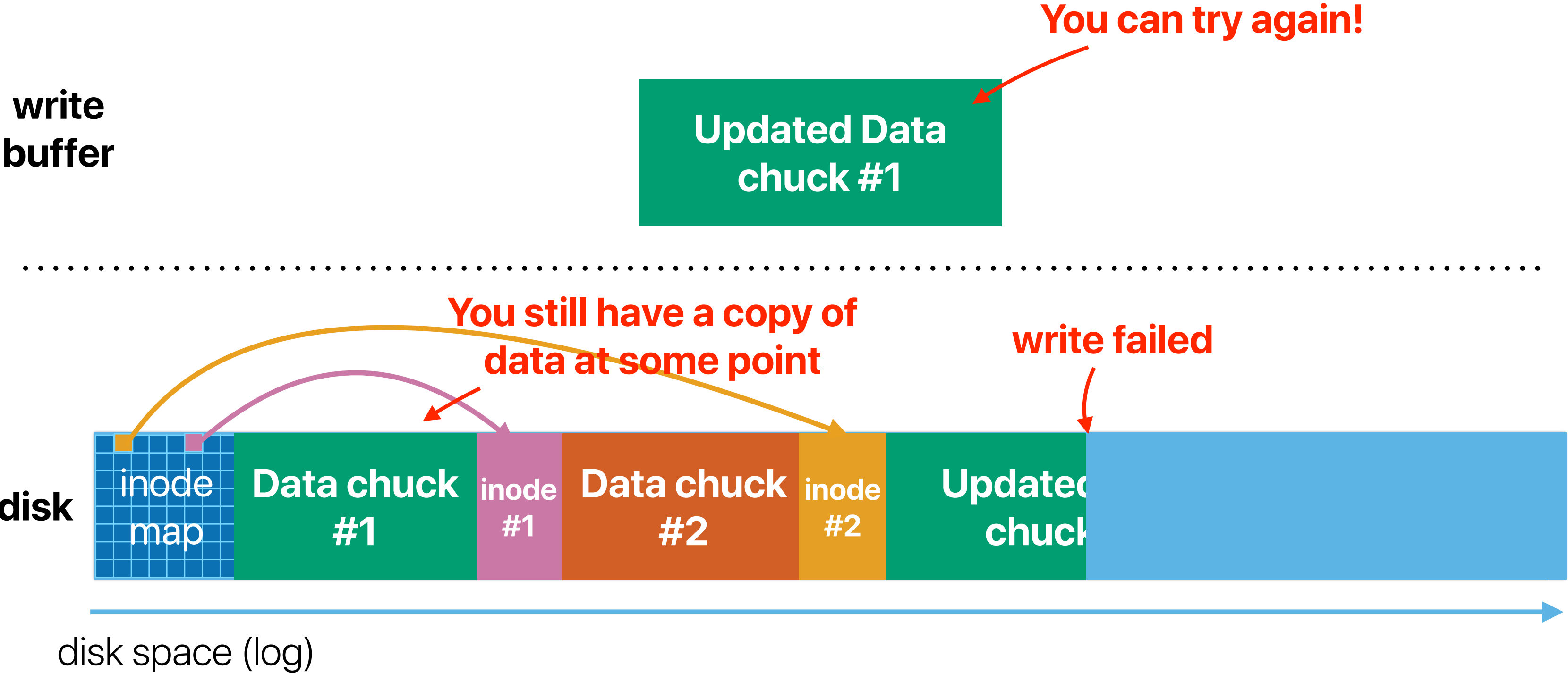


LFS v.s. crash

write
buffer

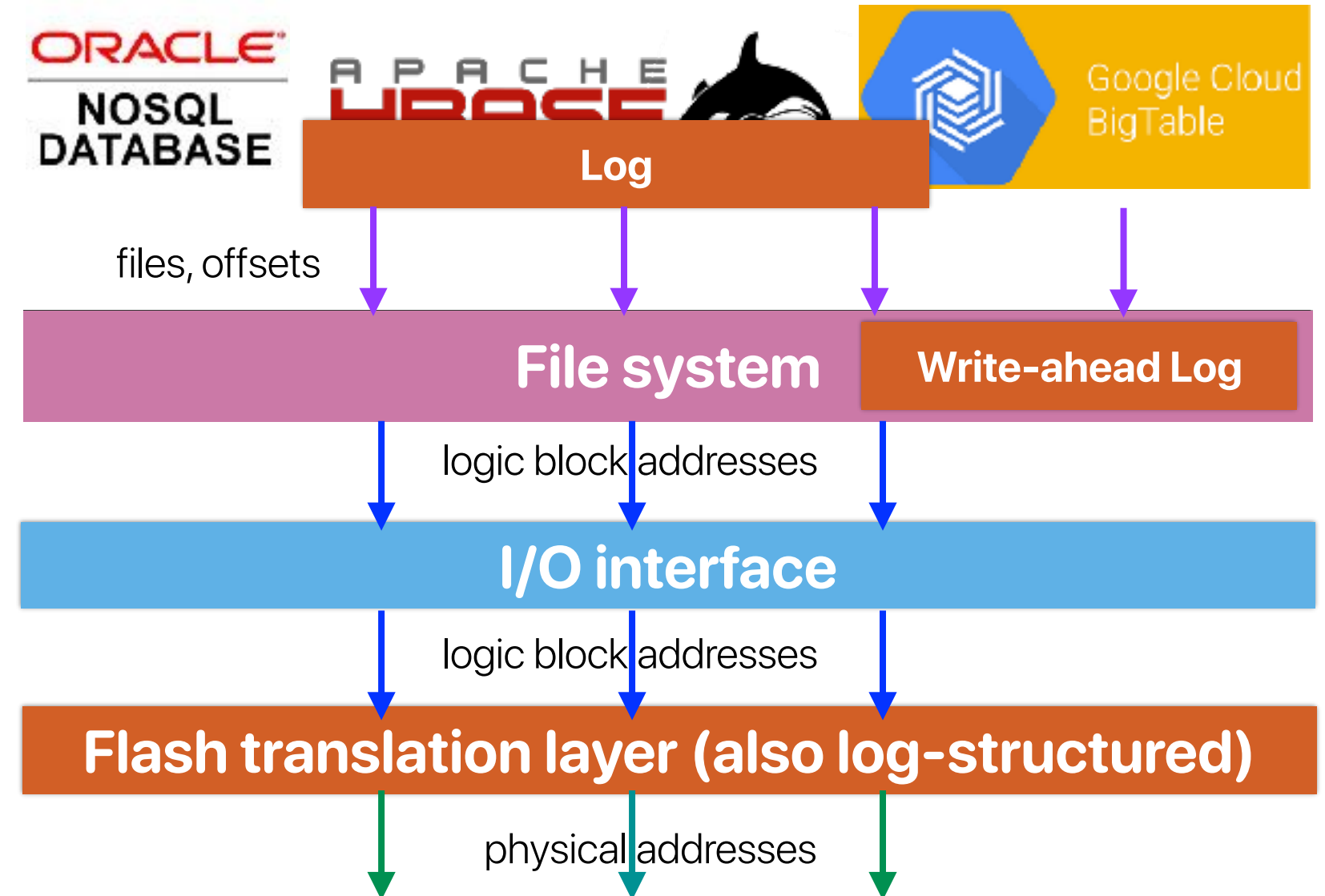


LFS v.s. write failed

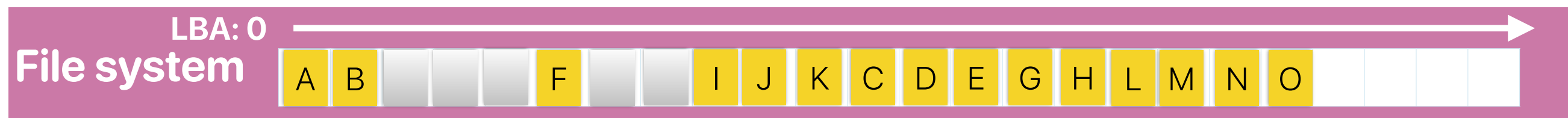


Why should we care about this paper?

- Log is everywhere
 - Application: database
 - File system
 - Flash-based SSDs
- They can interfere with each other!
- An issue with software engineering nowadays



For example, garbage collection

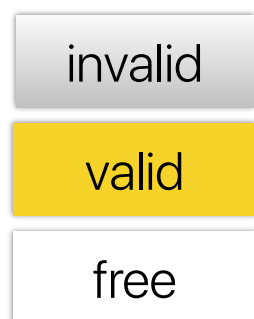
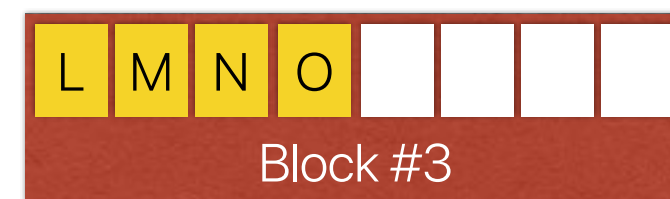
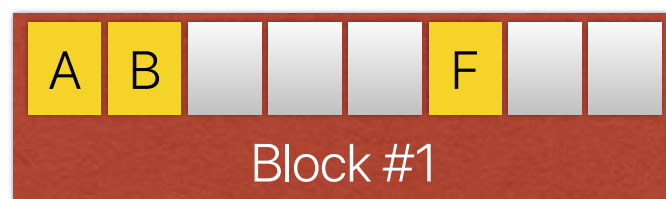


logic block addresses

I/O interface

logic block addresses

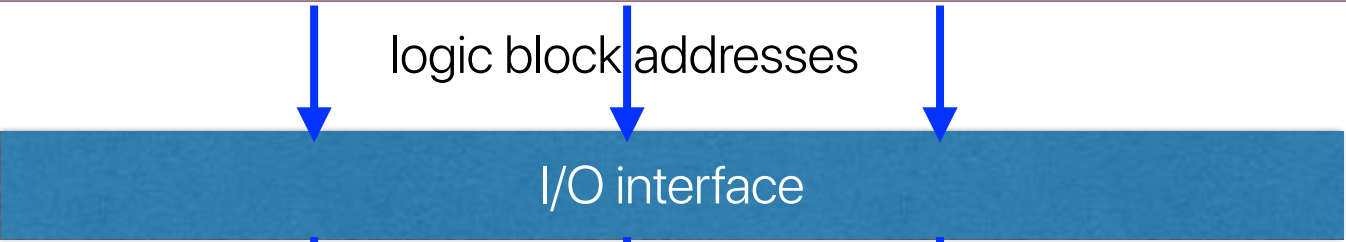
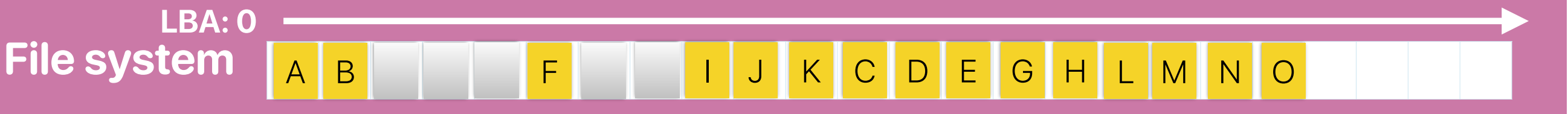
SSD



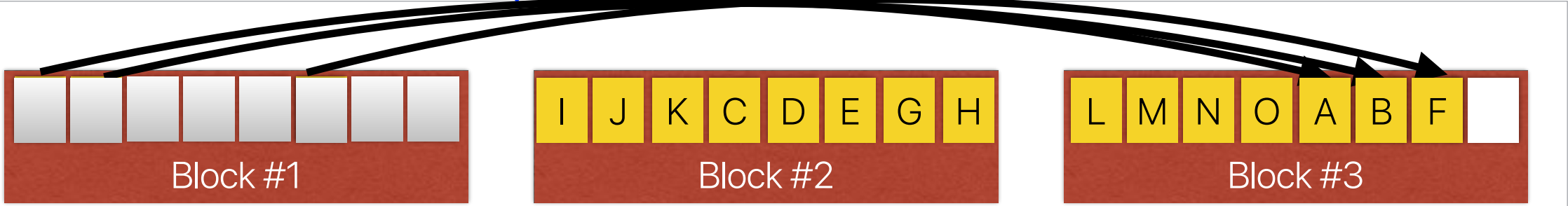
FTL mapping table

LBA	block #	page #
0	1	0
1	1	1
2	-	-
3	-	-
4	-	-
5	1	5
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	-	-
21	-	-
22	-	-
23	-	-

Now, SSD wants to reclaim a block



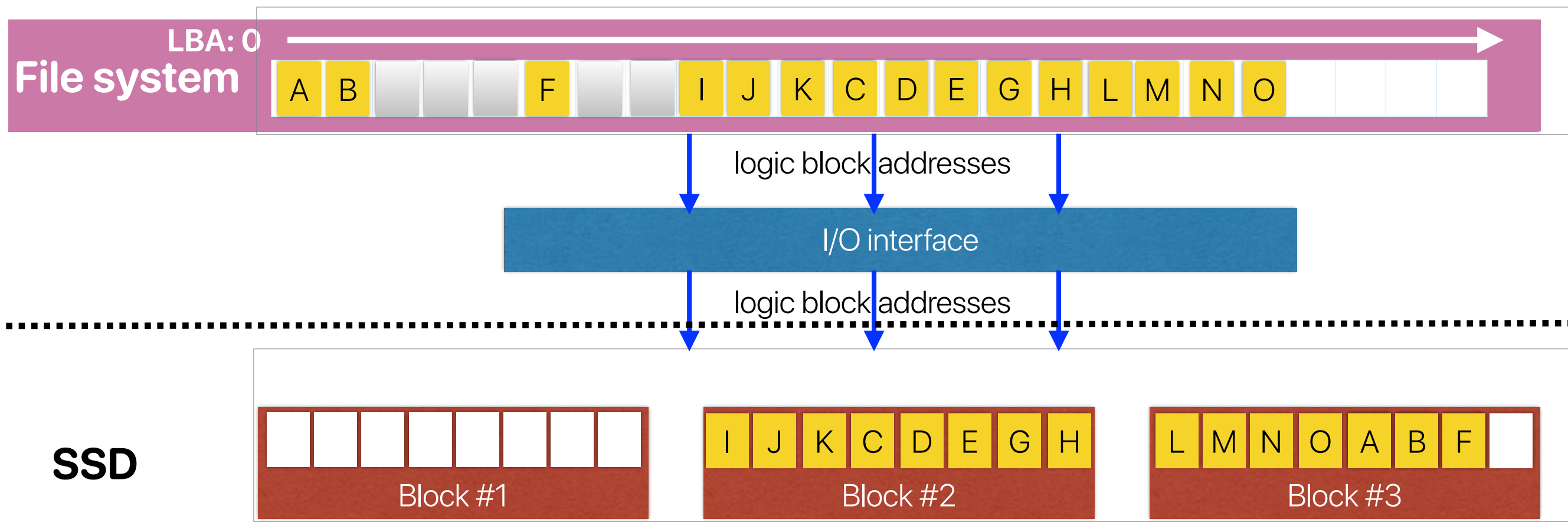
SSD



FTL mapping table

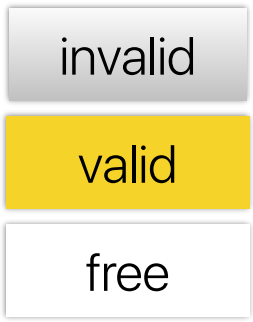
LBA	block #	page #
0	3	4
1	3	5
2	-	-
3	-	-
4	-	-
5	3	6
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	-	-
21	-	-
22	-	-
23	-	-

Garbage collection on the SSD done!

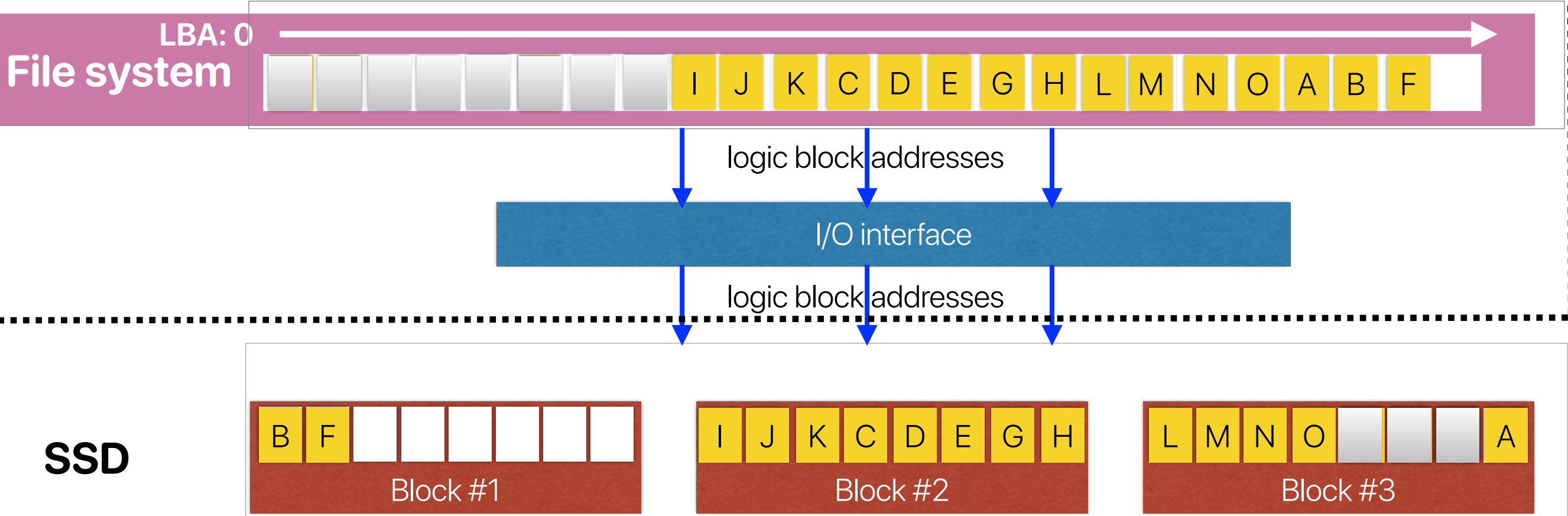


FTL mapping table

LBA	block #	page #
0	3	4
1	3	5
2	-	-
3	-	-
4	-	-
5	3	6
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	-	-
21	-	-
22	-	-
23	-	-

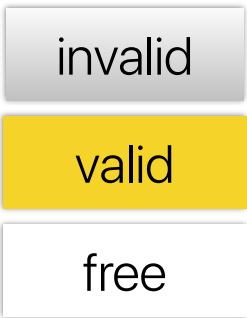


What will happen if the FS wants to perform GC?



FTL mapping table		
LBA	block #	page #
0	-	-
1	-	-
2	-	-
3	-	-
4	-	-
5	-	-
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	3	7
21	1	0
22	1	1
23	-	-

We could have avoided writing the stale A, B, F if they are coordinated!



All problems in computer science can be solved by another level of
indirection

—David Wheeler

...except for the problem of too many layers of indirection.

File systems for flash-based SSDs

- Still an open research question
- Software designer should be aware of the characteristics of underlying hardware components
- Revising the layered design to expose more SSD information to the file system or the other way around

BGR

TECH

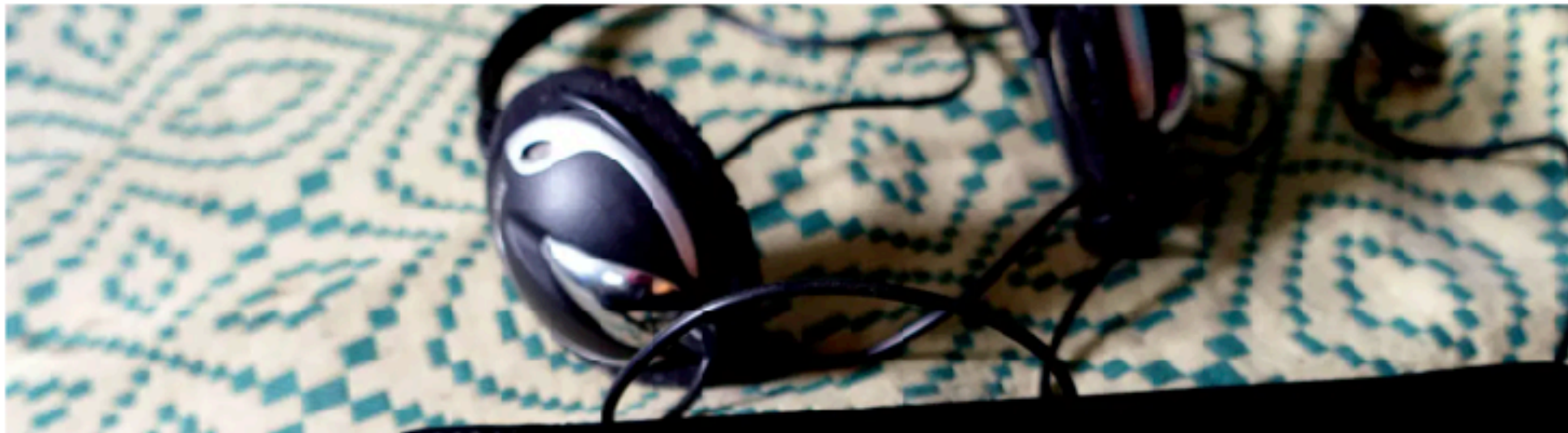
ENTERTAINMENT

DEALS

BUSINESS

TECH

Spotify has been quietly killing your SSD's life



Apple M1 Macs appear to be chewing through their SSDs

By Alan Dexter 4 hours ago

The same SSDs that are soldered-in and almost impossible to rep

COMMENTS



(Image credit: Apple)

CPU-centric programming model

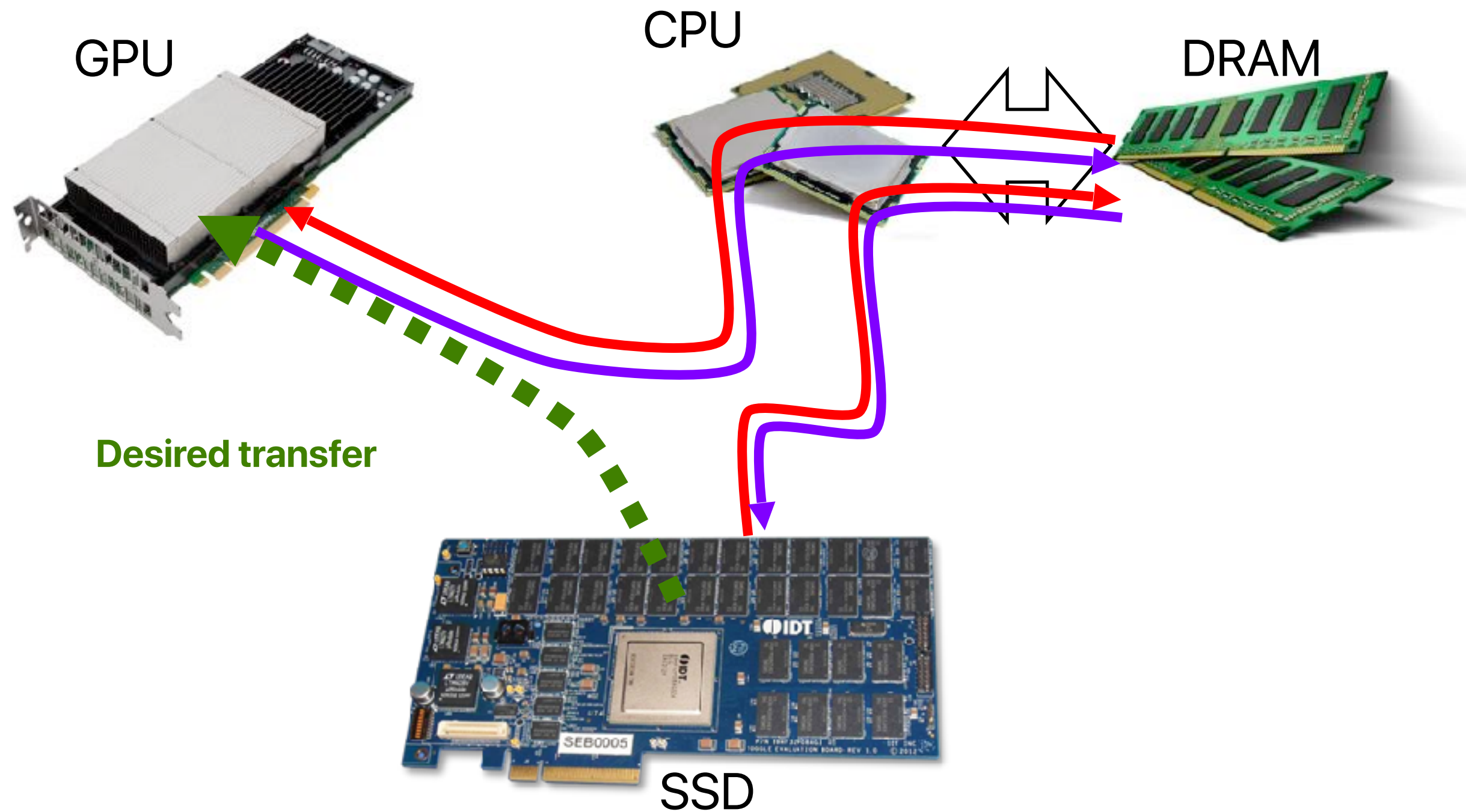
Moving data

```
1: m = (float*) malloc(sizeof(float)*matrix_dim*matrix_dim); // allocate m
2: fp = open(filename, O_RDONLY|O_DIRECT); // open file
3: read(fp,m,sizeof(float)*matrix_dim*matrix_dim); // read file
4: close(fp);
   // allocate GPU memory
5: cudaMalloc((void*)&d_m, matrix_dim*matrix_dim*sizeof(float));
   //copy from main memory to GPU memory
6: cudaMemcpy(d_m, m, matrix_dim*matrix_dim*sizeof(float),
             cudaMemcpyHostToDevice);
7: lud_diagonal<<<1,BLOCK_SIZE>>>(d_m, matrix_dim, i); GPU computing
```

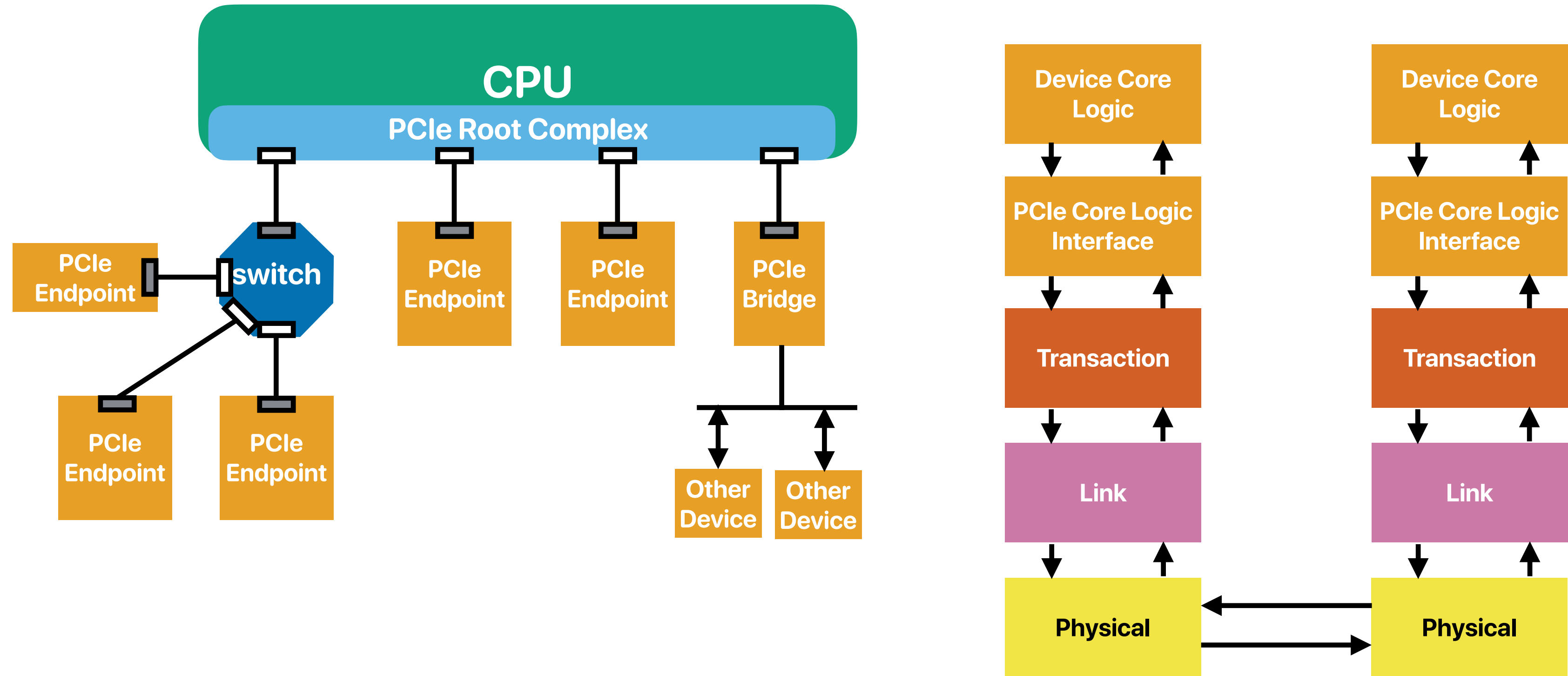
```
//copy from GPU memory to main memory
8: cudaMemcpy(m, d_m, matrix_dim*matrix_dim*sizeof(float),
             cudaMemcpyDeviceToHost);
9: output_matrix(m, matrix_dim);
```

Moving data

GPU-accelerated architecture



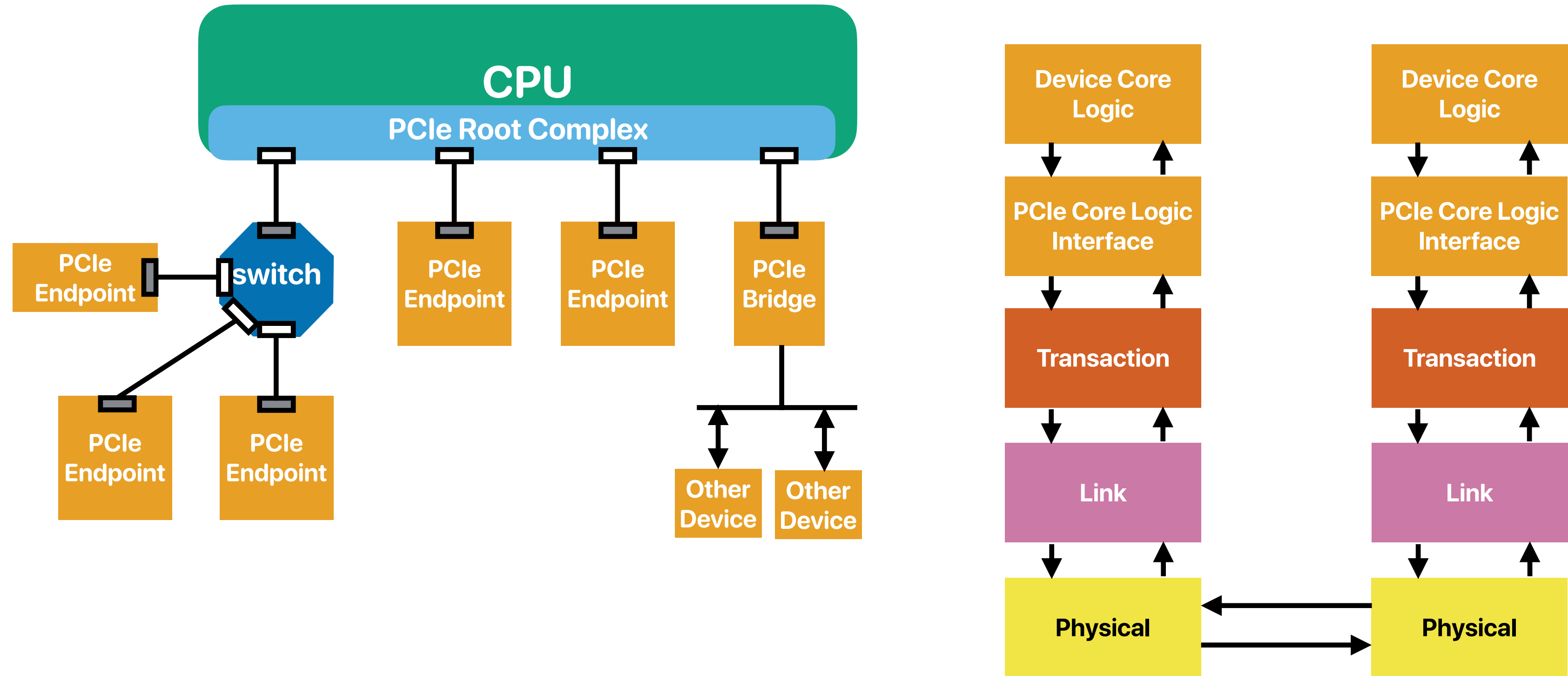
PCIe "Interconnect"



**How can you let SSDs to
communicate with GPUs directly &
why does SSD-GPU P2P make sense?**

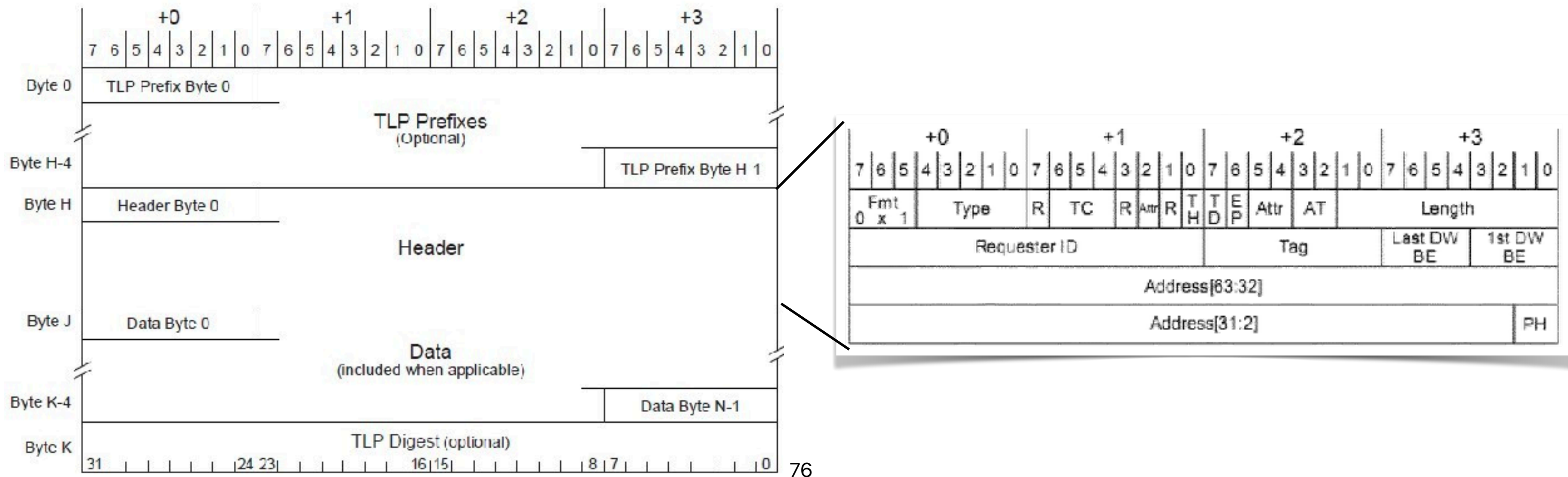
How to implement GPU-SSD P2P?

PCIe "Interconnect"

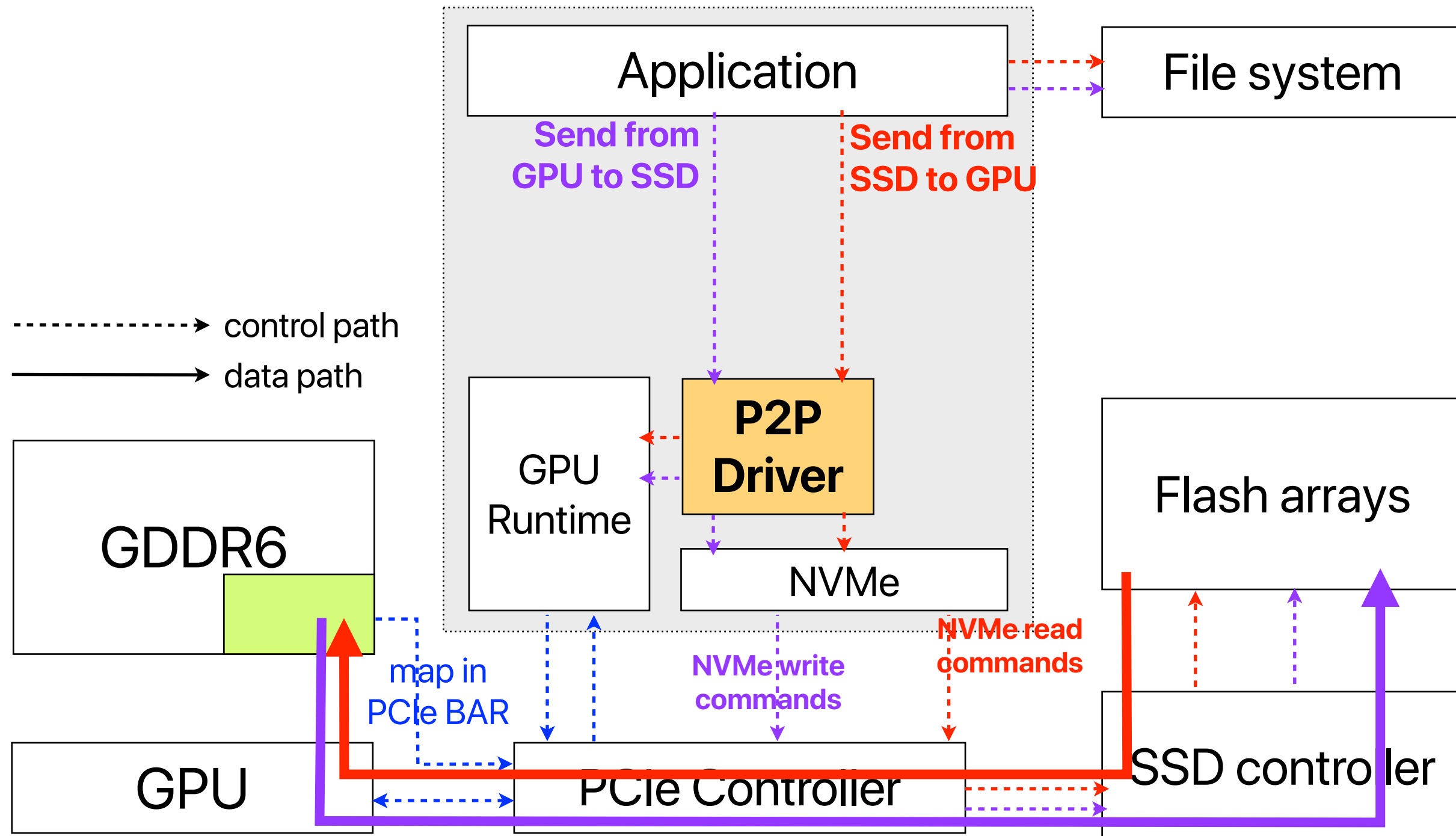


PCIe interconnect

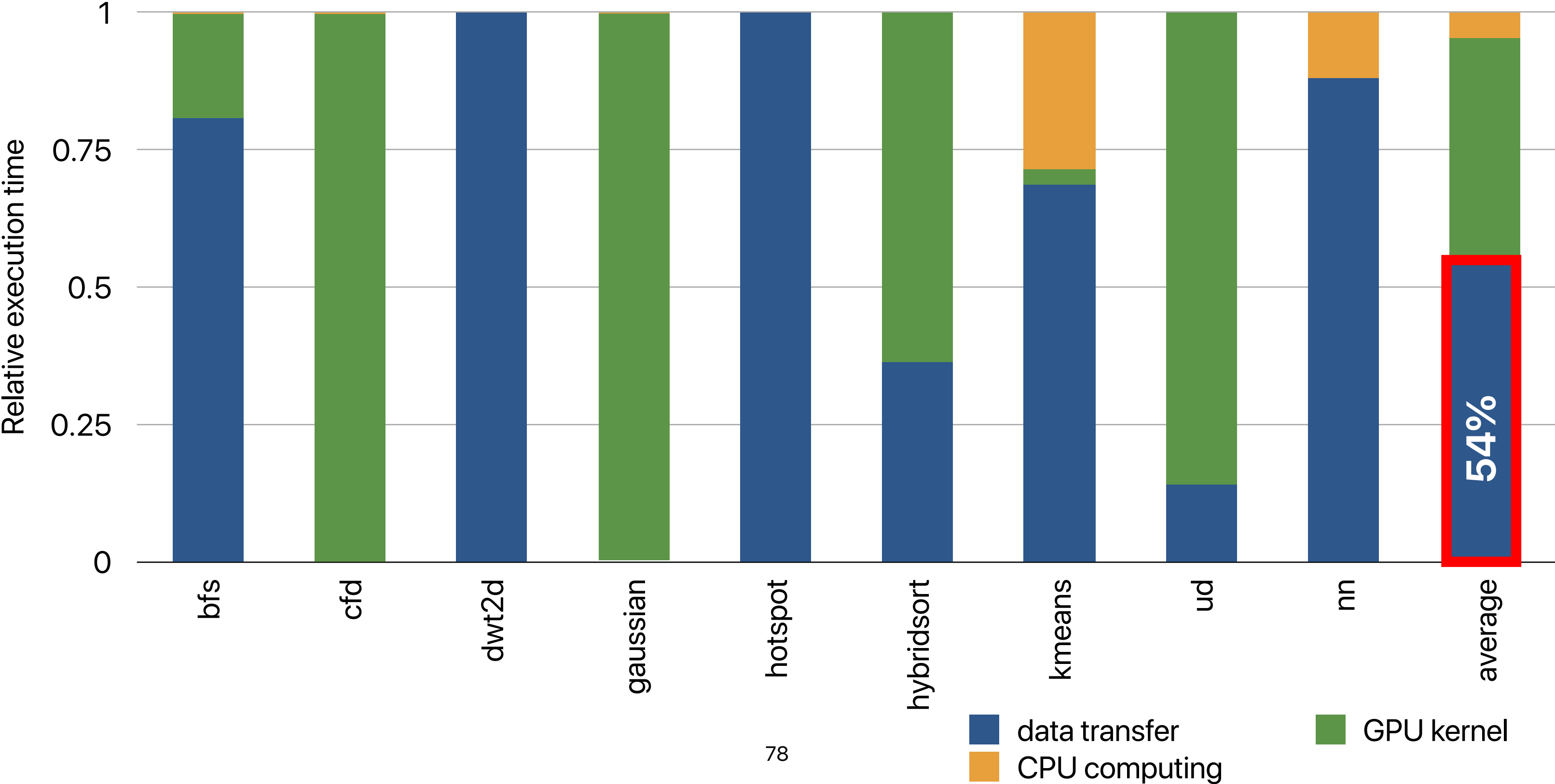
- Very similar to computer networks
- Use "memory addresses" as the identifier for routing
- Peer-to-peer communication is possible



P2P between GPU and SSD



Cost of moving data



Code revisited

Moving data

storing input in m

```
1: m = (float*) malloc(sizeof(float)*matrix_dim*matrix_dim); // allocate m
2: fp = open(filename, O_RDONLY|O_DIRECT); // open file
3: read(fp, m, sizeof(float)*matrix_dim*matrix_dim); // read file
4: close(fp);
   // allocate GPU memory
5: cudaMalloc((void**)&d_m, matrix_dim*matrix_dim*sizeof(float));
   //copy from main memory to GPU memory
6: cudaMemcpy(d_m, m, matrix_dim*matrix_dim*sizeof(float),
             cudaMemcpyHostToDevice);
```

copying m to GPU memory

```
7: lud_diagonal<<<1, BLOCK_SIZE>>>(d_m, matrix_dim, i); GPU computing
```

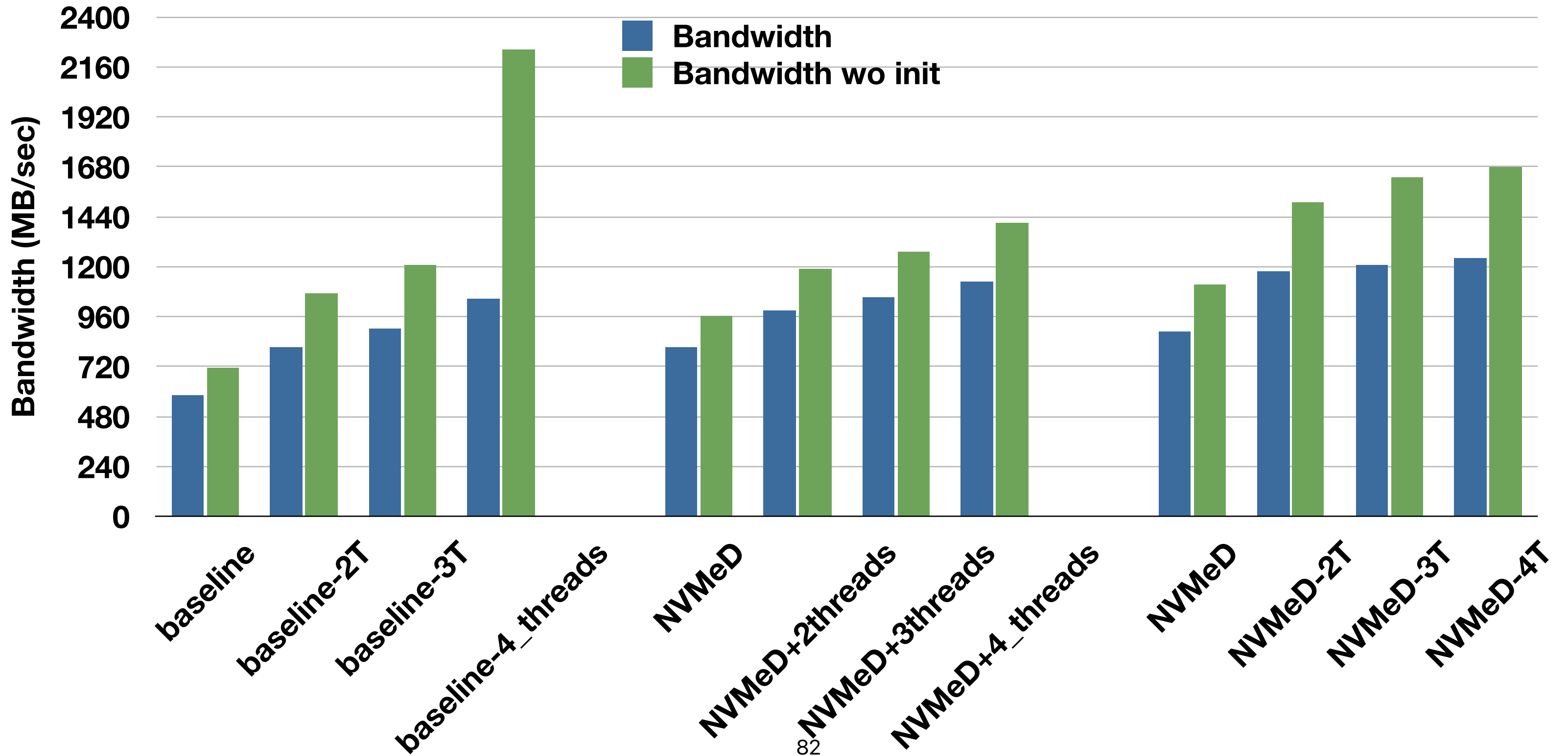
```
//copy from GPU memory to main memory
8: cudaMemcpy(m, d_m, matrix_dim*matrix_dim*sizeof(float),
             cudaMemcpyDeviceToHost);
9: output_matrix(m, matrix_dim);
```

Moving data

m is overwritten, previous m is
only used as a "buffer"



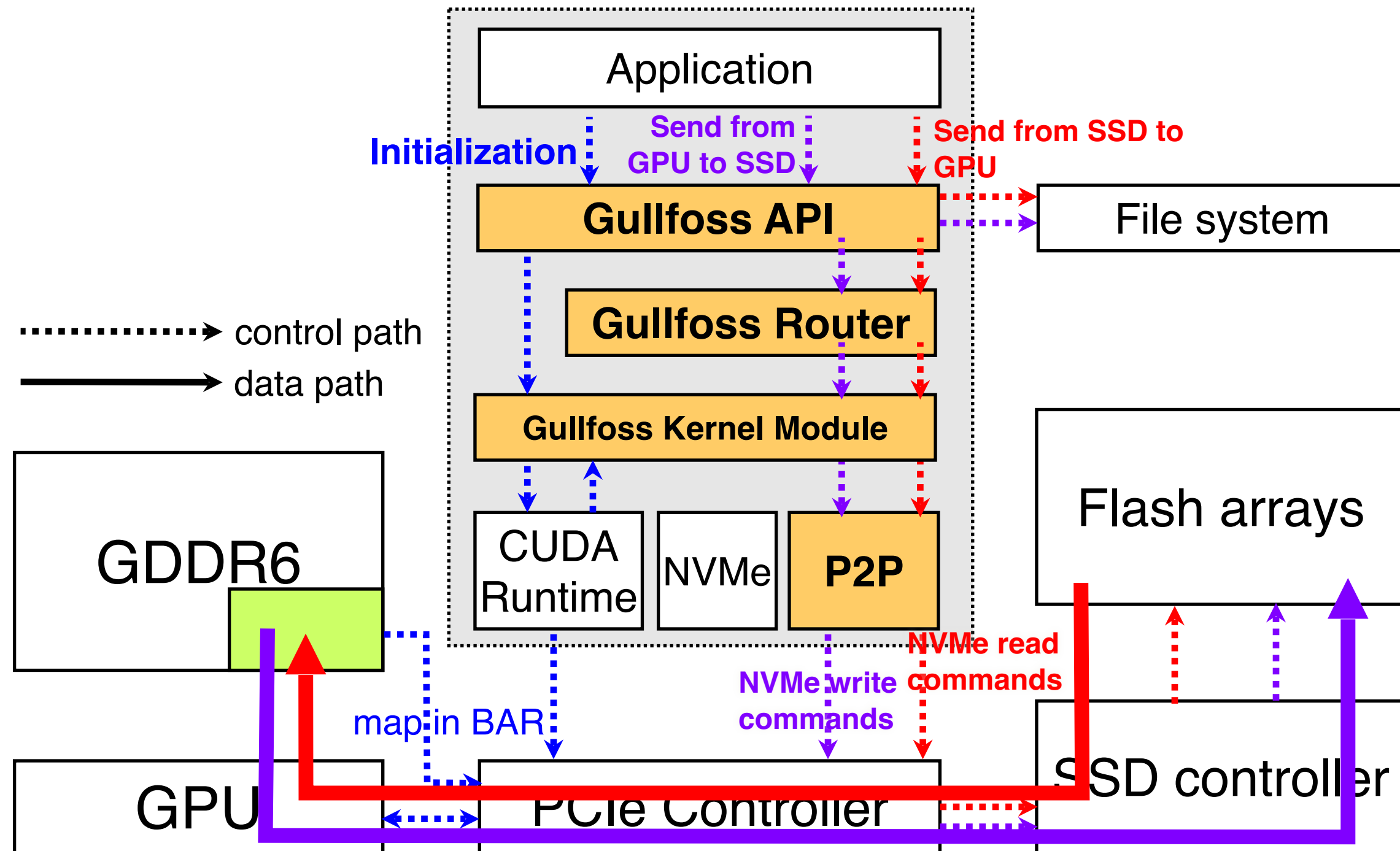
Multi-threaded I/O helps



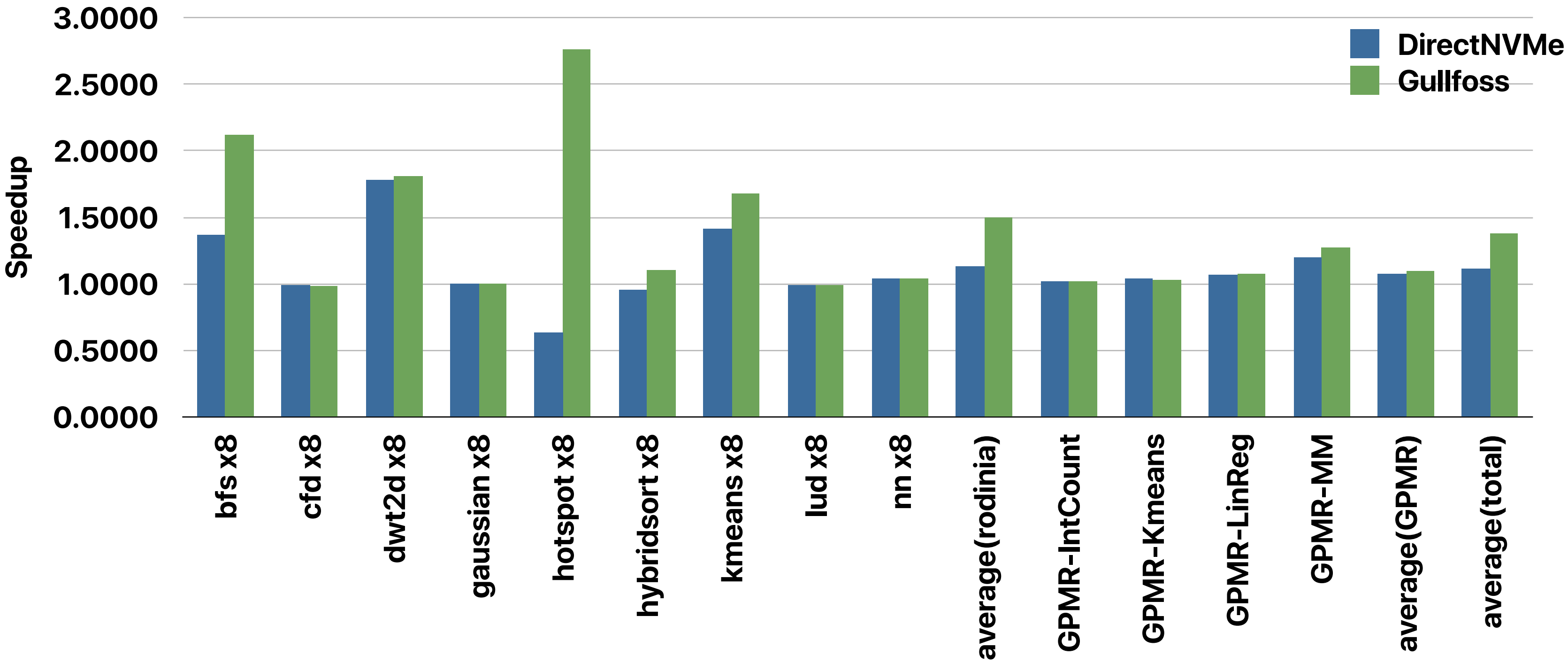
When can GPU-SSD P2P be a bad idea?

When doesn't GPU-SSD P2P make sense?

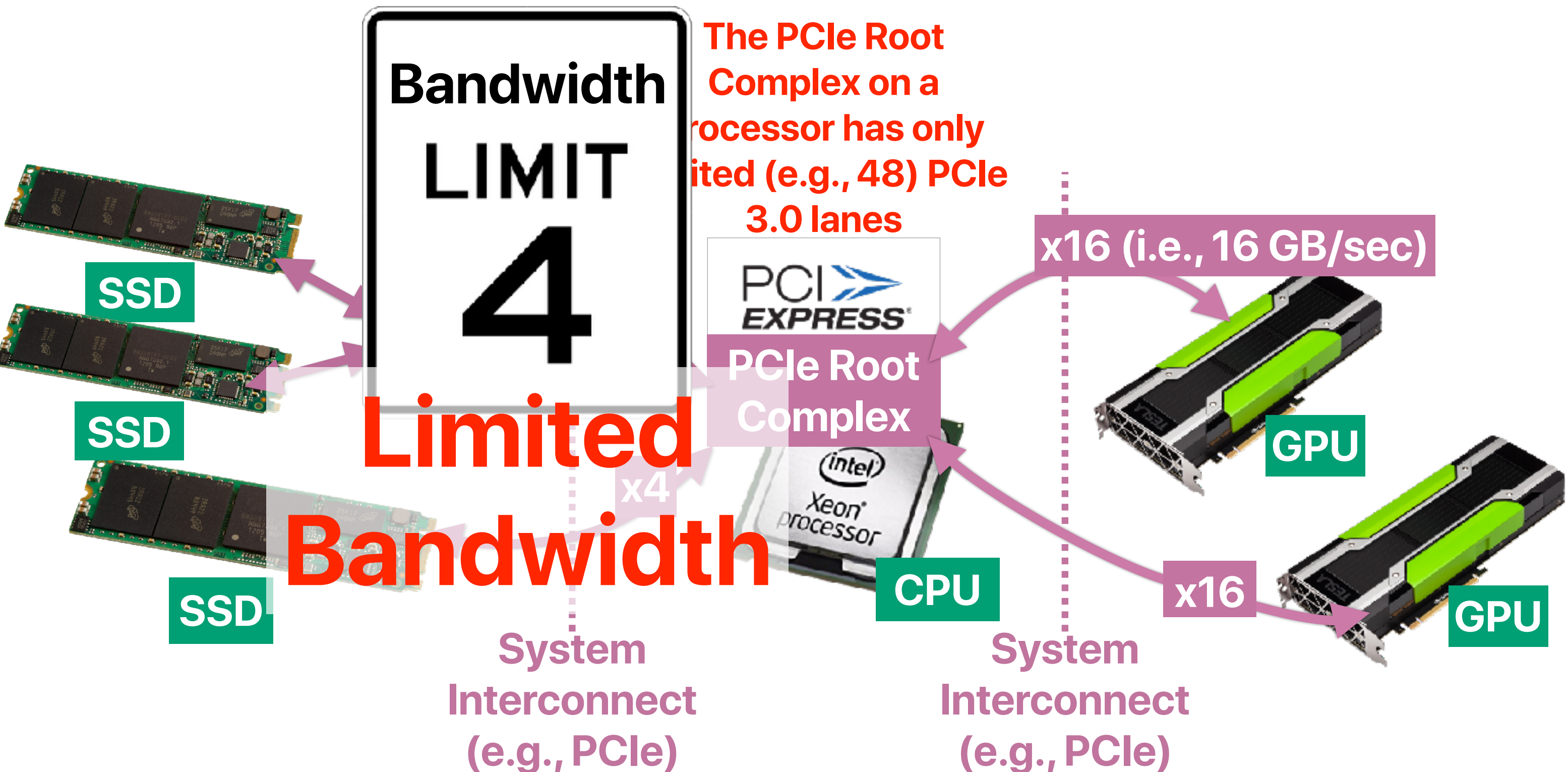
Gulfoss: decouples control/data planes



GPU-SSD P2P + traditional route and main memory buffers



Limited Interconnect Bandwidth

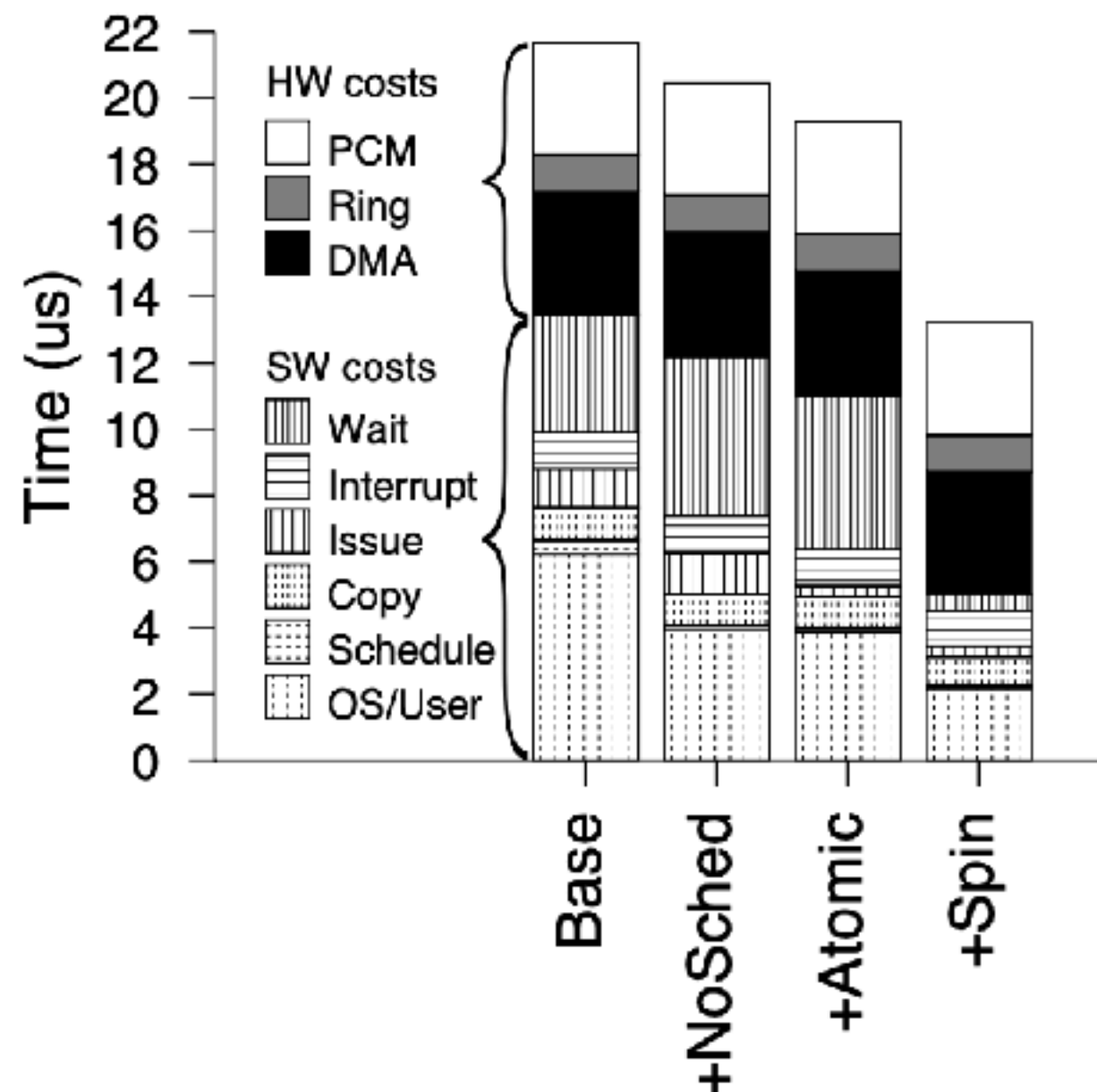


If interconnect bandwidth is the bottleneck, how can we improve over there?



Software overhead

Label	Description	Baseline latency (μ s)	
		Write	Read
OS/User	OS and userspace overhead	1.98	1.95
OS/User	Linux block queue and no-op scheduler	2.51	3.74
Schedule	Get request from queue and assign tag	0.44	0.51
Copy	Data copy into DMA buffer	0.24/KB	-
Issue	PIO command writes to Moneta	1.18	1.15
DMA	DMA from host to Moneta buffer	0.93/KB	-
Ring	Data from Moneta buffer to mem ctrl	0.28/KB	-
PCM	4 KB PCM memory access	4.39	5.18
Ring	Data from mem ctrl to Moneta buffer	-	0.43/KB
DMA	DMA from Moneta buffer to host	-	0.65/KB
Wait	Thread sleep during hw	11.8	12.3
Interrupt	Driver interrupt handler	1.10	1.08
Copy	Data copy from DMA buffer	-	0.27/KB
OS/User	OS return and userspace overhead	1.98	1.95
Hardware total for 4 KB (accounting for overlap)		8.2	8.0
Software total for 4 KB (accounting for overlap)		13.3	12.2
File system additional overhead		5.8	4.2

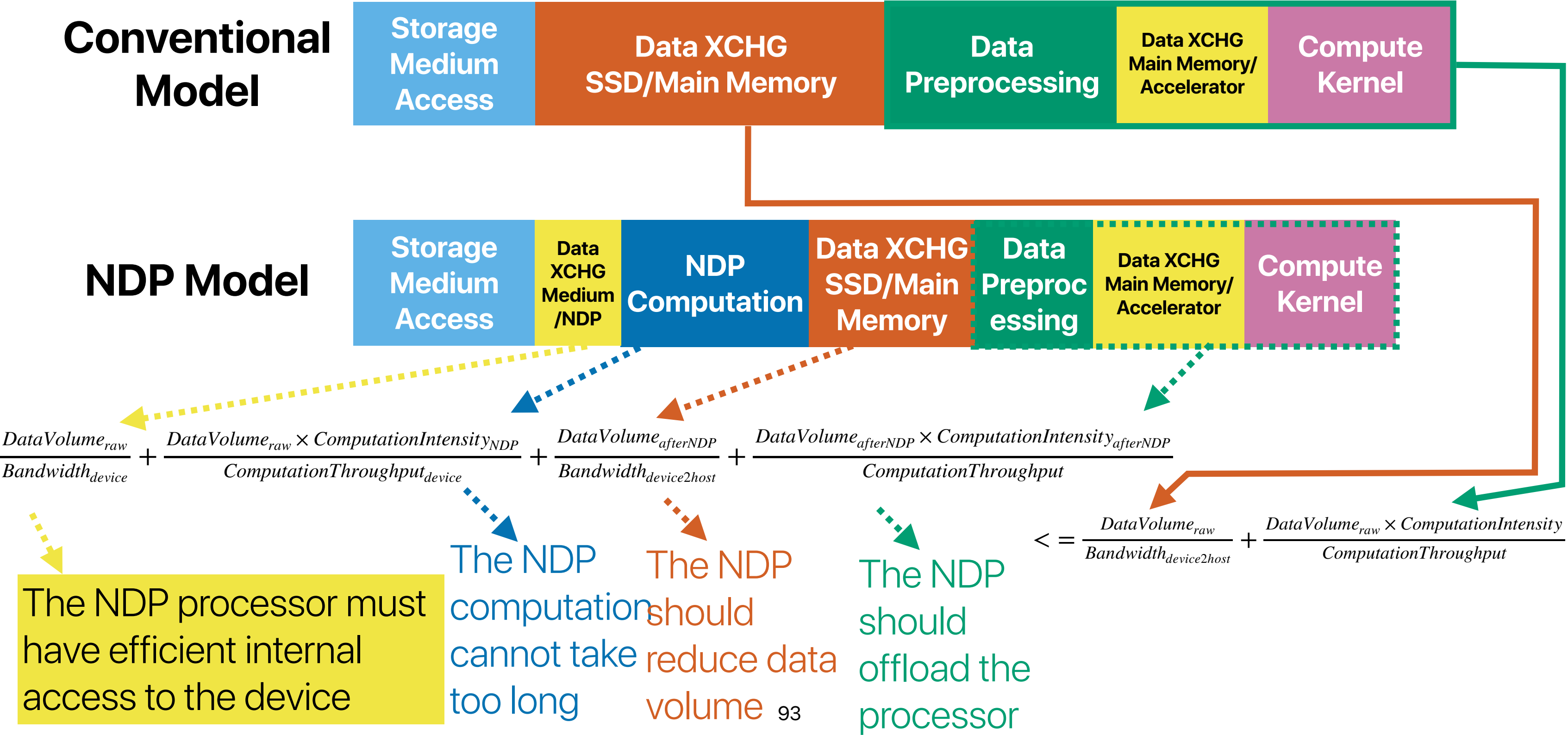


A. M. Caulfield, A. De, J. Coburn, T. I. Mollow, R. K. Gupta and S. Swanson, "Moneta: A High-Performance Storage Array Architecture for Next-Generation, Non-volatile Memories," 2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture, 2010, pp. 385-395, doi: 10.1109/MICRO.2010.33.

**Where do you see the opportunity
of SmartSSDs?**

Why does SmartSSD make sense?

The “Winning Formula” of Near-Data Processing



Multi-channel to optimize bandwidth

Flashtec™ NVMe2032 and NVMe2016 Controllers

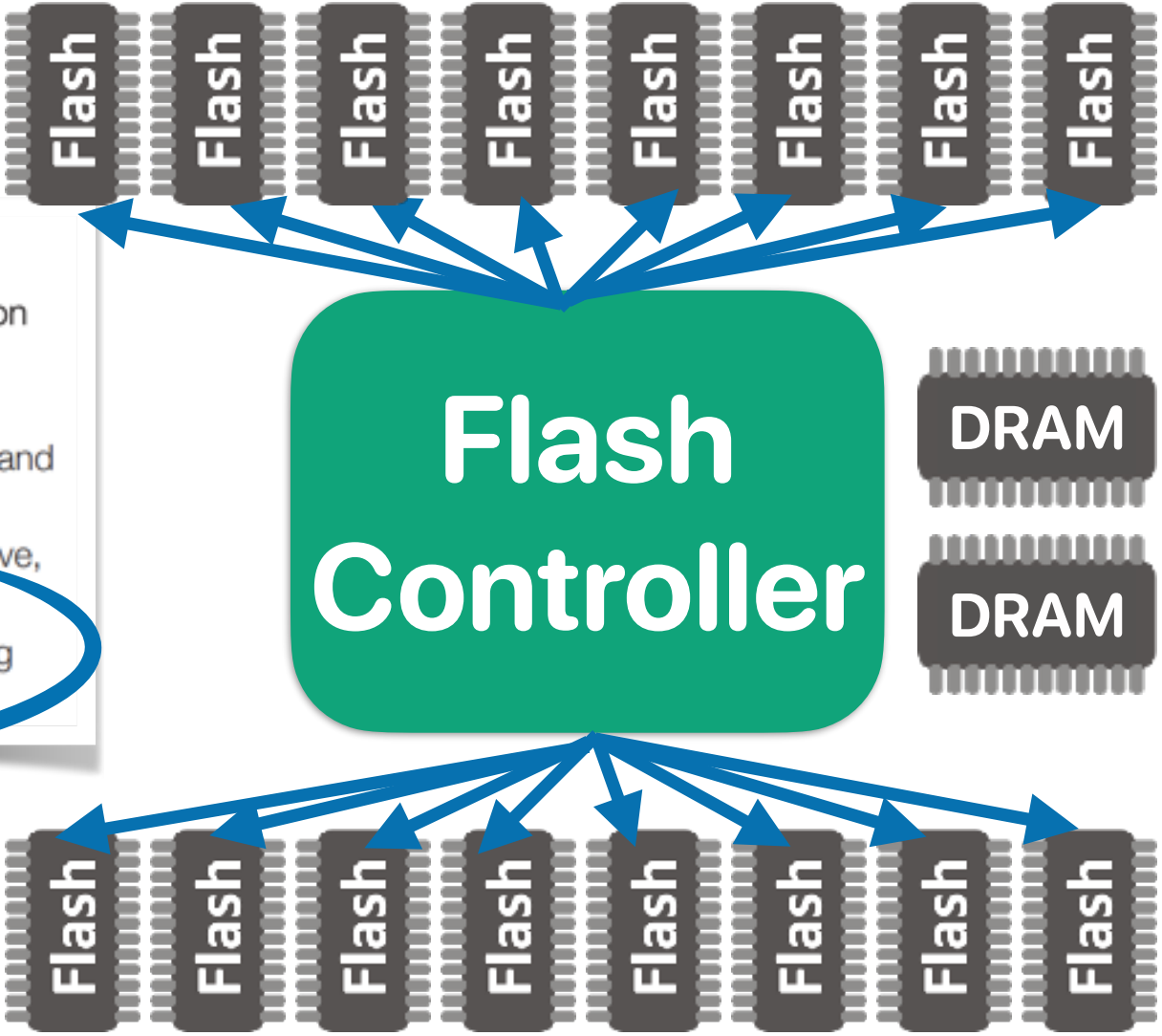
32- and 16-Channel PCIe Flash Controller Products

Summary

The Flashtec™ 2nd generation NVMe Controller Family enables the server and data centers to realize the highest performance SSDs utilizing new technologies. Combining world-class capacity and flexibility, the Flashtec NVMe2032 and NVMe2016 controllers are a reliable choice. The Flashtec NVMe2032 and NVMe2016 controllers support the NVMe Express (NVMe) host interface and are optimized for high-performance operations, performing all flash management operations on-chip and processing and memory resources.

Features

- Flashtec NVMe2032 controller can achieve up to 1 million random read IOPS on 4 KB operations
- Up to 20 TB Flash capacity using 256 GB Flash
- SLC, MLC, Enterprise MLC, and TLC Flash with toggle and ONFI interface
- PCIe Gen 3 x8 or dual independent PCIe Gen 3 x4 (active, active/standby) host interface
- 16 and 32 independent Flash channels, each supporting up to 8 CE



Each ~500MB/sec
8 channels == 4GB/sec
16 channels == 16 GB/sec



- Organization
 - Page size x8: 18,592 bytes (16,384 + 2208 bytes)
 - Block size: 2304 pages, (36,864K + 4968K bytes)
 - Plane size: 4 planes x 504 blocks
 - Device size: 512Gb: 2016 blocks; 1Tb: 4032 blocks; 2Tb: 8064 blocks; 4Tb: 16,128 blocks; 8Tb: 32,256 blocks

TLC 512Gb-8Tb NAND – Array Characteristics

is suspended or ongoing					
READ PAGE operation time without/with V _{pp}	^t R	57/41	60	μs	2, 12
SNAP READ operation time	^t RSNAP	27	37	μs	
Cache read busy time	^t RCBSY	11	60	μs	2, 8, 12

The “Winning Formula” of Near-Data Processing

Conventional Model



NDP Model



- The NDP computation can help offload computation
- The NDP computation can help reduce data volume
- The NDP computation can facilitate data preprocessing
- The NDP device has larger internal bandwidth

Opportunities of SmartSSDs

SSDs already have processors inside

Parse NVMe/SATA/PCIe commands

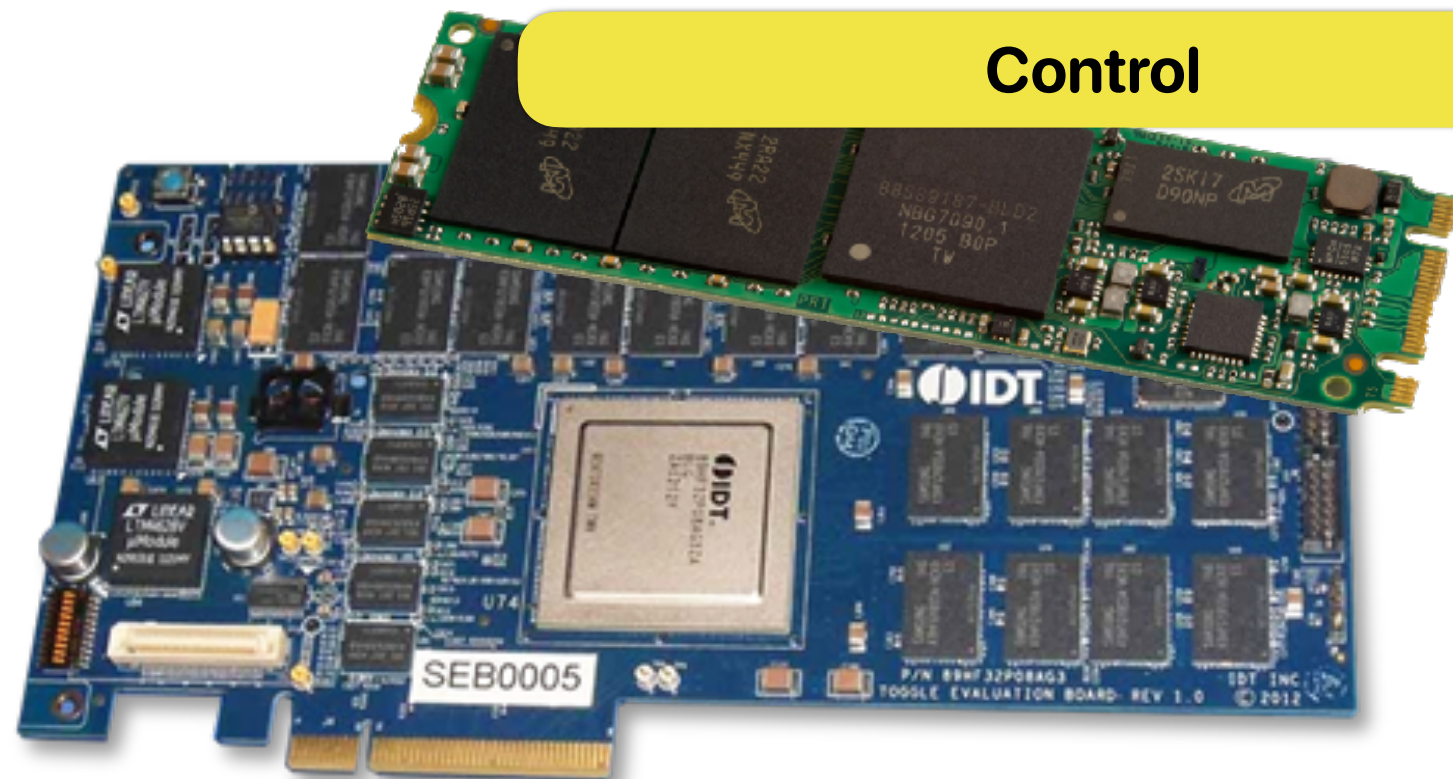
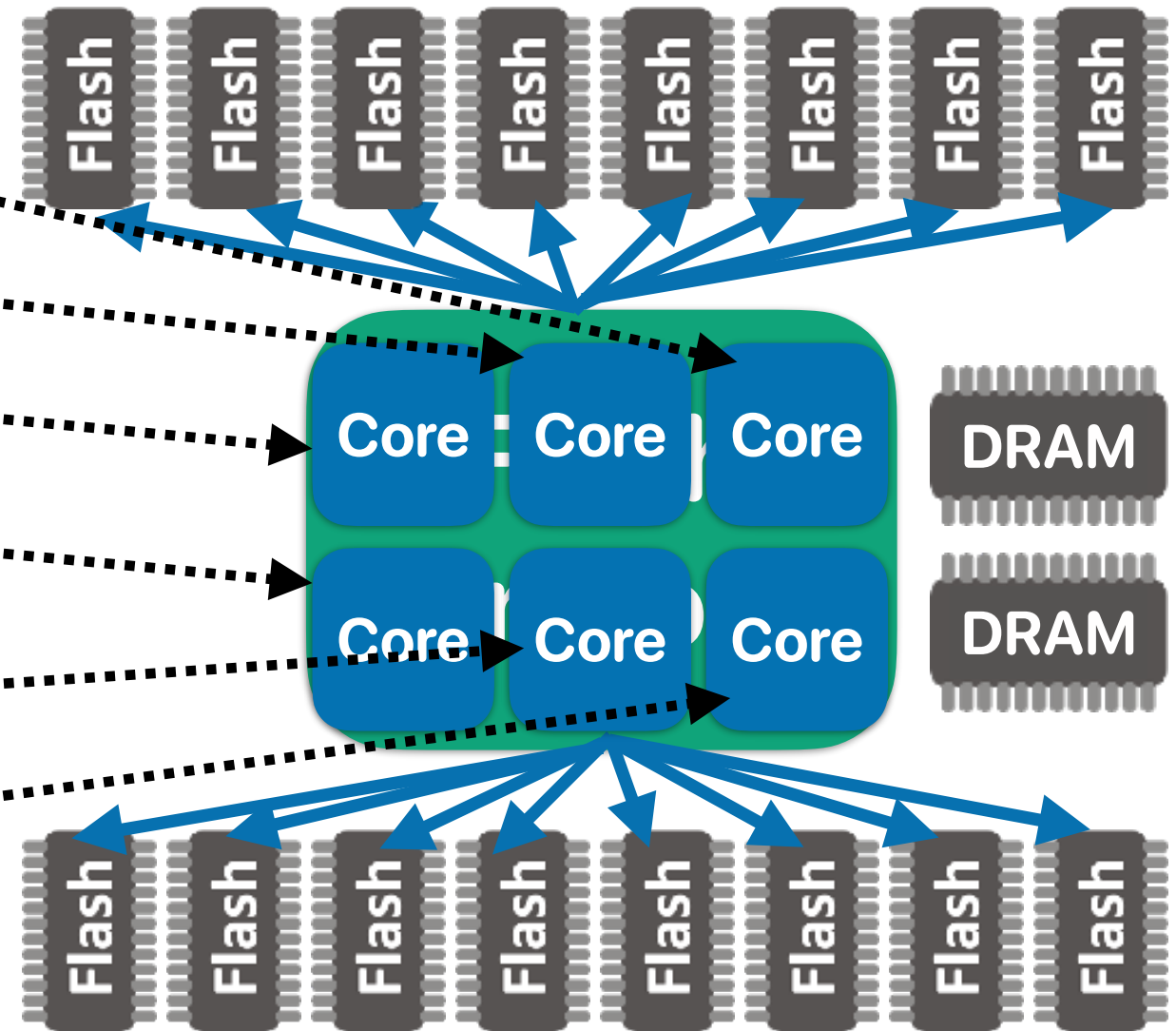
Address mapping/lookup

Wear-leveling/Garbage Collection

Read

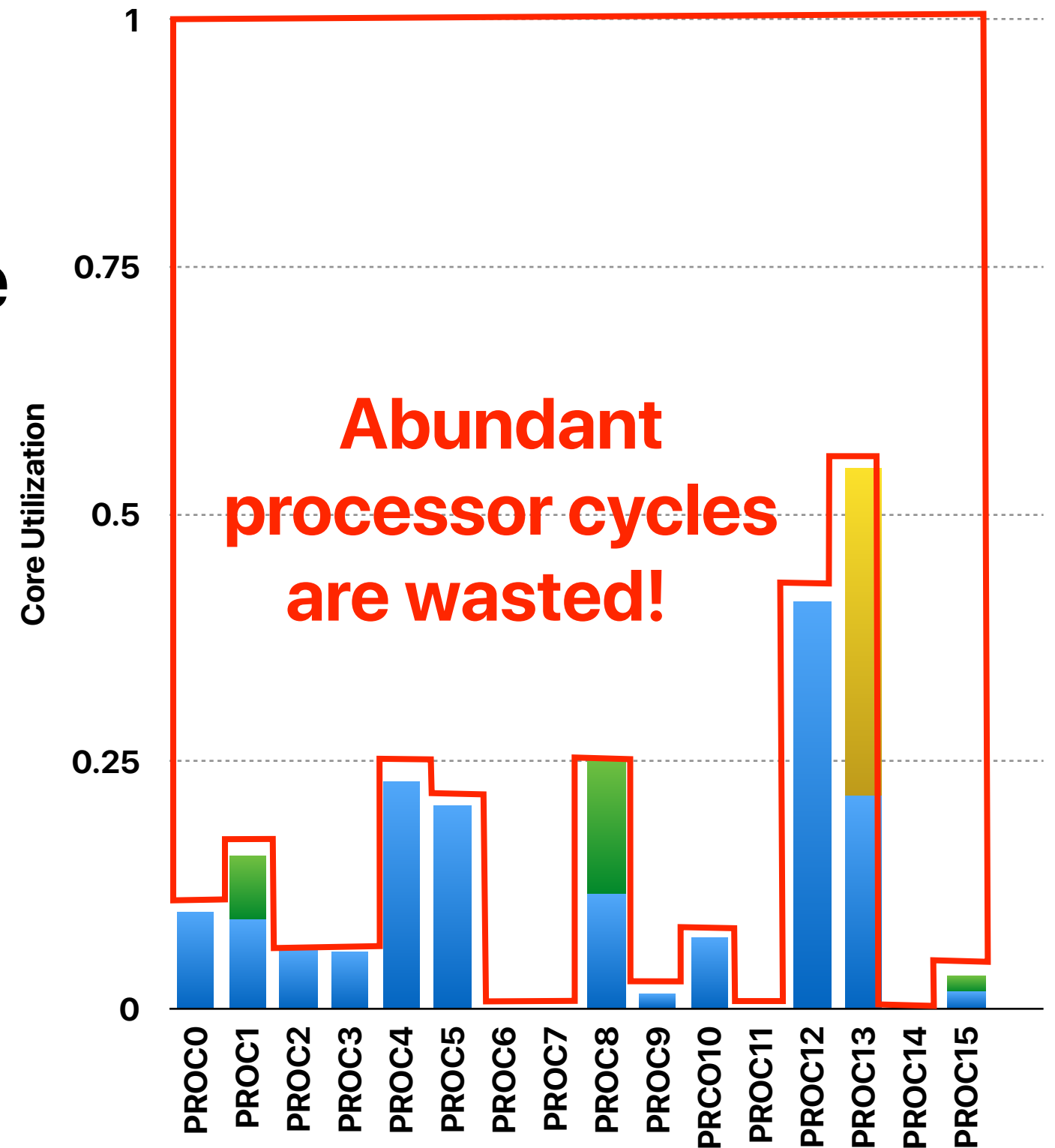
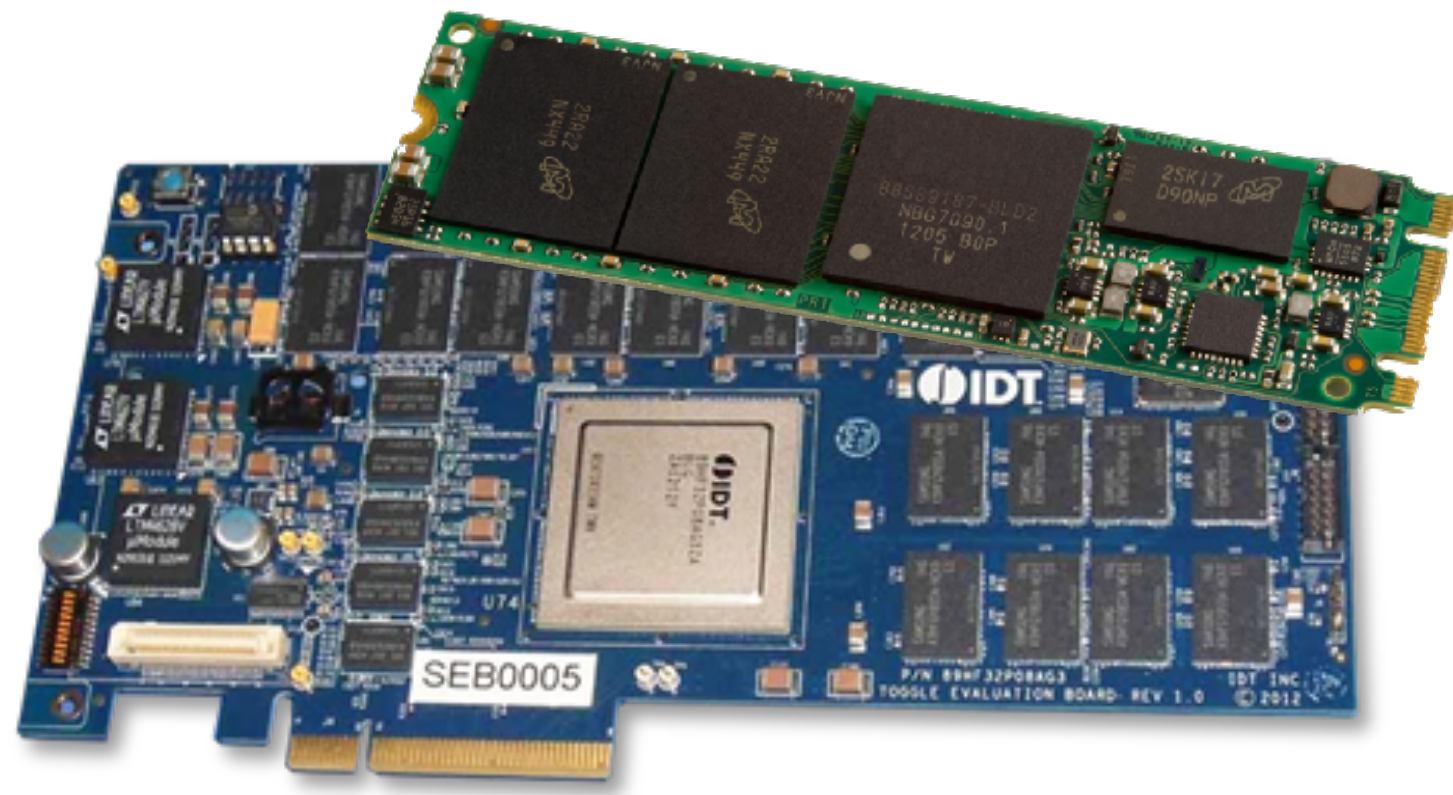
Program

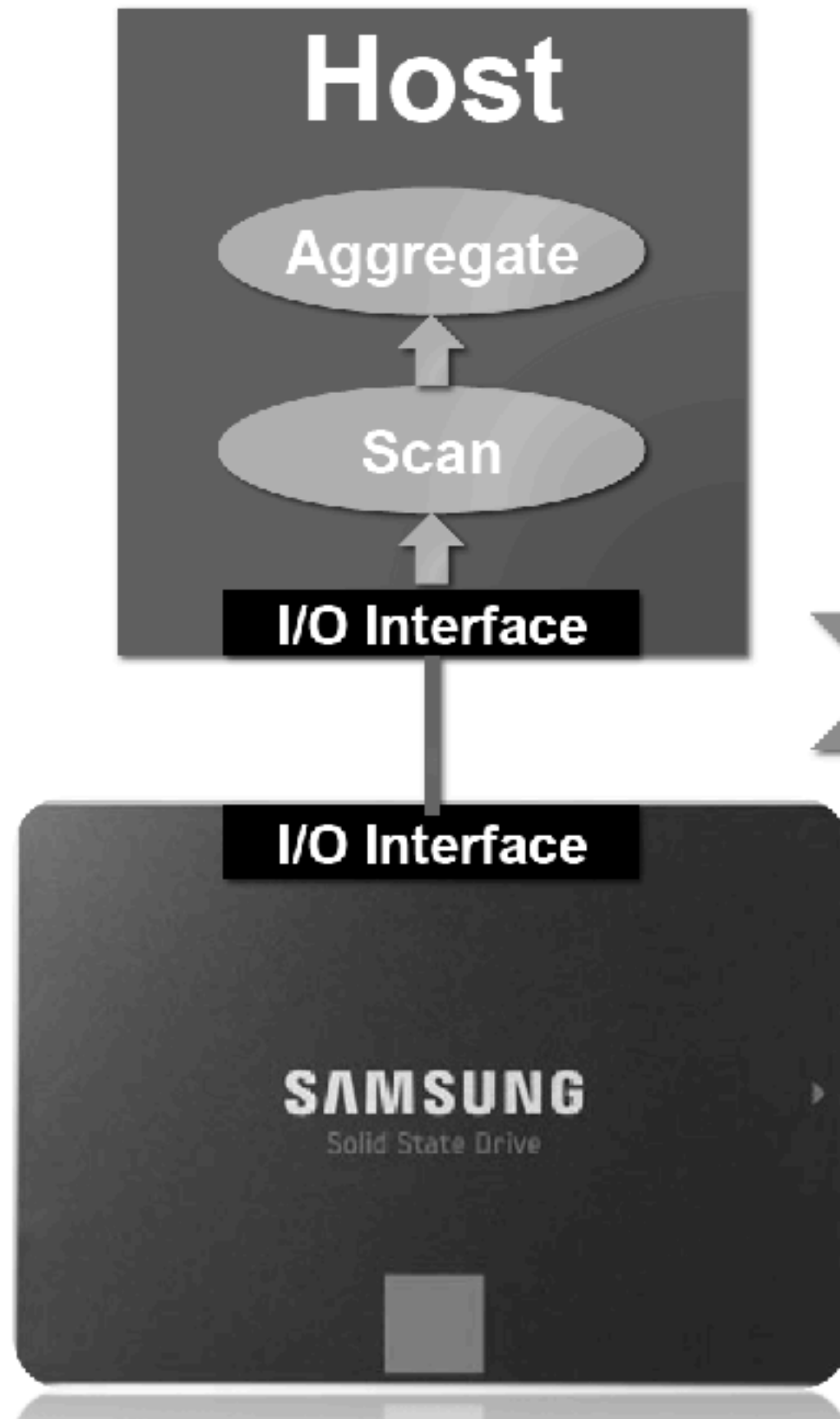
Control



SSDs already have processors inside

- Firmware does not rely on OS
- Utilization is not optimized as well
- We can "tweak" and "extend" the firmware to perform computation!





Change in the System

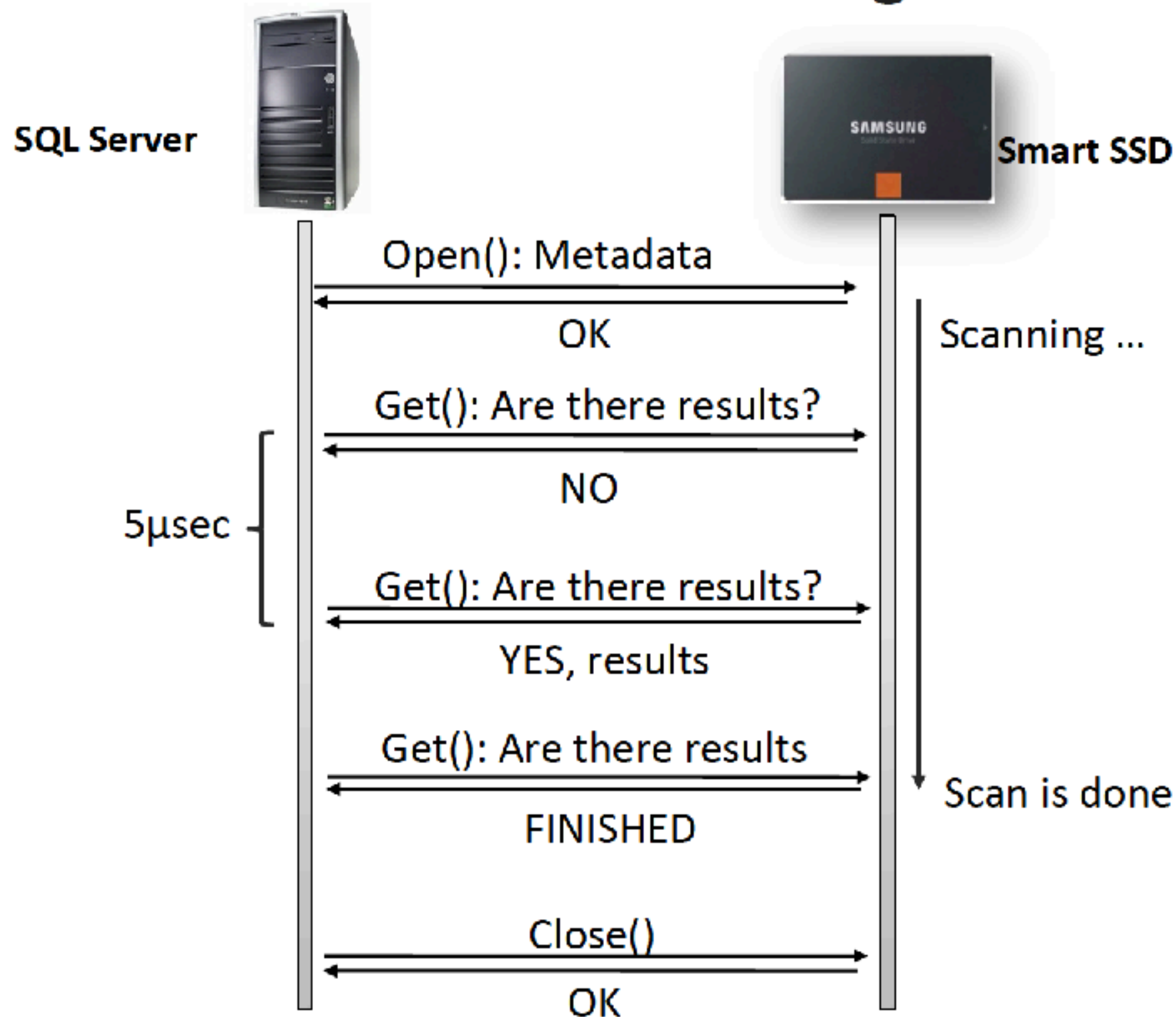
Samsung
Smart
SSD

- **New Smart SSD APIs** to run database operations inside the SSD
- Open(), Get(), Close(), SendToHost()

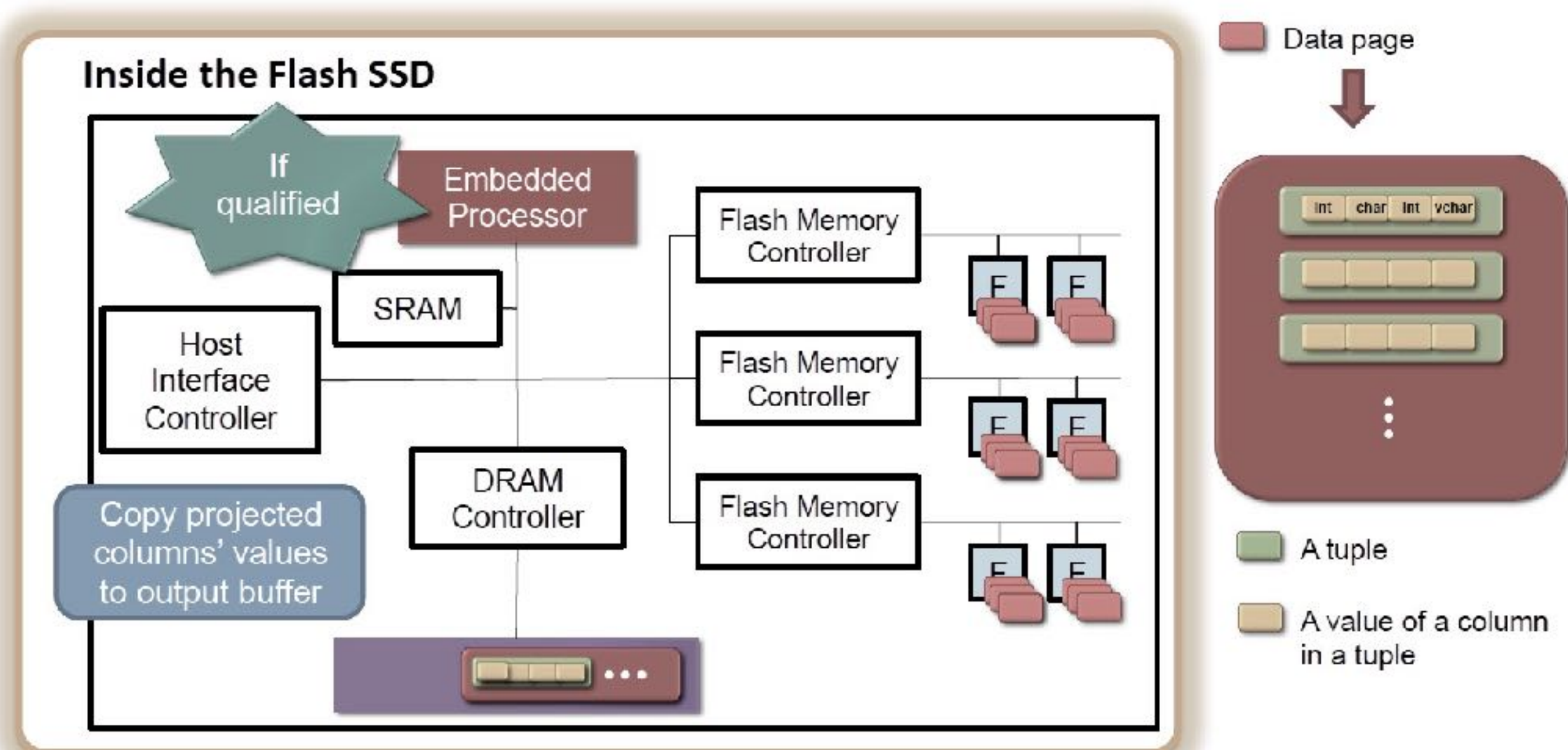
SQL
Server

- Modified the query processor to hard-code selected Smart SSD plans
- Implement Smart SSD executors

Communication Flow – e.g. Scan



Data Format in SSD



Data is read in tuples from the table

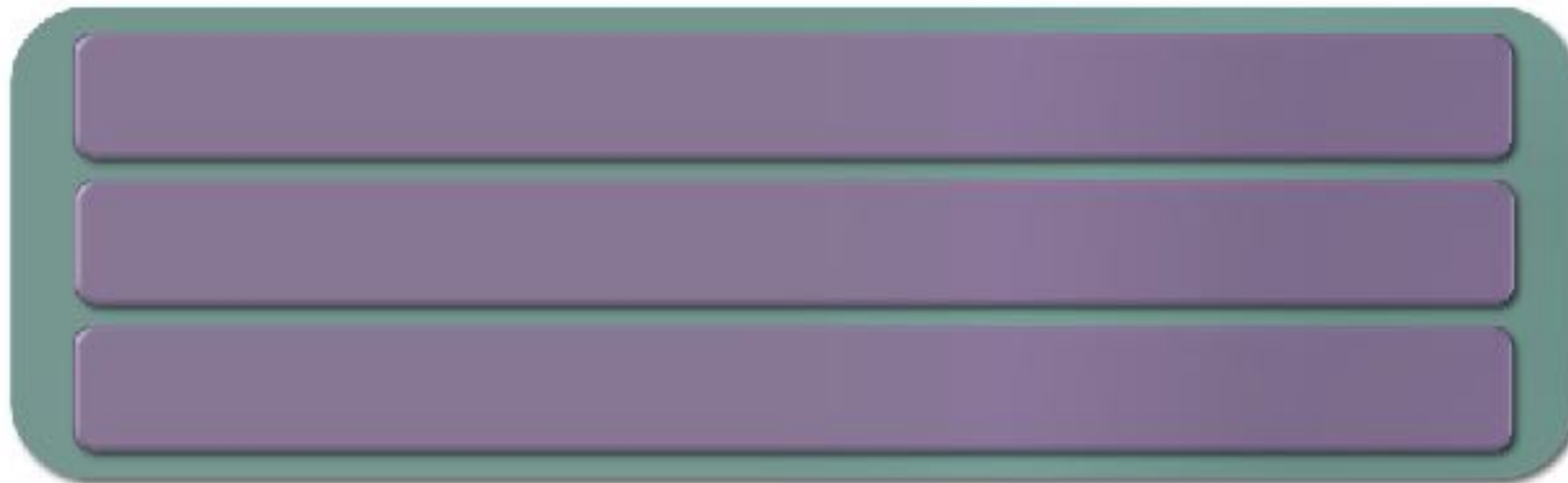
Data Format in SSD

- NSM layout



Tuple based organization

- PAX layout



Column based organization

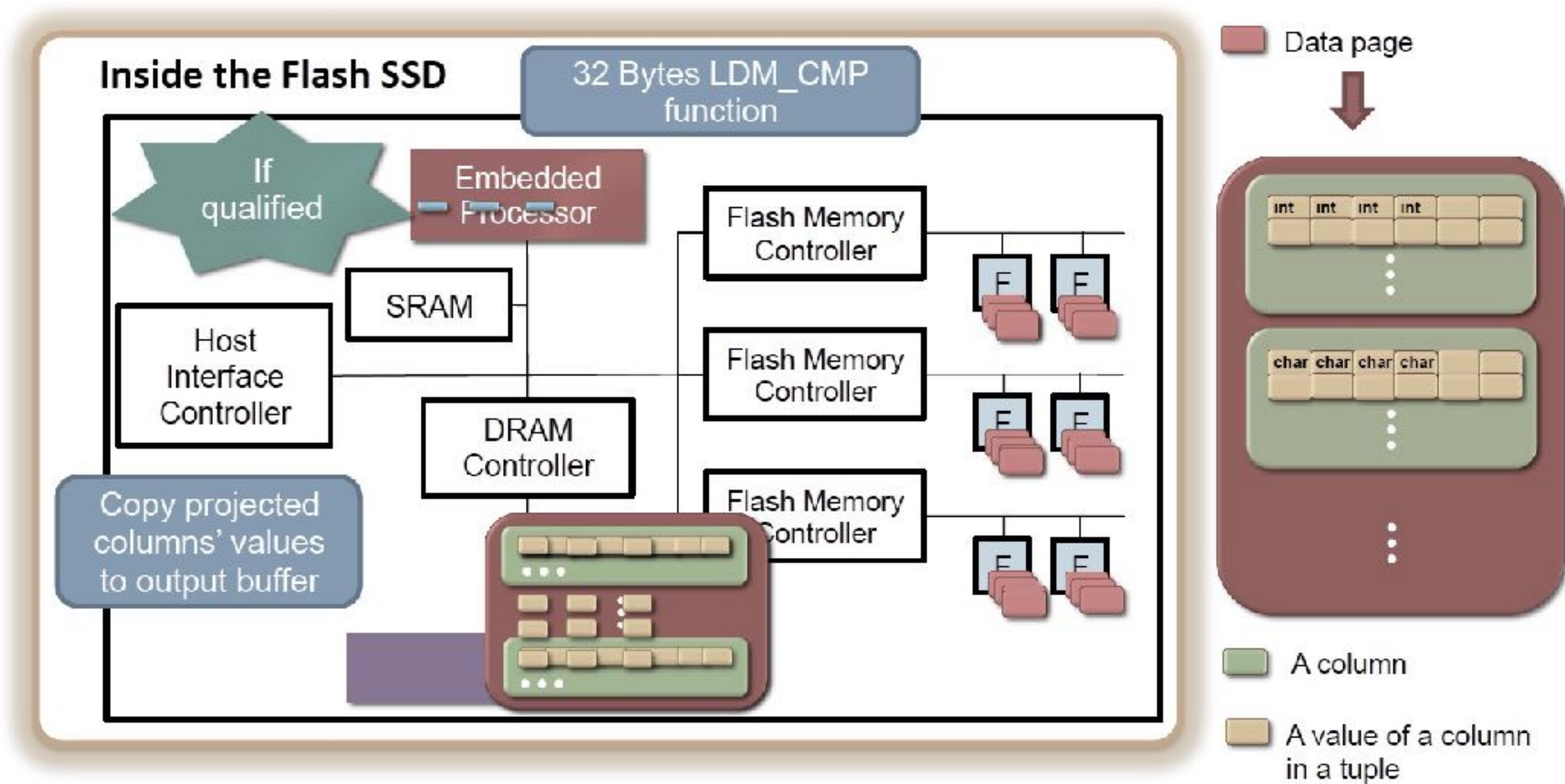
Data Page

Tuple

Column

Operations are performed on values in the same column

Column Format in SSD



Operations are performed on values in the same column

System Configuration



SQL server (Host machine)

64-bit Windows 7

32GB DRAM (24GB dedicated to the DBMS)

2.66 GHz Dual Quad Core
Intel Xeon5430
32KB L1 cache, 6MB L2 cache

Two 7.5 RPM SATA HDDs one for the OS and one for the Log

LSI four-port SATA/SAS 6Gbps HBA (host bus adapter)



Flash SSD

**Samsung 400GB
SAS SSD**

Two ARM Cores

16KB SRAM for each core

256KB DRAM for each core

1MB DRAM output buffer



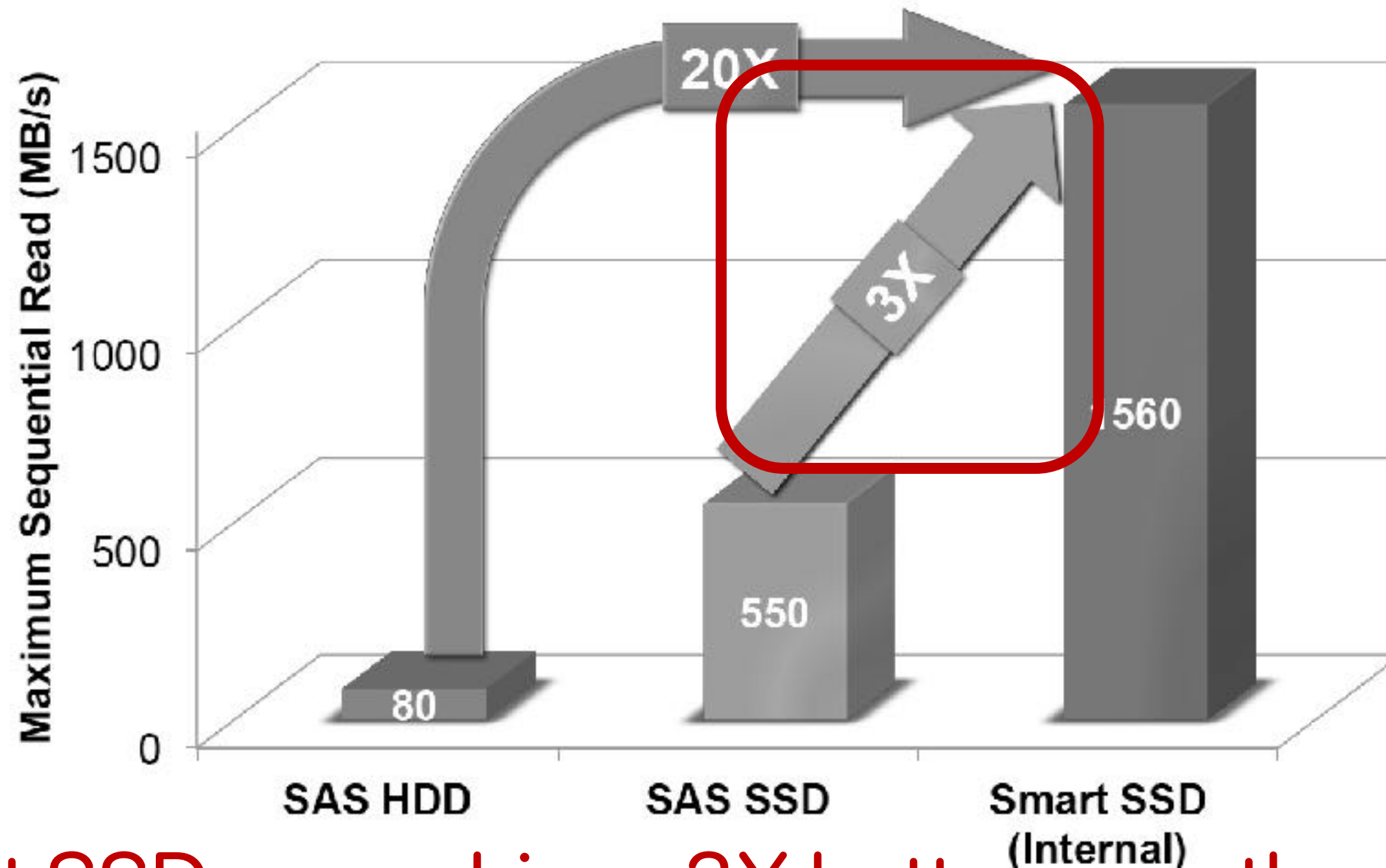
HDD

**HP 146GB 10K
RPM SAS HDD**

SSD cores are lightweight, DRAM capacity is small

Upper Bound on Expected Performance Gains

Speed of a Simple Sequential Scan Program



Smart SSD can achieve 3X better scan throughput than normal SSD

Evaluated Data Sets

Each table has a number of integer-typed columns

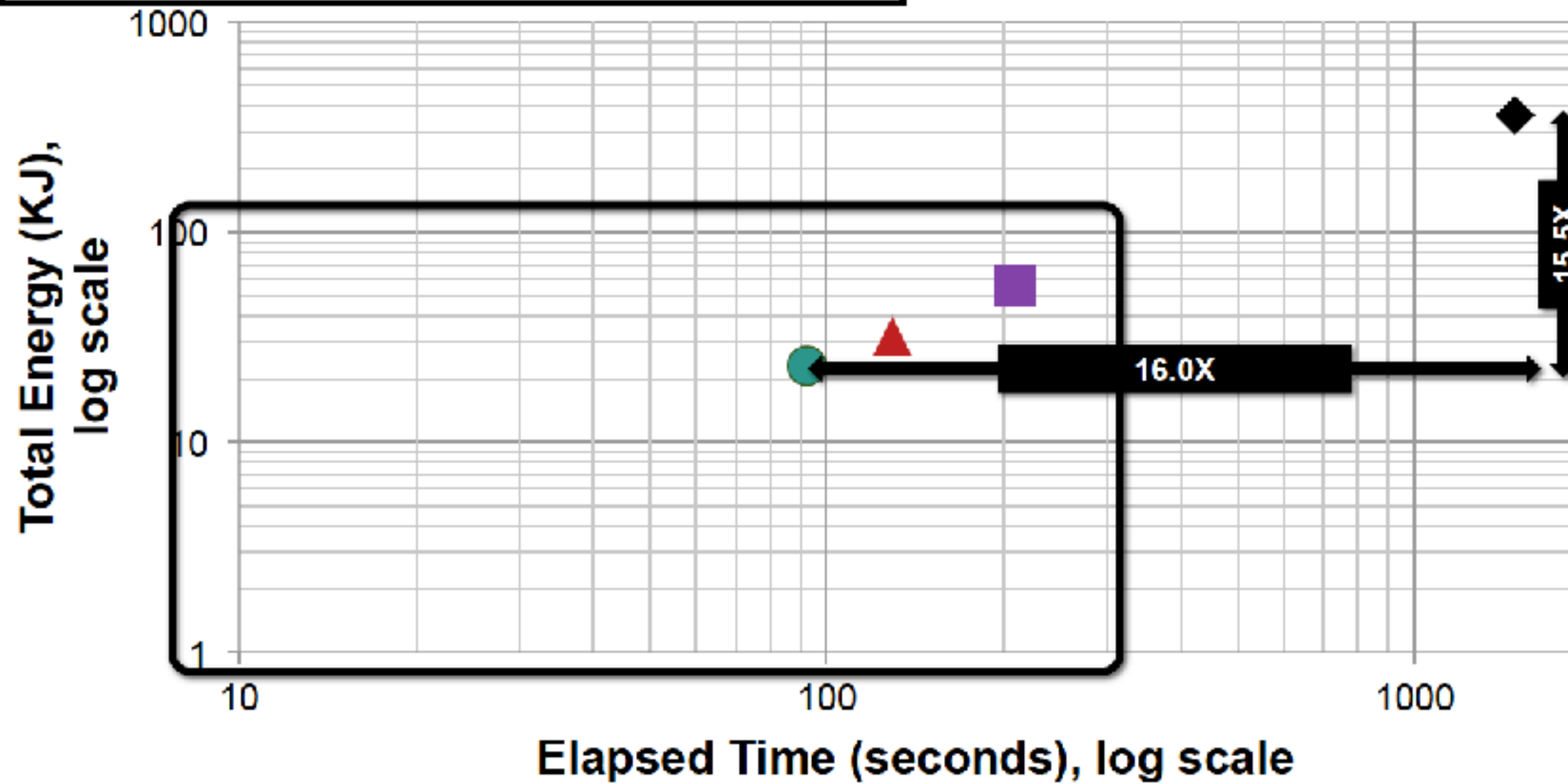
Table	# Columns	Table Size (GB)	# tuples (millions)	# tuples/ page
Synthetic 4	4	10	400	323
Synthetic16	16	30	400	109
Synthetic64	64	110	400	29

Synthetic64 has the largest data size

10% Selection Query

```
SELECT COLUMN_2  
FROM Synthetic64  
WHERE COLUMN_1 < [VALUE]
```

- ◆ SAS HDD
- SAS SSD
- ▲ Smart SSD (NSM)
- Smart SSD (PAX)

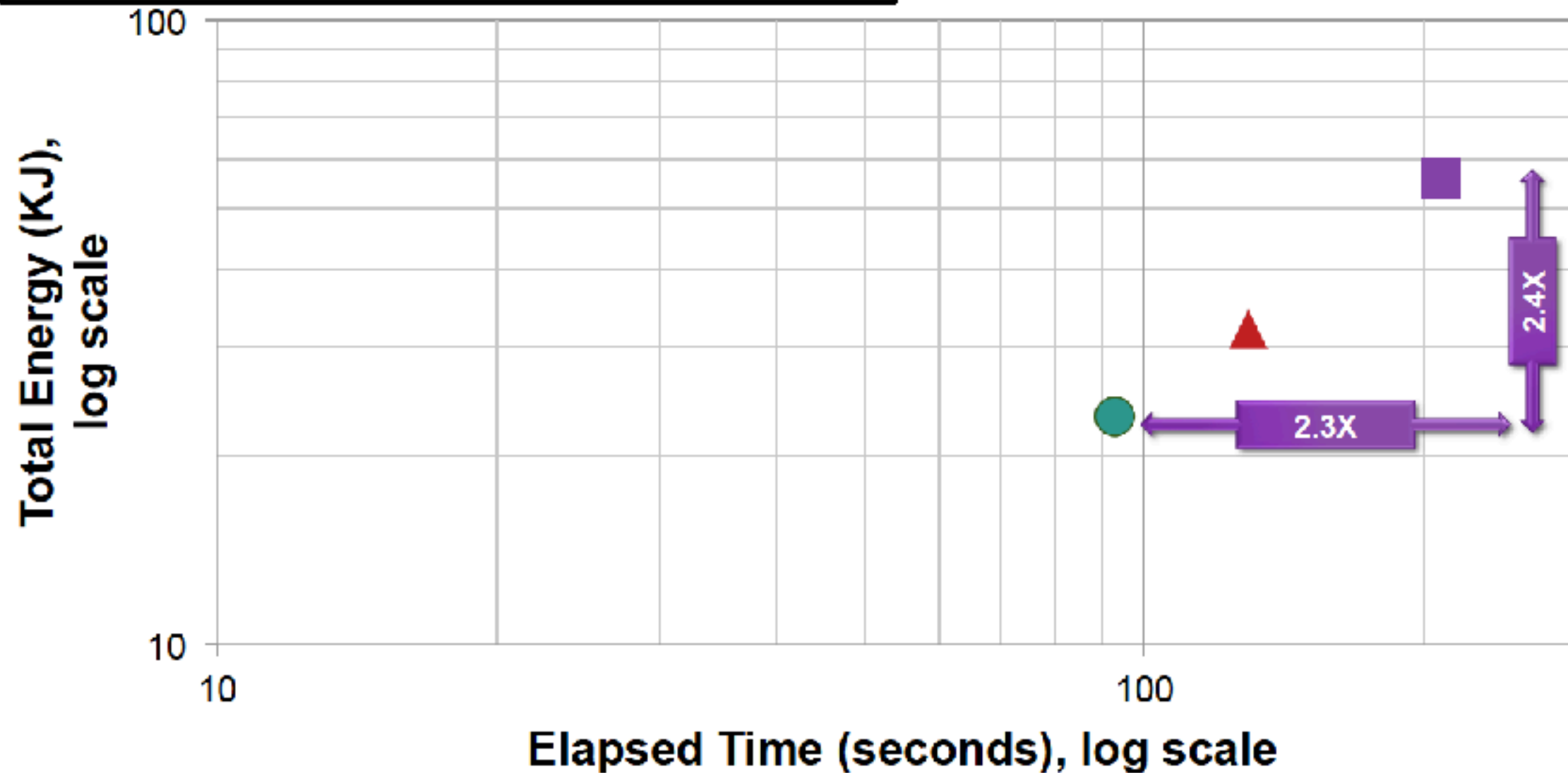


16X better energy and performance than disk

10% Selection Query

```
SELECT  COLUMN_2  
FROM    Synthetic64  
WHERE   COLUMN_1 < [VALUE]
```

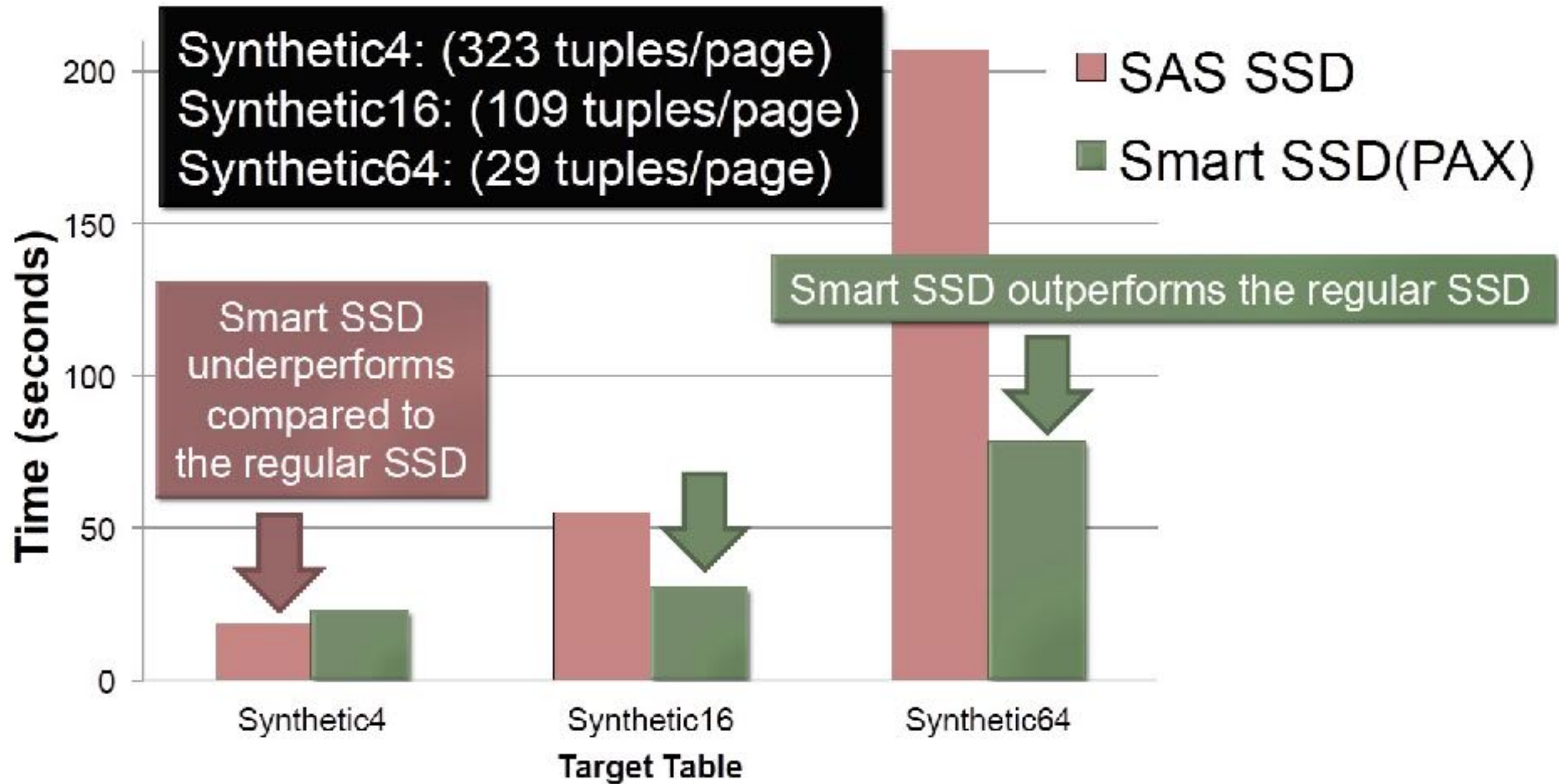
- ◆ SAS HDD
- SAS SSD
- ▲ Smart SSD (NSM)
- Smart SSD (PAX)



2.5X better energy and performance than SSD

Effect of the # of Tuples on a Page

```
SELECT COLUMN_2, FROM [Synthetic_table] WHERE COLUMN_1 < [VALUE]
```



Benefit depends on the dataset size

**Under what situation can
SmartSSDs become the bottleneck?**

Is SmartSSD always a good idea?

Recap: The “Winning Formula” of Near-Data Processing

Conventional Model



NDP Model



- The NDP computation can help offload computation
- The NDP computation can help reduce data volume
- The NDP computation can facilitate data preprocessing
- The NDP computation itself cannot be too slow

Recap: The "Winning Formula" of Near-Data Processing

Conventional Model



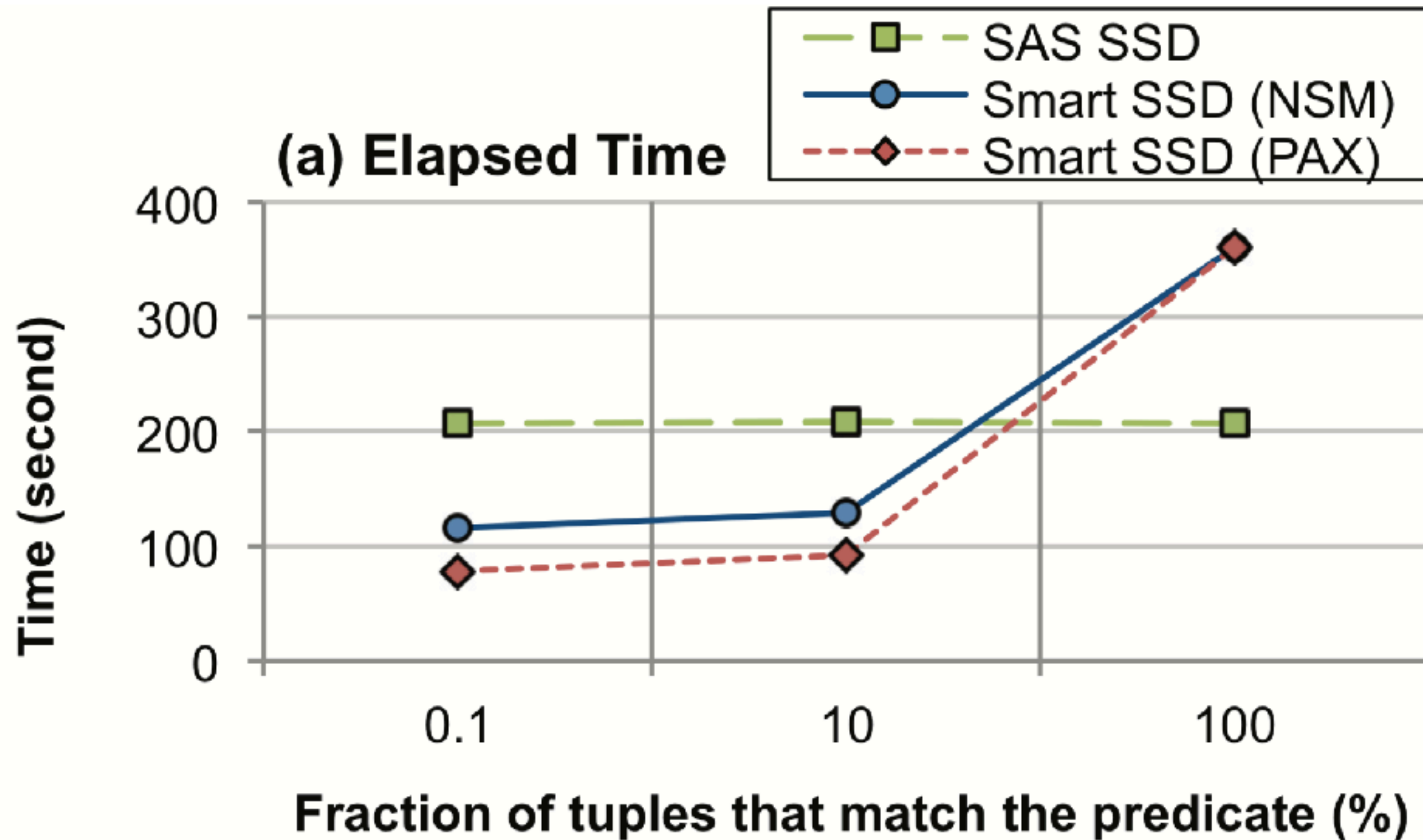
NDP Model



$$\frac{DataVolume_{raw}}{Bandwidth_{device}} + \frac{DataVolume_{raw} \times ComputationIntensity_{NDP}}{ComputationThroughput_{device}} + \frac{DataVolume_{afterNDP}}{Bandwidth_{device2host}} + \frac{DataVolume_{afterNDP} \times ComputationIntensity_{afterNDP}}{ComputationThroughput}$$

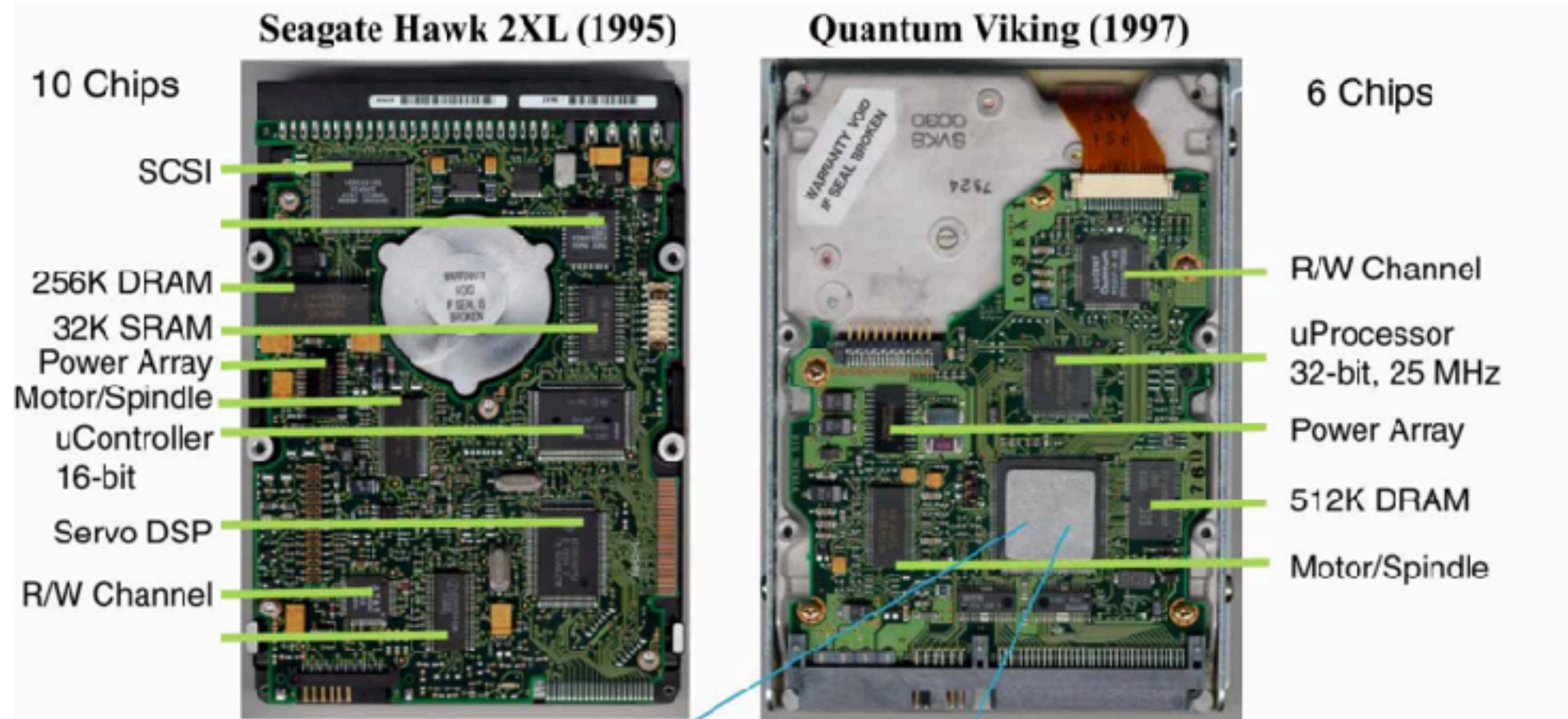
$$< = \frac{DataVolume_{raw}}{Bandwidth_{device2host}} + \frac{DataVolume_{raw} \times ComputationIntensity}{ComputationThroughput}$$

SSD becomes a bottleneck



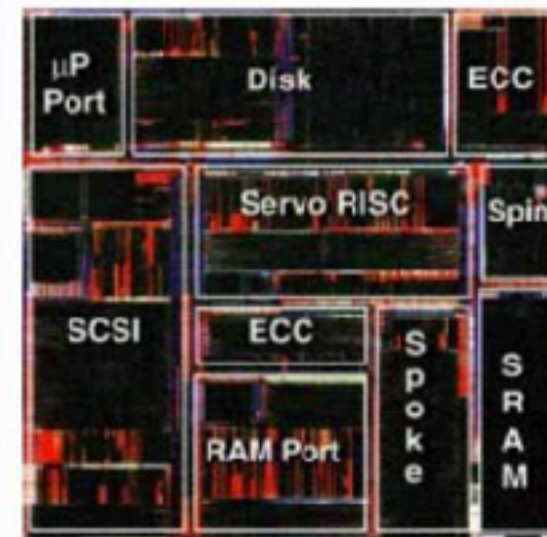
Memory and compute in SSD is a bottleneck

Evolution of Disk Drive Electronics

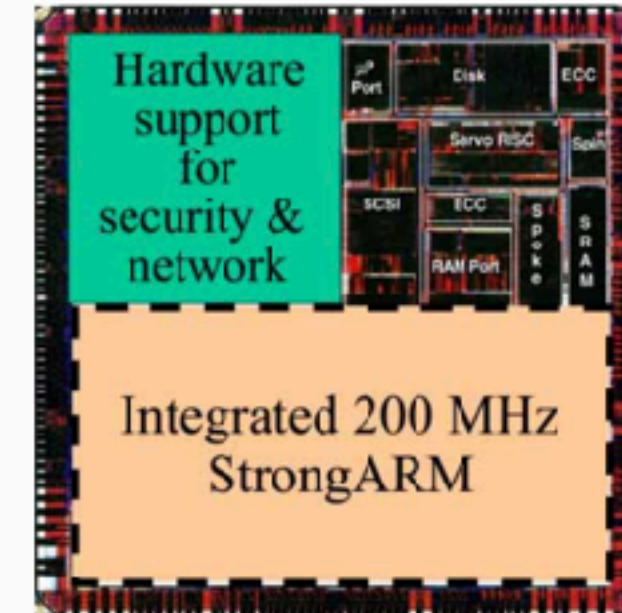


Integration

- reduces chip count
- improves reliability
- reduces cost
- future integration to processor on-chip
- but there must be at least *one* chip



Trident ASIC

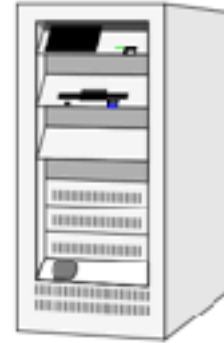


Future Generation ASIC

Opportunity

TPC-D 300 GB Benchmark, Decision Support System

Database Server



Digital AlphaServer 8400

- 12 x 612 MHz 21164
- 8 GB memory
- 3 64-bit PCI busses
- 29 FWD SCSI controllers

= 7,344 total MHz

3 x 266 = 798 MB/s

29 x 40 = 1,160 MB/s

Storage

- 520 rz29 disks
- 4.3 GB each
- 2.2 TB total

**= 104,000 total MHz
(with 200 MHz drive chips)**

**= 5,200 total MB/s
(at 10 MB/s per disk)**



Why “active disks” did not work out?

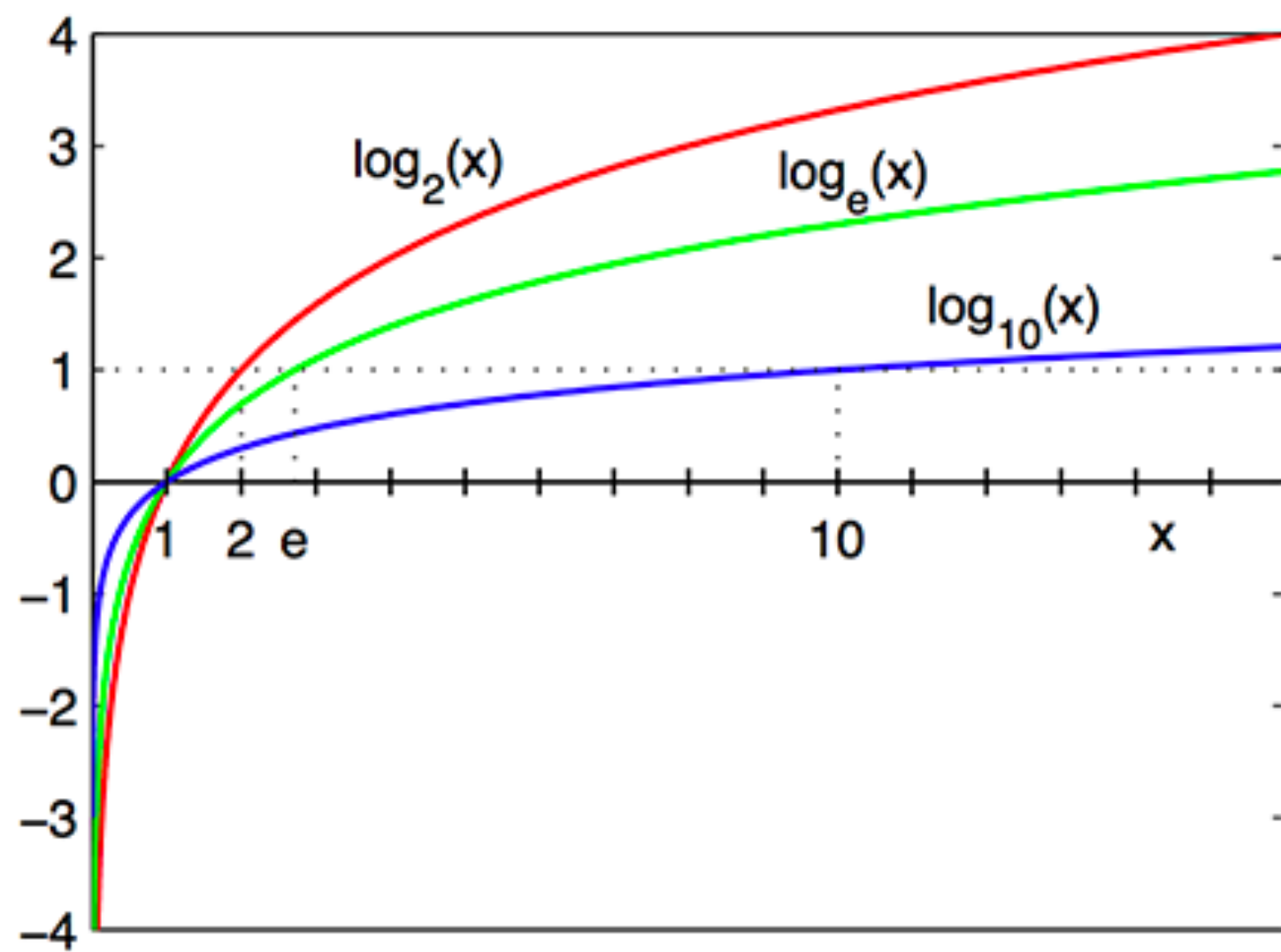
Why isn't ActiveDisk working?

Why Active Disks did not work out?

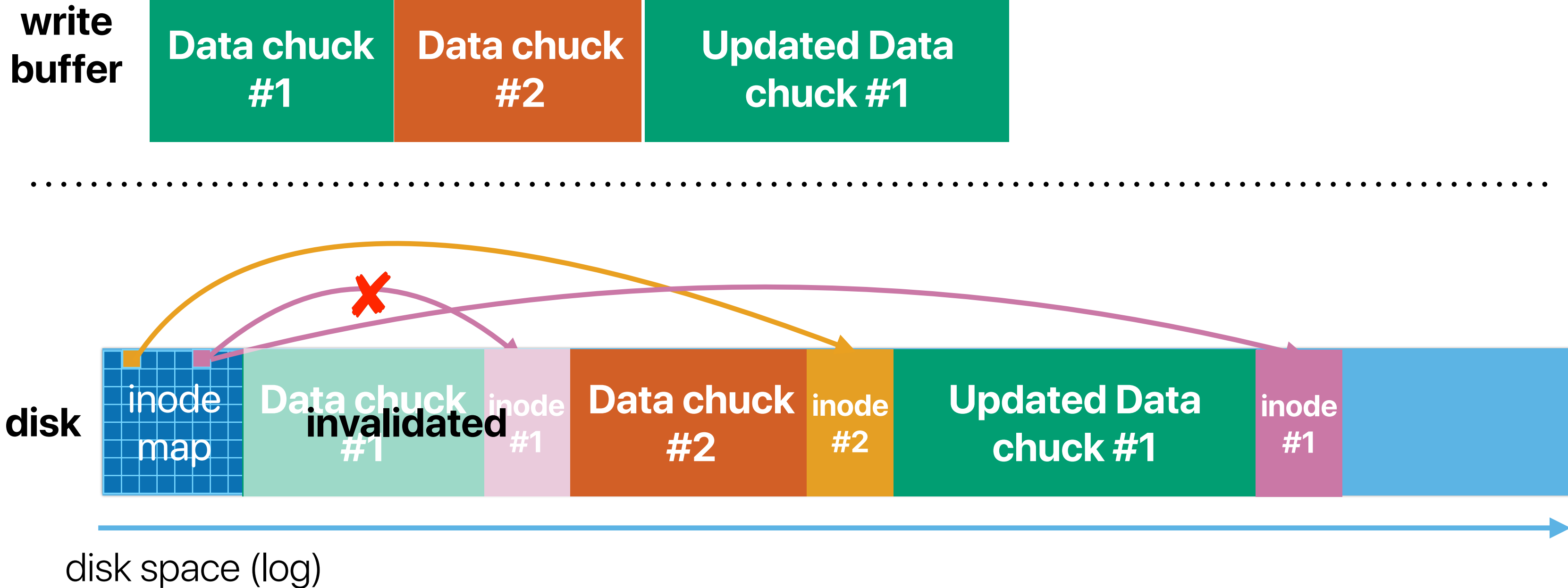
- Computation still dominates at that era
- Magnetic disks do not really have too much to do with “internal bandwidth”

Don't stack your log on my log

**Jingpei Yang, Ned Plasson, Greg Gillis, Nisha Talagala, and
Swaminathan Sundararaman
SanDisk Corporation**



LFS in motion

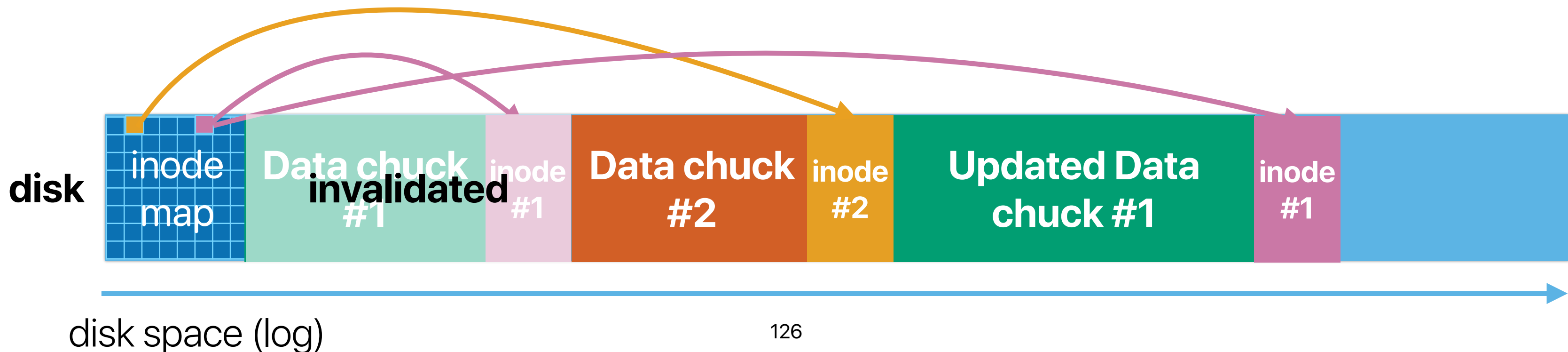


What's the good things and bad things about "Logs"

Pros/Cons of Logs

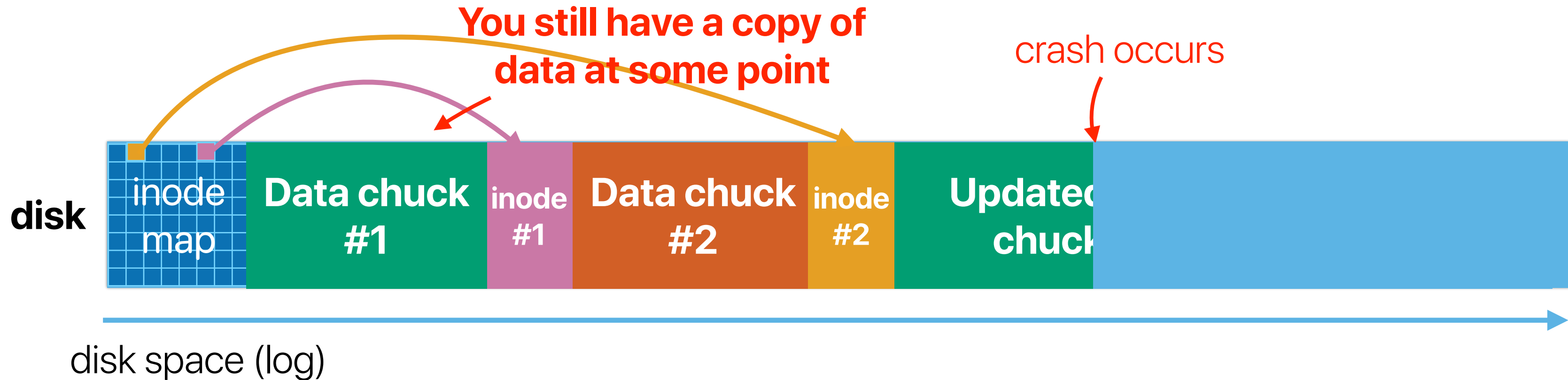
Log

- An append only data structure that records all changes
- Advantages of logs
 - Better performance — always sequential accesses
 - Faster writes — you just need to append without sanitize existing data first
 - Ease of recovery — you can find previous changes within the log
- Disadvantage of logs — you will need to explicit perform garbage collections to reclaim spaces

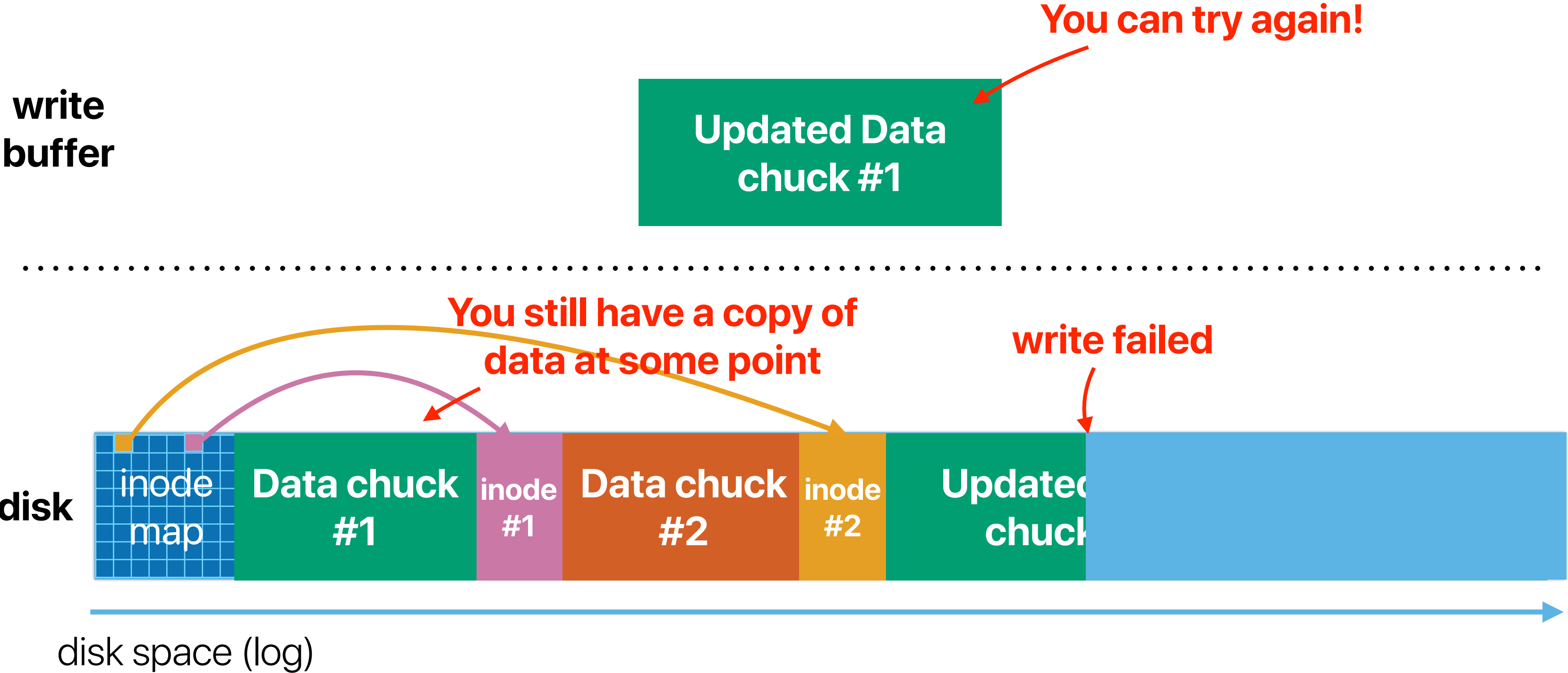


LFS v.s. crash

write
buffer

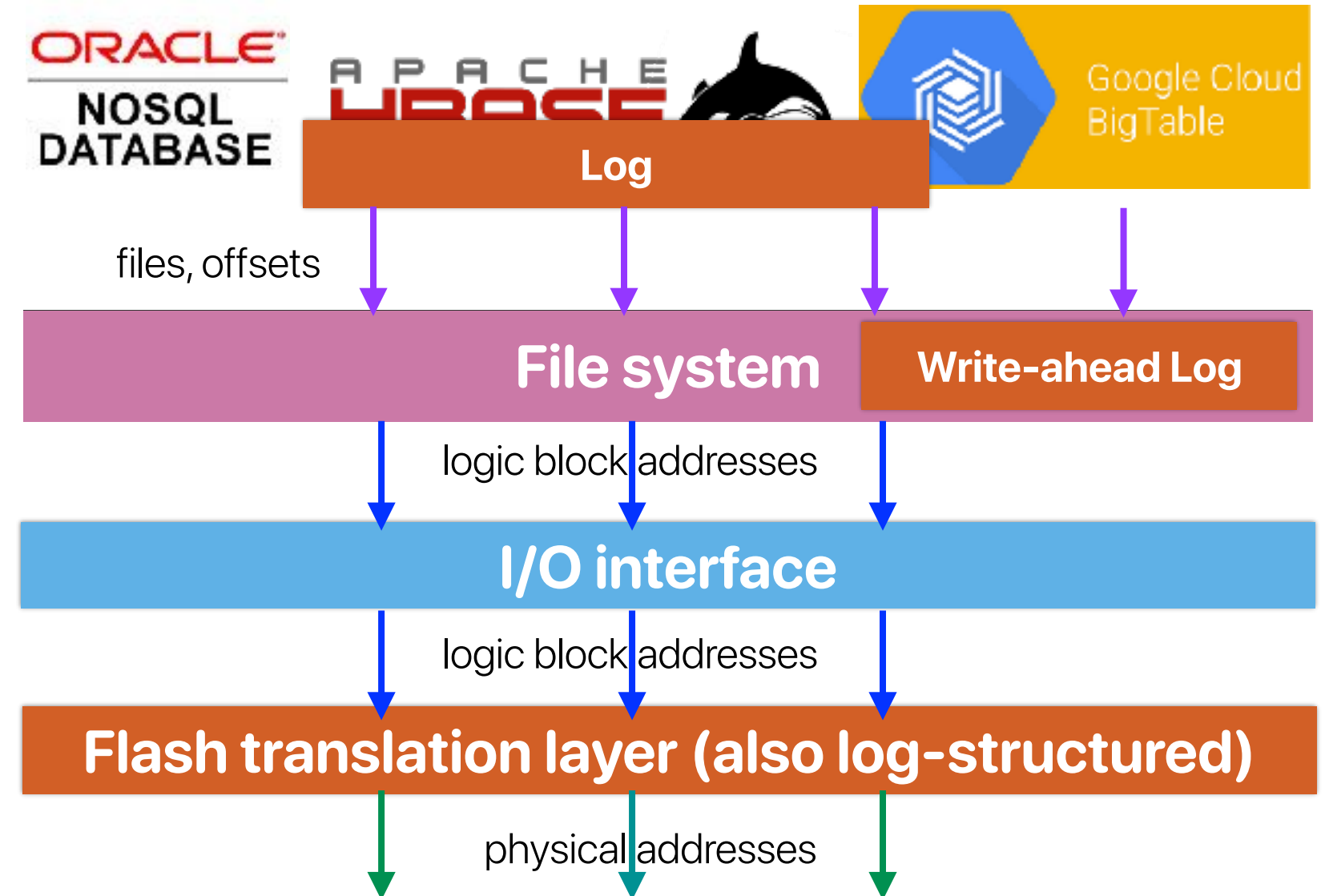


LFS v.s. write failed

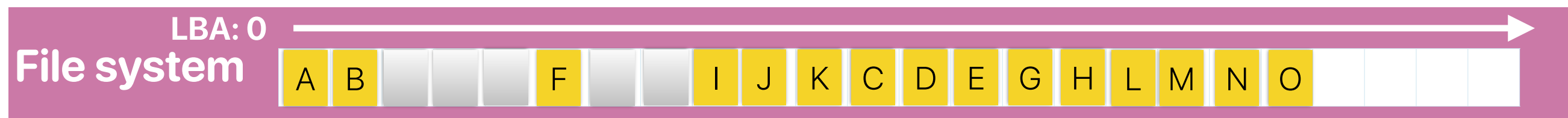


Why should we care about this paper?

- Log is everywhere
 - Application: database
 - File system
 - Flash-based SSDs
- They can interfere with each other!
- An issue with software engineering nowadays



For example, garbage collection

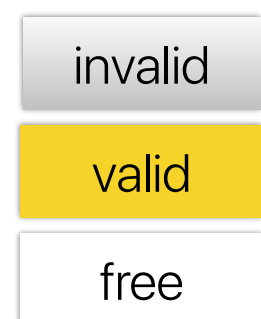
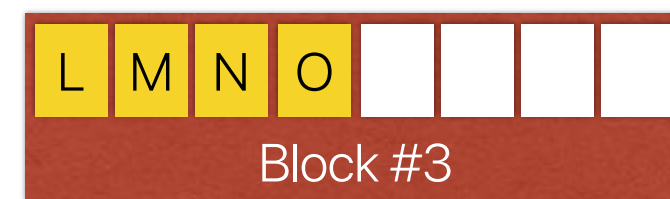
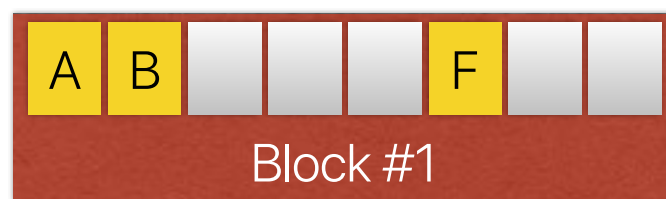


logic block addresses

I/O interface

logic block addresses

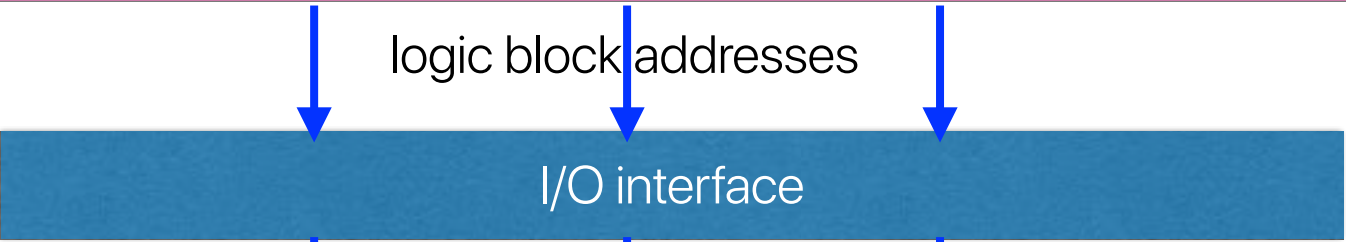
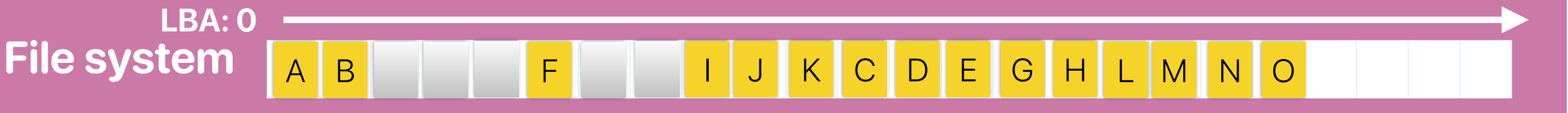
SSD



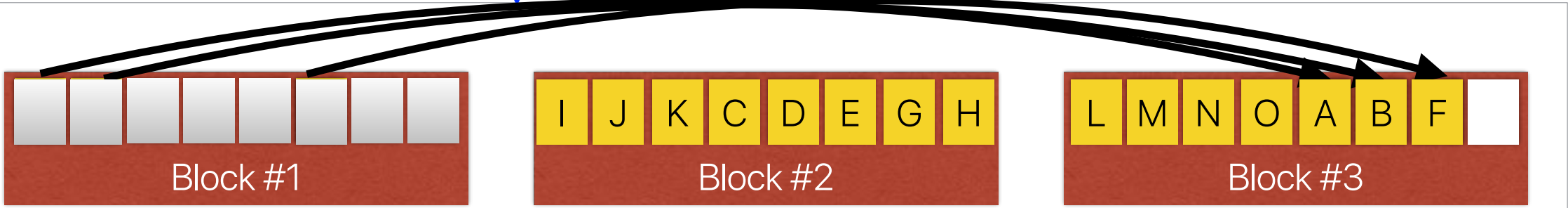
FTL mapping table

LBA	block #	page #
0	1	0
1	1	1
2	-	-
3	-	-
4	-	-
5	1	5
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	-	-
21	-	-
22	-	-
23	-	-

Now, SSD wants to reclaim a block



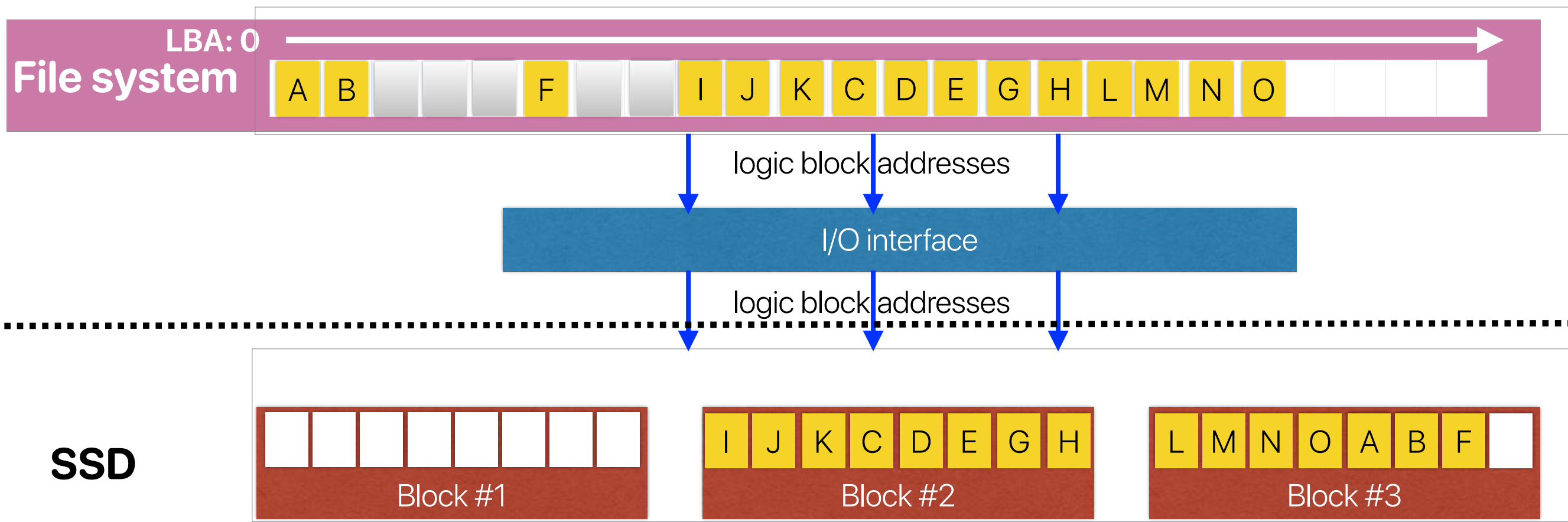
SSD



FTL mapping table

LBA	block #	page #
0	3	4
1	3	5
2	-	-
3	-	-
4	-	-
5	3	6
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	-	-
21	-	-
22	-	-
23	-	-

Garbage collection on the SSD done!



FTL mapping table

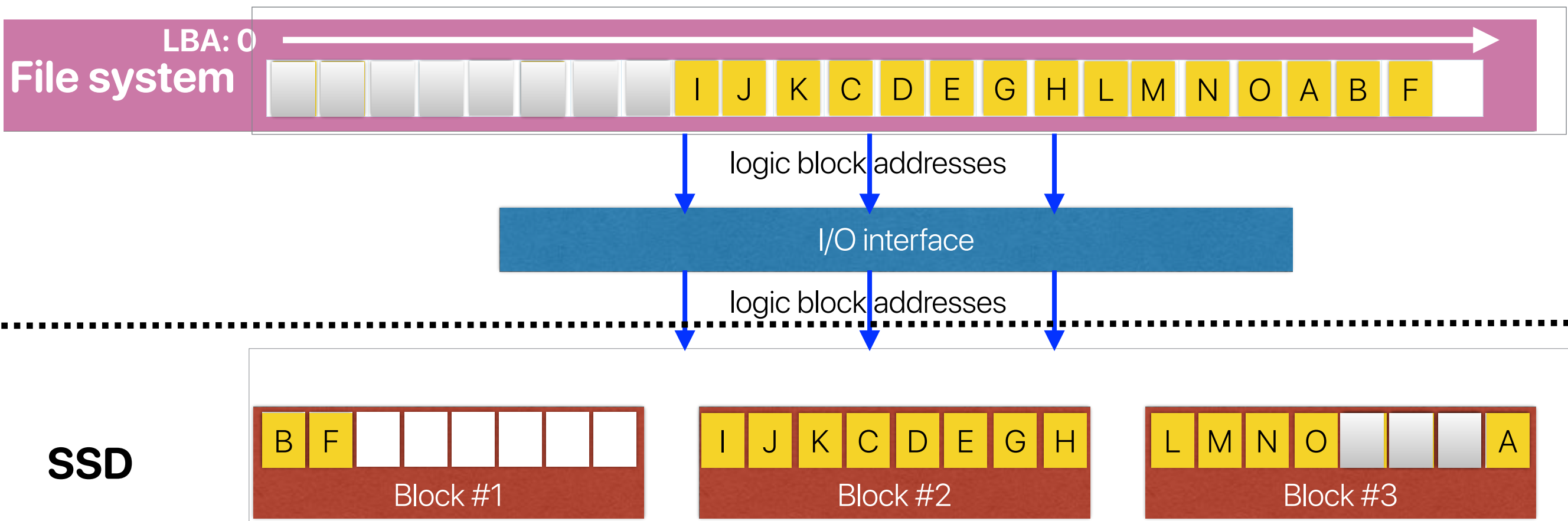
LBA	block #	page #
0	3	4
1	3	5
2	-	-
3	-	-
4	-	-
5	3	6
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	-	-
21	-	-
22	-	-
23	-	-

invalid

valid

free

What will happen if the FS wants to perform GC?



FTL mapping table		
LBA	block #	page #
0	-	-
1	-	-
2	-	-
3	-	-
4	-	-
5	-	-
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	3	7
21	1	0
22	1	1
23	-	-

We could have avoided writing the stale A, B, F if they are coordinated!



All problems in computer science can be solved by another level of
indirection

–David Wheeler

...except for the problem of too many layers of indirection.

Roog^{le} Final Project Presentations

- Make an appointment on **3/12** through the Google Calendar **ASAP**
- 12 minute presentation with 3 minute Q & A
- Why & what & how!!! — considering you're giving a presentation at Apple's keynote
 - 5-minute why & what — remind everyone what's your project about
 - 7-minute result — what lessons have you learned
 - Experimental result? Insights from the result?
 - Or why can't you get the experimental result?
 - Or why can't you get the result as you expected?
- Please reference this article to make a good presentation <https://cseweb.ucsd.edu/~swanson/GivingTalks.html>

Electrical Computer Science Engineering

277

つくづく

